



C Shell [200]

General Requirements

- Break the project down into multiple `.c` and `.h` files based on functionality. Monolithic code will be penalized.
- Use only C POSIX library headers and functions. You can use [this](#) for reference.
- Compile with the following flags for POSIX compliance:

```
gcc -std=c23 \  
-D_POSIX_C_SOURCE=200809L \  
-D_XOPEN_SOURCE=700 \  
-Wall -Wextra -Werror \  
-Wno-unused-parameter \  
-fno-asm \  
your_file.c
```

Final build must compile with `make all` in the shell directory, producing `shell.out` in that same directory.

A reference `Makefile` is provided for your convenience:

<> [Code](#) [Makefile](#)

References

These could be helpful for you to understand things required to implement this section.

- [POSIX.1-2017 spec](#)
- [man 2](#) pages for `fork`, `exec`, `wait`, `pipe`, `dup2`, `open`. These cover most of what you'll need.
- Use the source of actual `bash` (will be helpful to understand how to do certain parts of the mini-project)
- Use the source of [BusyBox Shell](#) for further reference since it is a smaller implementation with more understandable code.

i'm not really sure what they mean by "monolithic code will be penalized". if i have one `exec.c` that handles `fork/exec`, redirection and pipes, is that already considered too much or is the point mainly that i shouldn't put the entire shell including parsing and all the builtins into one file?



Yeah bro, **your interpretation is basically right.**

"Monolithic code will be penalized" usually means they don't want something like:

```
shell.c  
├─ main()  
├─ command parsing  
├─ tokenization  
├─ builtins (cd, exit, ...)  
├─ fork/exec  
├─ pipes  
├─ redirection  
├─ history  
└─ everything else
```

all crammed into one huge file.

Having an `exec.c` that handles **fork/exec + redirection + pipes** is actually pretty reasonable because those things are all part of the **execution layer**.

I'd structure it roughly like:

```

main.c
  ↓
parser.c / parser.h
  ↓
executor.c / executor.h
  ├── simple command execution
  ├── fork/exec
  ├── pipelines
  └── redirection
    ↓
builtins.c / builtins.h
  ├── cd
  ├── exit
  └── export, etc.

```

And potentially:

```

lexer.c / lexer.h      → tokenize input
parser.c / parser.h    → turn tokens into commands/pipelines
executor.c / executor.h → execute parsed commands
builtins.c / builtins.h → builtin commands
utils.c / utils.h      → shared helper functions
main.c                 → shell loop / coordination

```

So is `exec.c` too much?

Not inherently. For example, this is perfectly defensible:

```

// exec.c

void execute_command(...);
void execute_pipeline(...);
void setup_redirection(...);
void execute_external(...);

```

Because `execute_pipeline()` naturally needs `fork()`, `pipe()`, `dup2()`, and `exec*()`, while redirection is also part of preparing a command for execution.

I'd only start splitting it further if `exec.c` becomes something like **800-1000 lines** containing completely unrelated responsibilities.

For example, this would be a bad sign:

```

exec.c
  ├── parse_command()
  ├── tokenize()
  ├── handle_cd()
  ├── handle_exit()
  ├── execute_pipeline()
  ├── setup_redirection()
  └── manage_history()

```

```
├─ variable_expansion()
├─ signal_handling()
└─ 50 utility functions
```

At that point, the filename is basically just hiding a monolith.

What I think the assignment is actually asking for

The important phrase is:

“Break the project down into multiple `.c` and `.h` files based on functionality.”

So they're probably evaluating whether you've identified **logical modules**, not whether every 30-line piece gets its own file.

Something like this would demonstrate that nicely:

```
shell/
├─ Makefile
├─ main.c
├─ shell.c
├─ shell.h
├─ lexer.c
├─ lexer.h
├─ parser.c
├─ parser.h
├─ executor.c
├─ executor.h
├─ builtins.c
├─ builtins.h
├─ redirection.c
├─ redirection.h
└─ utils.c
    utils.h
```

You don't *need* exactly this structure, though. In fact, I'd avoid blindly creating 15 files just to satisfy the requirement.

Rule of thumb: each `.c` file should have a clear sentence describing its responsibility.

```
parser.c parses commands.
builtins.c implements builtins.
executor.c executes commands/pipelines.
redirection.c handles file descriptor redirection.
```

If you can say that naturally for each file, you're probably doing it right.

And yes: `exec.c` **containing fork/exec + pipes + redirection is much more likely to be considered good modularization than “the entire shell is in `shell.c`.”**



why are they giving us both `\_POSIX\_C\_SOURCE=200809L` and `\_XOPEN\_SOURCE=700`? are these actually changing which functions are available to the compiler, or are they just telling the compiler that we're writing POSIX code?



Yep — **they actually change what gets exposed by the system headers**. They're not merely comments saying “hey, this is POSIX code.”

Think of them as **feature-selection macros** for libc headers.

What happens without them?

When you do:

```
#include <unistd.h>
#include <sys/wait.h>
```

the headers don't necessarily expose every function/type/constant they know about. They use feature-test macros to decide what declarations to make visible.

For example, conceptually:

```
#ifdef _POSIX_C_SOURCE
    /* expose POSIX functions */
#endif

#ifdef _XOPEN_SOURCE
    /* expose X/Open / SUS interfaces */
#endif
```

So defining:

```
-D_POSIX_C_SOURCE=200809L
```

before compilation tells the headers:

“Give me the interfaces specified by POSIX.1-2008.”

And:

```
-D_XOPEN_SOURCE=700
```

roughly says:

“Give me the interfaces corresponding to SUS/X/Open 7.”

So why both?

Because they're **related but not identical standards/features**.

`_POSIX_C_SOURCE=200809L` selects the POSIX.1-2008 feature set.

`_XOPEN_SOURCE=700` selects the X/Open/SUS version 7 feature set, which is associated with POSIX.1-2008 plus additional interfaces.

In practice, on systems like Linux/glibc, defining `_XOPEN_SOURCE=700` can itself imply a corresponding POSIX feature level, so having both is often **redundant or explicit rather than fundamentally necessary**.

The assignment is probably being deliberately explicit:

```
gcc -std=c23 \
    -D_POSIX_C_SOURCE=200809L \
    -D_XOPEN_SOURCE=700 \
    ...
```

meaning:

Compile this as C23, and expose the standardized POSIX/XSI interfaces we're expecting you to use.

A concrete example

Suppose you write:

```
#include <unistd.h>

int main(void)
{
    char *args[] = {"ls", NULL};
    execvp("ls", args);
}
```

`execvp()` is a POSIX function.

The feature-test macros affect whether the appropriate declaration is exposed by the headers. Without the right feature-test settings, you can sometimes get:

```
warning: implicit declaration of function '...'
```

even though the function exists in the system's libc.

So these flags don't **add the function to libc** or change how `execvp()` works.

They're basically controlling the **API surface that the headers expose to your program**.

One subtle thing

This:

```
-D_POSIX_C_SOURCE=200809L
```

is equivalent to putting this **before your headers**:

```
#define _POSIX_C_SOURCE 200809L
```

And similarly:

```
-D_XOPEN_SOURCE=700
```

is like:

```
#define _XOPEN_SOURCE 700
```

The assignment puts them on the compiler command line so **every source file gets the same feature-test configuration**.

So your mental model should be:

Not “this tells the compiler I’m writing POSIX.”

More like “this tells the system headers which standardized APIs they’re allowed to expose.”

And this is particularly relevant for your shell assignment because things like `fork()`, `execvp()`, `pipe()`, `dup2()`, `waitpid()`, `open()`, etc. are exactly the sort of POSIX interfaces they’re trying to make you use rather than non-portable GNU/Linux extensions.



if i add a new file like `reveal.c`, what exactly do i need to change in the Makefile so that `make all` knows to compile it? i'm trying to avoid a situation where the source exists but the final executable is still using an old object file.



Exactly — this is one of the things the Makefile is supposed to handle for you.

The key idea is:

Every `.c` file that contributes to `shell.out` must eventually become an `.o` file, and all those `.o` files must be listed as dependencies of the final executable.

For example, suppose your Makefile currently has:

```
CC = gcc
CFLAGS = -std=c23 -D_POSIX_C_SOURCE=200809L -D_XOPEN_SOURCE=700 -Wall -Wextra -Werror -Wno-unused-parameter -fno-asm

OBJ = main.o parser.o executor.o builtins.o

all: shell.out

shell.out: $(OBJ)
    $(CC) $(CFLAGS) -o shell.out $(OBJ)

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

clean:
    rm -f $(OBJ) shell.out
```

Now you create:

```
reveal.c
reveal.h
```

If `reveal.c` contains your implementation, you need to add `reveal.o` to `OBJ`:

```
OBJ = main.o parser.o executor.o builtins.o reveal.o
```

That's the important change.

Then:

```
make all
```

will see:

```
shell.out
  ↓ depends on
main.o parser.o executor.o builtins.o reveal.o
                        ↓
                        reveal.c
```

and automatically compile:

```
gcc ... -c reveal.c -o reveal.o
```

and then link everything:

```
gcc ... -o shell.out main.o parser.o executor.o builtins.o reveal.o
```

Why this prevents the "old object file" problem

`make` checks timestamps.

Suppose you have:

```
reveal.c      11:00
reveal.o      10:00
shell.out     09:00
```

Since `reveal.c` is newer than `reveal.o`, Make knows:

`reveal.o` is stale → recompile it.

Then because `reveal.o` becomes newer than `shell.out`, Make knows:

`shell.out` is stale → relink it.

So you **don't need to manually delete old `.o` files every time.**

One thing I'd be careful about

If you do this:

```
OBJ = main.o parser.o executor.o builtins.o
```

and forget to add `reveal.o`, then `reveal.c` basically **doesn't exist as far as the final build is concerned.**

You could happily have:

```
reveal.c
reveal.o
```

sitting in the directory, but if `reveal.o` isn't part of the `shell.out` dependency list, it won't be linked into the executable.

That's probably exactly the situation you're worried about.

Even nicer: automatically include all `.c` files

You *could* avoid manually updating `OBJ` every time by doing:

```
SRC = $(wildcard *.c)
OBJ = $(SRC:.c=.o)
```

Then if you add:

```
reveal.c
```

Make automatically gets:

```
SRC = main.c parser.c executor.c builtins.c reveal.c
```

and:

```
OBJ = main.o parser.o executor.o builtins.o reveal.o
```

So you don't have to touch the Makefile.

However, for an assignment where they explicitly want a clean modular structure, I'd personally prefer explicitly listing your source/object files. It's more transparent and makes it obvious which files constitute the project.

And headers are a slightly different story

If you have:

```
reveal.c
reveal.h
```

and later modify `reveal.h`, you also want `reveal.o` to be rebuilt.

The basic rule:

```
reveal.o: reveal.c reveal.h
```

would accomplish that.

For a small project, you can list those dependencies manually. Or you can have GCC generate dependency information automatically with `-MMD -MP`, which is a nice improvement once the project gets bigger.

But for your current question, the main thing to remember is:

New `.c` file → its `.o` must be included in the executable's dependency/object list.

If your Makefile uses `wildcard`, that happens automatically.



the assignment mentions ``fork``, ``exec``, ``wait``, ``pipe``, ``dup2``, and ``open`` as references. are these basically the core system calls i'll need for the later redirection and piping sections, or are some of them also relevant to the basic command execution part?



Yeah — **some of them are fundamental to basic command execution**, while the others become important when you add redirection/pipes.

A good way to think about the progression is:

1. Basic external command execution

For something like:

```
$ ls -l
```

you mainly need:

```
fork()
  ↓
child process
  ↓
execvp("ls", ...)
  ↓
parent waits with wait()/waitpid()
```

So the core three are:

- `fork()` — create a child process
- `exec*()` — replace the child with the requested program
- `wait()` / `waitpid()` — parent waits for the child to finish

So yes, `fork`, `exec`, and `wait` **are already relevant in the very first basic execution section.**

2. Redirection

For:

```
$ ls > output.txt
```

you add:

```
open("output.txt", ...)
  ↓
file fd
  ↓
```

```
dup2(file_fd, STDOUT_FILENO)
    ↓
execvp("ls", ...)
```

Here:

- `open()` → opens/creates the file
- `dup2()` → makes stdout point to that file

The important thing is that you generally do the `dup2()` **in the child before** `exec()`.

Conceptually:

```
fork()
|
└─ child
    |
    ├─ open("output.txt")
    ├─ dup2(fd, STDOUT_FILENO)
    └─ execvp("ls", ...)
```

After `exec`, `ls` doesn't need to know anything about your shell's redirection logic. Its stdout is simply already connected to the file.

3. Pipes

For:

```
$ ls | grep txt
```

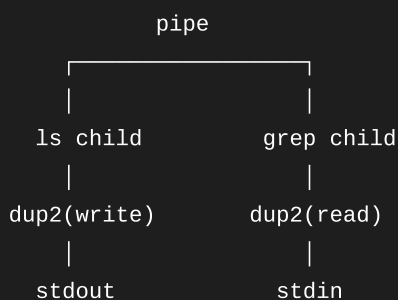
you introduce:

```
pipe()
```

which gives you two file descriptors:

```
pipefd[0] → read end
pipefd[1] → write end
```

Then you typically have **two children**:



So pipes involve:

- `pipe()` → create communication channel
- `fork()` → create processes
- `dup2()` → connect stdin/stdout to the pipe
- `exec()` → run the actual commands
- `wait()` / `waitpid()` → wait for the children

So the progression is basically

Function	Basic execution	Redirection	Pipes
<code>fork()</code>	✓	✓	✓
<code>exec*()</code>	✓	✓	✓
<code>wait()</code> / <code>waitpid()</code>	✓	✓	✓
<code>open()</code>	—	✓	sometimes
<code>dup2()</code>	—	✓	✓
<code>pipe()</code>	—	—	✓

So the assignment's reference list is actually a pretty good hint about the architecture they're expecting:

Basic shell:

```
fork → exec → wait
```

Add redirection:

```
fork → open → dup2 → exec → wait
```

Add pipes:

```
pipe → fork → dup2 → exec
    fork → dup2 → exec
    wait...
```

One subtle point: `exec` **isn't technically a system call itself in the same sense as** `fork` / `wait`. `execvp()`, `exec1()`, `execve()`, etc. are libc/POSIX interfaces that ultimately invoke the kernel's `execve` mechanism. But for your shell assignment, people commonly just call the whole family “exec system calls.”

And honestly, if you're implementing this incrementally, I'd do **basic** `fork + exec + wait` **first**, get `$ ls`, `$ echo hello`, etc. working, and *then* layer `open/dup2` and `pipe/dup2` on top. That makes the later pieces much easier to reason about.



if `parser.c` creates some command structure that `exec.c` needs, should i define that struct in `parser.h`, or would it be cleaner to have a separate header for the common command representation?



I'd lean toward a **separate header for the command representation** if the structure is genuinely the interface between parsing and execution.

For example:

```
main.c
parser.c / parser.h
executor.c / executor.h
command.h
```

Then:

```
// command.h

typedef struct {
    char **argv;
    char *input_file;
    char *output_file;
    int append;
} Command;
```

`parser.c` produces a `Command`, while `executor.c` consumes it:

```
// parser.h
#include "command.h"

Command *parse_command(...);
```

```
// executor.h
#include "command.h"

void execute_command(const Command *cmd);
```

This gives you a nice separation:



Why not just put it in `parser.h`?

You *can*, and for a small assignment it isn't wrong:

```
// parser.h
typedef struct {
    ...
} Command;

Command *parse_command(...);
```

But conceptually, `Command` isn't really **owned by the parser**. It's the data representation of a shell command that happens to be **created by the parser and used by the executor**.

If you put it in `parser.h`, then `executor.c` has to include a header called `parser.h` just to learn what a `Command` looks like:

```
#include "parser.h" // I need this because Command is here
```

That creates a slightly weird dependency:

executor depends on parser's interface, even though execution and parsing are separate modules.

With `command.h`:

```
#include "command.h"
```

the dependency says exactly what you mean:

executor needs the command representation.

I'd use this rule

If a type is:

internal to one module:

```
parser.c
  ↓
parser-private struct
```

keep it in `parser.c`.

If a type is:

shared between multiple modules:

```
parser → Command ← executor
```

put it in a common header such as `command.h`.

And this actually fits the assignment's **"multiple .c and .h files based on functionality"** requirement nicely. You're not splitting files arbitrarily; you're separating **interfaces and responsibilities**.

One caveat: don't go overboard and create `structs.h`, `types.h`, `common.h`, `constants.h`, etc. containing everything. A small `command.h` with the actual shared representation is much cleaner.



i'm trying not to make global variables everywhere, but things like the previous directory for hop- need to persist between commands. where would you normally keep that state in a small shell like this?



Yeah, this is exactly the kind of state where **a global variable feels tempting but isn't necessary**.

For a small shell, I'd make the shell itself own the persistent state.

For example:

```
typedef struct {
    char *previous_dir;
    int should_exit;
} ShellState;
```

Then your main loop has one `ShellState`:

```
int main(void)
{
    ShellState state = {
        .previous_dir = NULL,
        .should_exit = 0
    };

    while (!state.should_exit) {
        ...
        execute_command(&state, command);
        ...
    }

    free(state.previous_dir);
}
```

And your `hop / cd` builtin receives a pointer to that state:

```
void builtin_hop(ShellState *state, const Command *cmd)
{
    ...
}
```

So the lifetime looks like:

```
main()
|
| owns
↓
ShellState
|
```

```

├─ previous_dir
├─ should_exit
└─ whatever other persistent shell state you eventually need
    |
    ├─ builtin_hop(&state, ...)
    ├─ builtin_exit(&state, ...)
    └─ ...

```

Why this is nicer than a global

Instead of:

```
char *previous_dir;    // global 🤔
```

and having `hop.c`, `main.c`, etc. access it directly, you have:

```
ShellState state;
```

and explicitly pass it to things that need to modify shell state.

That makes the dependency very obvious:

```
builtin_hop(&state, command);
```

means:

`hop` is allowed to modify the shell's persistent state."

Whereas a global means any function anywhere can silently modify it.

Where should `ShellState` be defined?

I'd probably make:

```
shell.h
```

or, if the assignment gets larger:

```
state.h
```

For example:

```
// shell.h

typedef struct {
    char *previous_dir;
    int should_exit;
} ShellState;
```

Then builtins can include it.

You could even have:

```
main.c
shell.c
shell.h
parser.c
parser.h
command.h
executor.c
executor.h
builtins.c
builtins.h
```

where `ShellState` is part of the shell-level interface.

One important thing for your shell

Don't put this state **inside the child process**.

For example, if you do:

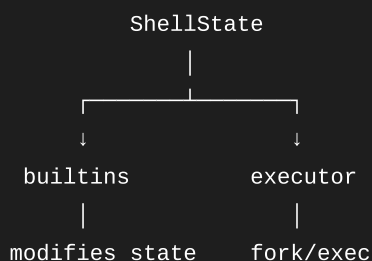
```
fork();

if (child) {
    builtin_hop(...);
}
```

then changing `previous_dir` in the child won't update the parent's state. Once `fork()` happens, they have separate memory.

That's also why builtins like `cd / hop` normally need to be executed **by the shell process itself**, rather than blindly going through your external-command `fork → exec` machinery.

So you can think of your architecture as:



For `hop`, this is particularly important because changing directories in a child would just change the child's directory and then disappear when the child exits.



after i read one command with ``getline()``, should i strip the newline immediately or should the lexer handle it as whitespace?



I'd **keep the newline and let the lexer handle it as whitespace**.

After:

```
getline(&line, &capacity, stdin);
```

you'll typically have:

```
"ls -l\n\0"
```

There's not much benefit to modifying it immediately. If your lexer is responsible for turning the raw input into tokens, then newline naturally belongs to that layer's handling of whitespace.

For example:

```
raw input
  ↓
"ls -l\n"
  ↓
lexer
  ↓
["ls", "-l"]
  ↓
parser
  ↓
Command
```

Your lexer can simply treat `'\n'` alongside spaces/tabs:

```
if (c == ' ' || c == '\t' || c == '\n') {
    ...
}
```

One reason I'd prefer this

It keeps responsibilities clean:

- `main.c`: reads a line
- **lexer**: decides what characters constitute whitespace/separators
- **parser**: decides what the tokens mean
- **executor**: executes the resulting structure

If `main.c` starts doing things like:

```
line[strcspn(line, "\n")] = '\0';
```

you're making the input-reading layer know something about how the lexer interprets the input.

But stripping it isn't *wrong*

For a very simple shell where each `getline()` result is guaranteed to represent exactly one command, doing:

```
if (len > 0 && line[len - 1] == '\n')  
    line[len - 1] = '\0';
```

is completely reasonable.

I'd just choose **one place** to handle it and be consistent.

Given that you're deliberately structuring this shell into lexer/parser/executor modules, I'd personally let **the lexer own whitespace**, including the trailing newline. It makes the pipeline cleaner and gives you fewer special cases in `main.c`.



the requirement says the prompt should appear whenever the shell isn't running a foreground process. does that mean i shouldn't print another prompt until the previous command has completely finished?



Yep, exactly. **For a foreground process, you should not print the next prompt until that process has finished.**

The basic shell loop is:



So for:

```
$ sleep 5
```

you should see:

```
$ sleep 5
```

and then **nothing for 5 seconds**. After `sleep` exits:

```
$
```

The reason is that the shell itself is waiting in `waitpid()`:

```
pid_t pid = fork();

if (pid == 0) {
    execvp(...);
    _exit(127);
}

waitpid(pid, NULL, 0);
```

```
/* only now print the next prompt */
```

What does "whenever the shell isn't running a foreground process" mean?

It's basically describing the shell's **interactive state**.

When a foreground child is running:

```
shell —waiting—> child process
```

the shell shouldn't accept another command or print another prompt.

When the foreground child finishes:

```
child exits
  ↓
waitpid() returns
  ↓
shell is interactive again
  ↓
print prompt
```

This becomes especially important later when you implement **background processes**, if the assignment has them.

For example, with something like:

```
$ sleep 10 &
$
```

the shell doesn't wait for the `sleep` process, so it can immediately return to the interactive state and print another prompt.

So the distinction is:

Foreground:

```
fork → wait → prompt
```

Background:

```
fork → prompt
```

assuming background execution is actually part of your assignment.

And one subtle point: the requirement doesn't mean you need some separate thread/process constantly checking whether the shell is "running a foreground process." Your normal `waitpid()` logic naturally enforces it.



how should i check whether the current cwd actually has the shell's starting directory as an ancestor? i don't want to just do a naive string prefix check because something like `/foo/bar2` would also start with `/foo/bar`.



Yeah, don't use a raw prefix. The clean way is to treat the paths as **path components**, not strings.

If your starting directory is:

```
/foo/bar
```

then these should count as descendants:

```
/foo/bar
/foo/bar/baz
/foo/bar/baz/qux
```

but these should **not**:

```
/foo/bar2
/foo/bar-old
/foo/barn
```

Simplest approach

Get both paths as absolute, canonical paths, then check:

1. The current path starts with the starting path.
2. The character immediately after the starting path is either `/` or `'\0'`.

So:

```
bool is_inside(const char *start, const char *cwd)
{
    size_t len = strlen(start);

    if (strncmp(start, cwd, len) != 0)
        return false;

    return cwd[len] == '\0' || cwd[len] == '/';
}
```

For example:

```
start = "/foo/bar"
cwd   = "/foo/bar/baz"
```

```
↑  
'/' → valid
```

but:

```
start = "/foo/bar"  
cwd   = "/foo/bar2"  
      ↑  
      '2' → invalid
```

And importantly:

```
start = "/foo/bar"  
cwd   = "/foo/bar"  
      ↑  
      '\0' → valid
```

But there's an important filesystem wrinkle

For your shell assignment, I'd be careful about assuming that `getcwd()` strings alone tell you the true **directory ancestry**.

Consider symbolic links. For example:

```
/home/me/start  
/home/me/link → /somewhere/else
```

If you `cd` through the symlink, the lexical path might look like it's underneath `start` while the actual directory isn't, or vice versa depending on how the shell handles paths.

If the requirement means **actual filesystem ancestry**, a more robust approach is to compare directory identities using `stat()`:

```
struct stat st;  
stat(path, &st);
```

and walk upward from the current directory with `..`, comparing `(st_dev, st_ino)` against the starting directory.

But if the assignment's `hop` requirement is simply about whether the **current working path is within the starting directory**, the component-boundary check above is probably what they're expecting.

One more edge case

If your starting directory is `/`, then every absolute path is underneath it. The boundary logic actually handles that nicely:

```
start = "/"  
cwd   = "/foo"
```

You just need to be aware that `strlen("/") == 1`, so `cwd[1] == 'f'`, meaning the simple code above would incorrectly reject it. You'd special-case `/`:

```
if (strcmp(start, "/") == 0)
    return true;
```

So for a **lexical path-based check**, I'd use:

```
bool is_inside(const char *start, const char *cwd)
{
    if (strcmp(start, "/") == 0)
        return true;

    size_t len = strlen(start);

    return strncmp(start, cwd, len) == 0 &&
        (cwd[len] == '\\0' || cwd[len] == '/');
}
```

The key idea is exactly what you noticed: **a directory relationship is based on a path-component boundary, not merely a string prefix.**



i'm a little confused by their use of "home directory" here because it doesn't seem to mean the user's actual `/home/user`. it's basically just the directory from which `./shell.out` was launched, right?



Yes — **if that's how the assignment defines it**, then you're understanding it correctly.

They're using **"home directory" as the shell's starting directory**, not the conventional Unix meaning of the user's home directory such as `/home/somsuta`.

So if you do:

```
cd ~/quantfest/shell
./shell.out
```

then the shell's "home directory" would be:

```
/home/.../quantfest/shell
```

because that's the **current working directory at the moment the shell starts**.

Then if the shell does:

```
hop ..
```

and moves to:

```
/home/.../quantfest
```

that's outside its home directory according to the assignment's terminology.

If it does:

```
hop src
```

and reaches:

```
/home/.../quantfest/shell/src
```

that's still inside its home directory.

Why this terminology is confusing

Normally, in Unix:

```
$ echo $HOME
/home/somsuta
```

means the user's actual home directory.

But a shell implementation can define its own concept of a **shell home/root**, and from the wording you're describing, that's what they're doing.

I'd actually name the variable something like:

```
char *shell_home;
```

rather than:

```
char *home;
```

and initialize it **once when your shell starts**, using the initial `getcwd()`.

That makes your intent crystal clear:

```
shell starts
  ↓
getcwd()
  ↓
shell_home = "/wherever/you/launched/it"
  ↓
never changes
```

Whereas the current directory is dynamic:

```
shell_home = "/project/shell" ← fixed

cwd = "/project/shell"          ← changes
cwd = "/project/shell/src"
cwd = "/project"
...
```

So yes: **don't use** `$HOME` **or** `/home/user` **for this unless the assignment explicitly says to.**



but bro if i say its opposite youll say oh YEAHH YOU'RE CORRECT



LMAO 🤖 **fair callout.**

I was too quick to agree with you there. I should've checked the *actual assignment wording* instead of reasoning from your interpretation.

And you're right to be suspicious because **"home directory" normally means** `$HOME` / `/home/user`, while "directory from which the shell was launched" is usually called the **initial/current working directory**.

Since this is specifically about your assignment, I shouldn't guess. If you want, I can **look at the uploaded assignment and tell you exactly what they mean by "home directory," with the relevant wording.**



nvm its correct



Lmao okay 🤖. Glad we got there. And yeah, **in this assignment's terminology**, your interpretation is correct.



bro i'm passing the starting directory into my prompt function, but i'm not sure whether i should pass the actual string around or just store it globally somewhere. what's a cleaner approach for a small shell?



Yeah, for a small shell I'd **not make it global**. I'd keep the starting directory as part of your shell's persistent state and pass a pointer to that state where needed.

Since you already have the idea of `ShellState`, this fits nicely:

```
typedef struct {
    char *home_dir;      // fixed: where shell started
    char *previous_dir;  // changes after hop
    int should_exit;
} ShellState;
```

Then:

```
int main(void)
{
    ShellState state = {
        .home_dir = getcwd(NULL, 0),
        .previous_dir = NULL,
        .should_exit = 0
    };

    while (!state.should_exit) {
        print_prompt(&state);
        ...
    }

    free(state.home_dir);
    free(state.previous_dir);
}
```

And your prompt:

```
void print_prompt(const ShellState *state)
{
    char *cwd = getcwd(NULL, 0);

    // use state->home_dir and cwd
    ...

    free(cwd);
}
```

Why I'd prefer this

You **don't actually want to pass the string itself everywhere**:

```
print_prompt(home_dir);
execute_command(home_dir);
builtin_hop(home_dir);
...
```

because now you're threading one particular piece of state through a bunch of functions.

Instead:

```
print_prompt(&state);
builtin_hop(&state, command);
```

makes it clear that these functions operate on **shell state**.

And you avoid:

```
char *home_dir; // global
```

which means any module can secretly modify it.

One distinction I'd make

Your `home_dir` and `previous_dir` have different semantics:

```
home_dir
  ↓
set once when shell starts
never changes

previous_dir
  ↓
changes whenever hop changes directory
```

So I'd definitely store both in `ShellState`, rather than repeatedly recomputing the starting directory.

Also, **don't store the starting directory as the current `getcwd()` result and then update it later**.

Take a copy at startup:

```
state.home_dir = getcwd(NULL, 0);
```

and leave it untouched.

So the architecture I'd use is basically:



```
↓  
print_prompt()  
|  
↓  
builtins / executor
```

For a small assignment, that's a very reasonable middle ground between **passing strings everywhere** and **global-variable soup**.



if the starting directory itself contains spaces, does that affect how the prompt should be constructed? i assume not because i'm just printing the path rather than parsing it.



Exactly. **Spaces in the starting directory don't matter for constructing the prompt**, because you're not passing the path to a shell command or reparsing it — you're just displaying a string.

For example, if the shell starts in:

```
/home/somsuta/My Projects/shell
```

you can simply do:

```
printf("<%s@%s:%s> ", username, hostname, cwd);
```

and get:

```
<somsuta@hostname:/home/somsuta/My Projects/shell>
```

No quoting or escaping is needed **for display**.

The distinction is:

- **Printing a path:** spaces are ordinary characters.
- **Passing a path as a command argument:** spaces can matter because the shell/parser may interpret them as argument separators.
- **Comparing paths:** spaces are also just characters, so your `strcmp` /component-boundary logic works normally.

So your `home_dir` can safely just be:

```
char *home_dir = getcwd(NULL, 0);
```

even if it contains spaces.

The only place you need to think about spaces specially is when your **lexer/parser processes user input**, e.g.:

```
hop "My Projects"
```

That's a parsing problem, not a prompt problem.



what's a good test for the case where the current directory has a similar-looking prefix to the starting directory but isn't actually inside it?



A very good test is to deliberately create **sibling directories where one name is a prefix of the other**.

For example, make:

```
/tmp/shelltest
/tmp/shelltest2
```

Start your shell from:

```
/tmp/shelltest
```

Then change the current directory to:

```
/tmp/shelltest2
```

Now:

```
starting directory = /tmp/shelltest
current directory  = /tmp/shelltest2
```

A naive check like:

```
strncmp(cwd, home, strlen(home)) == 0
```

would incorrectly say **inside**, because:

```
/tmp/shelltest2
/tmp/shelltest
    ↑
```

has the same prefix.

But your correct component-aware check should say **outside**, because after `/tmp/shelltest` the next character is `2`, not `/`.

Even better: test several cases

I'd test these:

```
Starting directory: /tmp/shelltest

Current directory:
1. /tmp/shelltest      → INSIDE
```

```
2. /tmp/shelltest/foo      → INSIDE
3. /tmp/shelltest/foo/bar  → INSIDE
4. /tmp/shelltest2        → OUTSIDE
5. /tmp/shelltesting       → OUTSIDE
6. /tmp/shelltest-old      → OUTSIDE
7. /tmp                   → OUTSIDE
```

The particularly important one is `shelltest2`, because that's the exact bug your boundary check is meant to prevent.

If you're testing your actual `hop` behavior, you could make:

```
/tmp/test/shell
/tmp/test/shell2
```

start the shell in `/tmp/test/shell`, then `hop ../shell2`.

If your prompt or `hop` logic claims you're still inside the shell's starting directory, you've got the prefix bug.



if i launch the shell from `/a/b` and go to `/a/bc`, that shouldn't become `~/c`, right? i'm thinking this is an easy edge case for accidentally implementing the prefix check incorrectly.



Correct. **Absolutely not** `~/c`. That's exactly the edge case you should test.

If the shell starts at:

```
/a/b
```

and the current directory is:

```
/a/bc
```

then `/a/bc` is **outside** the shell's starting directory.

So if your prompt abbreviates paths under the starting directory with `~/`, it should **not** transform:

```
/a/bc
```

into:

```
~/c
```

It should display the actual path (or whatever your assignment specifies for outside paths).

The incorrect implementation would be something like:

```
if (strncmp(cwd, home, strlen(home)) == 0) {  
    // assume cwd is inside home  
}
```

because:

```
home = /a/b  
cwd  = /a/bc  
      ^
```

The prefix matches, but there's no **path-component boundary**.

Your check needs:

```
strncmp(cwd, home, strlen(home)) == 0 &&  
(cwd[strlen(home)] == '\0' || cwd[strlen(home)] == '/')
```

Then:

/a/b	→ \~/	(inside)
/a/b/c	→ \~/c	(inside)
/a/b/c/d	→ \~/c/d	(inside)
/a/bc	→ /a/bc	(outside)
/a/bcd	→ /a/bcd	(outside)

So yes — `/a/b → /a/bc` **is probably one of the best tests you can write for this part.** It catches exactly the naive-prefix bug.



A2: User Input [2]

****Requirements:****

1. The shell should allow a user to type input.
2. On enter/return, the shell should consume the input.
3. After consuming input, the shell should display the prompt again.
4. You may assume that the user input shall be limited to a max of 1024 characters.

****Example****

```

```
<goatam@iiit:~> Hi there guys!
<goatam@iiit:~> This shell is cool!
<goatam@iiit:~>
```

i'm trying to understand what "consume the input" actually means here. does it just mean i need to read the whole line when enter is pressed, or am i supposed to process the characters somehow at this stage?

```



Yep — **at A2, “consume the input” basically just means read the line from stdin when the user presses Enter.** You don't need to parse or interpret it yet.

The intended flow is essentially:

```
print prompt  
  ↓  
user types: Hi there guys!  
  ↓  
presses Enter  
  ↓  
getline() reads "Hi there guys!\n"  
  ↓  
input has been consumed  
  ↓  
print prompt again
```

So for A2, something as simple as:

```
char *line = NULL;  
size_t capacity = 0;  
  
while (1) {  
    print_prompt();
```

```

    ssize_t n = getline(&line, &capacity, stdin);

    if (n == -1)
        break;

    // A2: input has been consumed.
    // Don't need to parse it yet.

    // next iteration prints prompt again
}

free(line);

```

Given their example:

```

<goatam@iiit:\~> Hi there guys!
<goatam@iiit:\~> This shell is cool!
<goatam@iiit:\~>

```

they're essentially demonstrating:

1. Prompt appears.
2. User enters arbitrary text.
3. Shell reads that entire line.
4. Shell doesn't necessarily **do** anything with it yet.
5. Prompt appears again.

"Consume" vs "process"

These are different stages:

Consume:

```

stdin → getline() → string in memory

```

Process:

```

string
  ↓
lexer
  ↓
parser
  ↓
command structure
  ↓
executor

```

A2 is only asking for the first one.

And your choice of `getline()` is actually a very natural fit because the requirement explicitly says input is limited to 1024 characters. You can read one complete line at a time and then hand that same string to your lexer later.

So I **wouldn't strip, tokenize, parse, or execute anything in A2** unless another requirement you're implementing at the same time needs it. Just make sure pressing Enter causes the input to be read and the next prompt to appear.



what should happen when the user presses enter without typing anything? should i consume the newline and immediately print another prompt?



Yes. **For an empty line, you should consume the newline and immediately show the next prompt.**

With `getline()`:

```
ssize_t n = getline(&line, &capacity, stdin);
```

if the user just presses Enter, you'll get:

```
line = "\n"
n = 1
```

You don't need to treat that as an error or execute anything.

So your loop can naturally be:

```
while (1) {
    print_prompt();

    ssize_t n = getline(&line, &capacity, stdin);

    if (n == -1)
        break;

    // If line is just "\n", do nothing.
    // Otherwise, eventually pass it to the lexer/parser.

    if (strcmp(line, "\n") == 0)
        continue;

    ...
}
```

Although you don't even necessarily need the explicit `if` at the A2 stage. If you're only testing input consumption:

```
while (1) {
    print_prompt();

    if (getline(&line, &capacity, stdin) == -1)
        break;
}
```

already gives you:

```
<user@host:\~>  
<Enter>  
<user@host:\~>  
<Enter>  
<user@host:\~>
```

The important distinction is that **empty input is still input**: the newline was consumed by `getline()`. You just don't have a command to execute afterward.

Later, your parser can simply produce something like "empty command" for a blank line, and the executor does nothing.



what happens if the user somehow enters more than 1024 characters even though the assignment says we can assume it won't happen? do i need to handle it anyway or can i ignore that case?



You can **ignore it for the assignment**. If they explicitly say:

"You may assume that the user input shall be limited to a max of 1024 characters."

that's a contract: your implementation doesn't need to defend against longer input.

And if you're using `getline()`, there's an additional nice property: **you don't have to manually enforce the 1024 limit anyway**. `getline()` dynamically grows its buffer and will happily read more than 1024 characters.

So:

```
char *line = NULL;
size_t capacity = 0;

ssize_t n = getline(&line, &capacity, stdin);
```

will read the entire line even if someone pastes 5000 characters.

You could deliberately impose a 1024-character limit, but **I wouldn't bother unless the assignment explicitly asks you to reject/truncate longer input**. It adds behavior that they didn't require.

The important thing is not to confuse:

"Input is guaranteed ≤ 1024 characters"

with:

"You must allocate exactly 1024 bytes."

You don't need:

```
char line[1025];
fgets(line, sizeof(line), stdin);
```

just because of that statement. `getline()` is perfectly fine and gives you a robust implementation for free.

So for your current A2:

≤ 1024 chars: must work.

> 1024 chars: unspecified; you can let `getline()` handle it however it naturally does.

Empty line: consume `\n`, print next prompt.

EOF (`ctrl-D`): handle separately as appropriate for the later shell requirements.



if a foreground command takes a long time, should the prompt really stay hidden until it finishes? i know that's in A1, but i'm trying to understand how the input loop and execution loop interact.



Yes — **for a foreground command, the prompt should stay hidden until it finishes.** That's actually the cleanest way to think about the shell's control flow.

Your shell isn't really doing:

```
prompt loop
  ↓
input loop
  ↓
execution loop
```

as three independent loops. It's more like **one main loop where execution temporarily blocks the shell:**

```
while (shell_running) {

    print_prompt();

    read_line();

    parse();

    execute();

    // execute() waits here if it's foreground

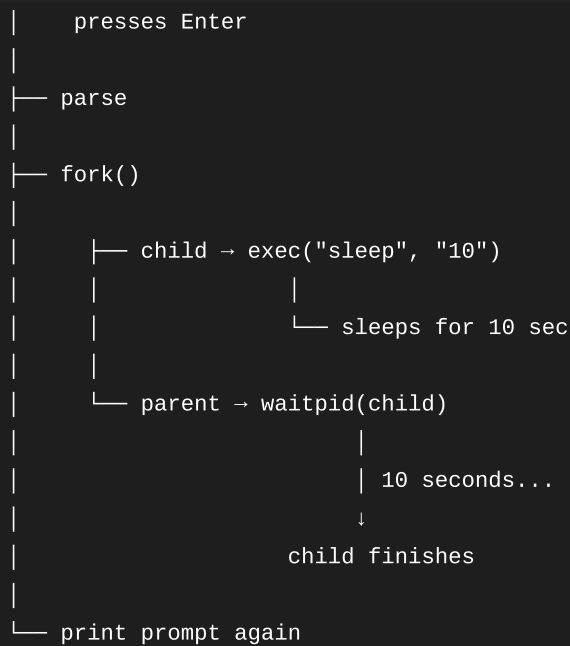
    // only after execute returns:
    // loop back and print prompt
}
```

For:

```
$ sleep 10
```

the actual sequence is:

```
shell
|
|— print prompt
|
|— getline()
|   ↑
|   user types "sleep 10"
```



So the important thing is:

The shell does not call `getline()` again while `waitpid()` is waiting.

That's what makes the process foreground.

Why this matters for A1

Suppose you did this incorrectly:

```
while (1) {
    print_prompt();
    getline(...);
    fork();
    // don't wait
}
```

Then you'd get something like:

```
$ sleep 10
$ $ $
```

The shell would immediately accept more input while `sleep` is still running. That's effectively background behavior.

Instead:

```
while (1) {  
    print_prompt();  
  
    getline(...);  
  
    pid_t pid = fork();  
  
    if (pid == 0) {
```

```

        execvp(...);
        _exit(127);
    }

    waitpid(pid, NULL, 0);
}

```

gives:

```

$ sleep 10
    ← shell waits here
    ← no prompt
    ← no new input
    ← 10 seconds pass

$

```

And this also explains why the A1 wording says:

display the prompt whenever the shell isn't running a foreground process

The prompt is really your indication that **the shell is currently ready to accept another command**.

One thing that becomes interesting later is **builtins**. A command like `hop ..` doesn't necessarily involve `fork()` at all — the shell executes it itself, then returns to the top of the loop and prints the prompt. So conceptually:

```

external foreground command:
prompt → input → fork → exec → wait → prompt

builtin:
prompt → input → builtin → prompt

```

That's a useful architecture to keep in mind while you're building the later sections.



A3: Input Parsing [15]

Your shell must scan and validate every line before executing anything. This section defines two layers: a lexer that turns characters into tokens, and a grammar that validates the token sequence. Validation must be performed.

> Note: Since both Lexer and Parser follow a regular grammar, use your Automata Theory skills to the test.

****Lexer****: The lexer reads the raw input line and produces a sequence of tokens, some specifications are given below. Whitespace (space, tab, newline, carriage return) separates tokens and is otherwise discarded - it is never itself a token.

- ****Character Classes****:

```
...
special -> | & > < ;
quote   -> " | '
escape  -> \
space   -> ' ' | \t | \n | \r
ordinary -> any character that is none of the above
...
```

- ****Tokens****:

```
...
OP_PIPE   -> |
OP_AMP    -> &
OP_SEMI   -> ;
OP_LT     -> <
OP_GT     -> >
OP_GTGT   -> >>

WORD      -> fragment+

fragment  -> ordinary
           | escape any_char
           | " dq_body "
           | ' sq_body '

dq_body   -> ( escape any_char | any char except " and \ ) *
sq_body   -> ( any char except ' ) *
...
```

- ***Apply maximal munch***: i.e. at each position, take the longest match. e.g. `>>` is therefore one token, never two `>`.

****Some details on Quoting****:

ConstructEffect	
-----	-----

On the unquoted value `c` outside quotes	Contributes `c` literally;
the backslash is removed	
`"...`	Contributes the
body; `\"` becomes `\"`, `\\` becomes `\\`; any other `c` stays as both	
characters	
`'...'`	Contributes the body
verbatim; **no** escape processing, so `\\` is literal	
Special characters inside quotes or escaped	Lose their operator
meaning entirely	

****Lexical errors****: The lexer must reject and print `cshell: invalid syntax` when:

- A quote is opened and never closed before end of line.
- A `\\` appears as the final character of the line.

> Note: There is no line continuation and no multi-line input. Hint: use EOF to your advantage.

****Grammar****:

Terminals are the token classes above. Start symbol is `LINE`

```

` ` `
LINE -> ε
      | WORD ARG
ARG  -> ε
      | WORD  ARG
      | OP_LT  TGT
      | OP_GT  TGT
      | OP_GTGT TGT
      | OP_PIPE CMD
      | OP_SEMI CMD
      | OP_AMP  BG
CMD  -> WORD ARG
TGT  -> WORD ARG
BG   -> ε
      | WORD ARG
` ` `

```

> Note: This is a right linear grammar, enjoy.

`TGT` is the name of a file or a redirection output, `CMD` is a valid command

****Instructions****:

- You **do not need** a full AST, any structure you find convenient (like a Linked List) works. Just make sure you are representing enough information internally to allow further sections to be implemented.
- You can use any type of parser and lexer, as long as it conforms to the grammar provided.

Requirements

1. Verify whether an inputted command is valid or invalid per the grammar.
2. If invalid, print `cshell: invalid syntax`.
3. A line that is empty or contains only whitespace is valid: print a fresh prompt.
4. Whitespace between tokens is ignored, in any amount. Whitespace inside quotes is preserved.

Examples:

```
```\n<goatam@iiit:~/osnmp1> cat meow.txt | meow; meow > meow.txt &\n<goatam@iiit:~/osnmp1> echo a & echo b &\n<goatam@iiit:~/osnmp1> cat < a.txt < b.txt > out.txt\n<goatam@iiit:~/osnmp1> echo "line one" > a.txt\n<goatam@iiit:~/osnmp1> echo "a > b"\n<goatam@iiit:~/osnmp1> cat my\\ file.txt\n<goatam@iiit:~/osnmp1> echo ""\n<goatam@iiit:~/osnmp1> reveal -ta -aaa -tt\n<goatam@iiit:~/osnmp1> cat meow.txt | ; meow\ncshell: invalid syntax\n<goatam@iiit:~/osnmp1> echo hi ;\ncshell: invalid syntax\n<goatam@iiit:~/osnmp1> echo hi & &\ncshell: invalid syntax\n<goatam@iiit:~/osnmp1> cat <\ncshell: invalid syntax\n<goatam@iiit:~/osnmp1> | sort\ncshell: invalid syntax\n<goatam@iiit:~/osnmp1> echo "unclosed\ncshell: invalid syntax\n<goatam@iiit:~/osnmp1> echo trailing\\ncshell: invalid syntax\n<goatam@iiit:~/osnmp1>
```

if i see > and the next character is whitespace, that's obviously just OP\_GT, but if the next character is another >, it becomes OP\_GTGT, right?

```
```\n
```



Exactly. **That's maximal munch in action.**

When the lexer is at `>`:

- If the next character is **not** `>`, emit `OP_GT`.
- If the next character **is** `>`, consume both and emit **one** `OP_GTGT`.
- Whitespace after it doesn't change anything.

So:

```
>      → OP_GT
>>     → OP_GTGT
> foo   → OP_GT, WORD("foo")
>> foo  → OP_GTGT, WORD("foo")
```

For example:

```
echo hi > out.txt
```

becomes:

```
WORD("echo")
WORD("hi")
OP_GT
WORD("out.txt")
```

while:

```
echo hi >> out.txt
```

becomes:

```
WORD("echo")
WORD("hi")
OP_GTGT
WORD("out.txt")
```

The important part

Your lexer should decide this **without involving the parser.**

At position `i`:

```
current character = '>'
|
|— input[i+1] == '>'
|   ↓
|   consume 2 characters
|   emit OP_GTGT
|
|— otherwise
```

```
↓
consume 1 character
emit OP_GT
```

You don't need to look at the character *after* the `>` pair to determine the token.

So:

```
>>>
```

would lex as:

```
OP_GTGT
OP_GT
```

and then the **parser** decides whether that sequence is grammatically valid.

Likewise:

```
> >
```

is:

```
OP_GT
OP_GT
```

because whitespace prevents the two `>` characters from forming `>>`.

And this is a nice example of why **lexing and parsing need to stay separate**:

```
echo hi >> out.txt
```

The lexer says:

```
>> is OP_GTGT .
```

Then the parser says:

```
WORD WORD OP_GTGT WORD matches WORD ARG , where ARG → OP_GTGT TGT .
```

So yes: `>` **followed immediately by** `>` **=** `OP_GTGT` ; **otherwise the first** `>` **is** `OP_GT` .



what happens with something like ``echo 'a"b'``? since i'm inside single quotes, the double quote shouldn't have any special meaning, right?



Yep, exactly. **Inside single quotes, `"` has no special meaning at all.**

For:

```
echo 'a"b'
```

the lexer sees:

```
WORD("echo")  
WORD("a\"b")    // conceptually; the value is actually a"b
```

More precisely, the second token's value is:

```
a"b
```

The `'` characters are delimiters and aren't part of the resulting word.

Your grammar says:

```
sq_body -> ( any char except ' )*
```

Notice that it excludes **only** `'`. So these are all ordinary contents of a single-quoted string:

```
'a"b'  
'a > b'  
'a | b'  
'a & b'  
'a\ b'
```

They all produce a single `WORD`, and the special characters lose their operator meaning.

For example:

```
echo 'a > b'
```

is:

```
WORD("echo")  
WORD("a > b")
```

not:

```
WORD("echo")  
WORD("a")
```

```
OP_GT  
WORD("b")
```

And importantly, the backslash is also literal inside single quotes:

```
echo 'a\ b'
```

produces the word:

```
a\ b
```

No escape processing occurs.

Compare that with double quotes:

```
echo "a\"b"
```

Here `\` is processed and the resulting word is:

```
a"b
```

So the mental rule is:

```
'...' → everything is literal except '  
"..." → everything is literal except " and special handling of \
```

That's exactly why your lexer needs to remember **which quote mode it's currently inside**.



if the final character is a backslash inside single quotes, is it still an error? the spec says backslash is literal inside single quotes, so i'm wondering if that should actually be valid.



Yes — **your reasoning is correct.** A backslash at the end of a **single-quoted** string is valid, because inside `'...'`, the backslash has no escape meaning.

For example:

```
echo 'abc\'
```

is valid.

The single quote closes the quoted body, and the `\` before it is simply part of the body:

```
WORD("echo")
WORD("abc\")
```

The important distinction is **what state the lexer is in when it encounters the final** `\`.

Outside quotes

```
echo abc\
```

❌ Invalid syntax

The final `\` is an escape character that requires another character:

```
escape any_char
```

There is no `any_char` after it.

Inside double quotes

```
echo "abc\
```

❌ Invalid syntax

Same reason: in double quotes, `\` participates in escape processing, so it needs a following character.

Inside single quotes

```
echo 'abc\
```

❌ Invalid syntax, but for a different reason: the single quote was never closed.

The backslash itself is completely fine.

So:

```
echo 'abc\'
```

✓ valid

while:

```
echo 'abc\
```

✗ invalid because you're still inside `'...'`.

That's a really important lexer detail: **don't implement “if the last character is `\`, reject” as a blanket rule.** You need to know the current lexical state.

Something like:

```
state = OUTSIDE

'      → state = SINGLE_QUOTE
"      → state = DOUBLE_QUOTE
\      → escape handling only in OUTSIDE/DOUBLE_QUOTE
'      → exit SINGLE_QUOTE
"      → exit DOUBLE_QUOTE
```

Then at end-of-line:

```
if state == SINGLE_QUOTE or DOUBLE_QUOTE:
    invalid syntax

if state == OUTSIDE and previous char was an unprocessed '\':
    invalid syntax
```

That distinction follows directly from the grammar you pasted.



i'm trying to understand whether something like `echo hi & &` is a lexer problem or parser problem. i'm guessing the lexer should happily produce both `OP\_AMP` tokens and the grammar should reject the sequence.



Yep — **100% a parser/grammar problem**, not a lexer problem.

For:

```
echo hi & &
```

the lexer should happily produce:

```
WORD("echo")
WORD("hi")
OP_AMP
OP_AMP
```

Nothing is lexically wrong with either `&`. Each `&` is a perfectly valid `OP_AMP` token.

Then the parser tries to match the grammar:

```
ARG -> OP_AMP BG
```

After the first `OP_AMP`, it has to parse a `BG`.

And:

```
BG -> ε
      | WORD ARG
```

So after the first `&`, the parser can either:

1. stop the background section (ϵ), **but then there's still another `OP_AMP` left over**, or
2. expect a `WORD`, but it sees `OP_AMP`.

Either way → **invalid syntax**.

This distinction is useful

Think:

Lexer asks:

"Can I turn these characters into valid tokens?"

Parser asks:

“Can these valid tokens occur in this particular order?”

So:

```
echo hi & &
```

is:

```
characters
  ↓
LEXER
  ↓
WORD WORD OP_AMP OP_AMP      ← perfectly valid tokens
  ↓
PARSER
  ↓
invalid syntax
```

Whereas:

```
echo "unclosed
```

is a **lexer error**, because the lexer can't even produce a valid `WORD` token — the quoted fragment never closes.

Likewise:

```
echo trailing\
```

is a lexer error because the final `\` requires an `any_char` after it.

And:

```
echo hi <
```

is a **parser error**: `<` is a perfectly valid `OP_LT` token, but the grammar requires:

```
OP_LT TGT
```

and `TGT` ultimately requires a `WORD`.

So your division is exactly right:

Input	Lexer	Parser
echo hi & &	✓	✗
echo hi <	✓	✗
	✓ `WORD("`	")`
echo "unclosed	✗	never reached

Input	Lexer	Parser
echo trailing\	✗	never reached
echo hi >> out	✓ OP_GTGT	✓

That's also why the assignment emphasizes **“scan and validate every line before executing anything.”** You should first get either a valid token stream, then validate that stream against the grammar, and **only after both succeed** should execution happen.



the assignment says sequential and background operators should eventually be handled, but for this section do i only need to make sure the grammar accepts them in the right positions?



Yes — **for A3 specifically, your job is parsing/validation, not execution semantics.**

The key phrase is:

“Your shell must scan and validate every line before executing anything.”

So in A3, for `;` and `&`, you need to:

1. **Lex them** as `OP_SEMI` / `OP_AMP`.
2. **Parse them according to the grammar.**
3. Reject invalid placements.
4. Build some internal representation that preserves enough information for later execution.

You **do not yet need to actually implement what `;` or `&` does**, unless another section explicitly requires that at this stage.

For example:

```
echo a ; echo b
```

should be recognized as valid:

```
WORD(echo) WORD(a) OP_SEMI
WORD(echo) WORD(b)
```

and your internal representation should retain the `;` so that a later execution stage knows:

```
execute command 1
wait
execute command 2
```

Similarly:

```
echo a & echo b
```

is syntactically valid under the grammar, and you should preserve the `&` so later you can implement its background semantics.

But:

```
echo a ;
```

should be rejected **now**, because the grammar doesn't allow `OP_SEMI` at the end:

```
ARG -> OP_SEMI CMD
CMD -> WORD ARG
```

Likewise:

```
echo a & &
```

is rejected now because after the first `&`, `BG` can't consume another `OP_AMP`.

So think of A3 as

```
raw line
  ↓
LEX
  ↓
tokens
  ↓
PARSE / VALIDATE
  ↓
valid command representation
  ↓
_____
later sections
  ↓
EXECUTE
  ↓
actually implement ;, &, |, <, >, >>
```

And this is why the assignment says:

“any structure you find convenient ... Just make sure you are representing enough information internally to allow further sections to be implemented.”

That last sentence is the important one. **Don't throw away the operators just because you're not executing them yet.**

For example, don't turn:

```
echo a ; echo b
```

into just two unrelated `Command` structs and lose the fact that there was a `;` between them.

You want something conceptually like:

```
Command("echo", "a")
  |
SEMI
```

```
|  
Command("echo", "b")
```

Then later your executor can attach the appropriate behavior to `SEMI`.

So yes: **A3 validates the syntax and constructs the representation; later sections give those syntactically valid operators their actual process behavior.**



can you give me tests specifically for the difference between quotes and escaped special characters? i feel like that's where my lexer is most likely to be wrong.



Absolutely. This is probably the **highest-value set of lexer tests** for your A3, because you need to distinguish:

- special characters **outside quotes**
- special characters **escaped outside quotes**
- special characters **inside double quotes**
- special characters **inside single quotes**
- backslashes behaving differently in each context

I'd test these in groups.

1. Special characters normally have operator meaning

```
echo a|b
echo a&b
echo a>b
echo a<b
echo a;b
```

Expected tokenization:

```
echo a | b
echo a & b
echo a > b
echo a < b
echo a ; b
```

i.e. `|`, `&`, `>`, `<`, `;` should become operator tokens.

2. Escape each special character outside quotes

These should **all produce one WORD**, with the backslash removed:

```
echo a\|b
echo a\&b
echo a\>b
echo a\<b
echo a\;b
```

Expected word values:

```
a|b
a&b
a>b
a<b
a;b
```

This is a really important test because:

```
a\|b
```

must **not** become:

```
WORD("a") OP_PIPE WORD("b")
```

It must be:

```
WORD("a|b")
```

The escaped `|` has lost its operator meaning.

3. Same characters inside double quotes

```
echo "a|b"
echo "a&b"
echo "a>b"
echo "a<b"
echo "a;b"
```

Each should produce one word:

```
a|b
a&b
a>b
a<b
a;b
```

The operators have no special meaning inside `"..."`.

4. Same characters inside single quotes

```
echo 'a|b'
echo 'a&b'
echo 'a>b'
echo 'a<b'
echo 'a;b'
```

Again, each is one `WORD`.

This tests that your lexer doesn't accidentally apply operator recognition while inside single quotes.

5. The really important quote-vs-escape tests

These are the ones I'd **definitely** run.

Outside quotes

```
echo \"  
echo \'
```

Both should be valid `WORD`s:

```
"  
'
```

because outside quotes, `\` escapes the next character.

Inside double quotes

```
echo "\""  
echo "\' "
```

The first:

```
"\""
```

should produce:

```
"
```

because `\"` becomes `"`.

But:

```
"\' "
```

should produce the two characters:

```
\'
```

because your specification explicitly says:

in double quotes, any other `\c` stays as both characters.

So `\` **is retained** here.

Inside single quotes

```
echo '\\"'  
echo '\\''
```

First:

```
'\"'
```

is valid and produces:

```
\"
```

because backslash is completely literal inside single quotes.

The second:

```
'\''
```

is **not** an escaped quote. The `\` does nothing special. Therefore the first `'` after the backslash closes the string, and the remaining `'` opens another quote → ultimately unclosed → **invalid syntax**.

This is an excellent test for accidentally implementing C-style escaping inside single quotes.

6. Backslash itself

Test:

```
echo \\  
\\
```

Outside quotes, `\\` should produce a single:

```
\\  
\  
\\
```

Double quotes:

```
echo "\\\"  
\\\"
```

should also produce:

```
\\  
\  
\\
```

because `\\ → \`.

Single quotes:

```
echo '\\'  
\\'
```

should produce **two backslashes**:

```
\\  
\\  
\\
```

because there is no escape processing whatsoever.

That trio is particularly useful:

```
\\      → \
"\\\"   → \
'\\"'   → \\\
```

7. Whitespace is another good one

Outside quotes:

```
echo a\ b
```

should produce one word:

```
a b
```

while:

```
echo a b
```

produces two words:

```
a
b
```

Double quotes:

```
echo "a b"
```

produces one word:

```
a b
```

Single quotes:

```
echo 'a b'
```

also produces one word:

```
a b
```

8. Operators adjacent to quoted/escaped things

These catch bugs where your lexer accidentally treats the contents of a word as separate tokens:

```
echo "a|b"|cat
echo 'a|b'|cat
echo a\|b|cat
```

Expected:

```
WORD("echo")
WORD("a|b")
OP_PIPE
WORD("cat")
```

for all three.

Similarly:

```
echo "a>b">out
echo 'a>b'>out
echo a\>b>out
```

should all have:

```
WORD("echo")
WORD("a>b")
OP_GT
WORD("out")
```

The first `>` is protected by quotes/escape; the second is a real operator.

9. Mixed pieces forming ONE word

This is another **very good lexer test**:

```
echo abc"def"ghi
```

This should be **one** `WORD`, with value:

```
abcdefghi
```

Similarly:

```
echo abc'def'ghi
```

→

```
abcdefghi
```

and:

```
echo abc\|def
```

→

```
abc|def
```

Your lexer shouldn't assume that quotes always constitute an entire token.

If I had to pick only 10 tests

I'd run these:

```
echo a|b
echo a\|b
echo "a|b"
echo 'a|b'

echo a\ b
echo "a b"
echo 'a b'

echo "\"
echo '\\"'

echo abc"def"ghi
```

And then these **three nasty ones**:

```
echo "\"
echo '\\"'
echo '\\"'
```

Those last three are particularly good at exposing whether your lexer actually understands the different quote states rather than just doing simplistic “find the next quote” logic.



can you give me a few cases where the parser should reject the line even though every individual character is perfectly valid?



Yep — these are exactly the tests you want for separating **lexical validity** from **grammatical validity**.

The lexer should happily tokenize all of these; the **parser** should reject them because the token sequence doesn't match your grammar.

1. Operator at the beginning

```
| sort
```

Tokens:

```
OP_PIPE WORD("sort")
```

✗ Reject.

`LINE` must start with `WORD`.

Same for:

```
> file
< file
; echo hi
& echo hi
```

All lexically valid, but invalid grammar.

2. Operator at the end

```
echo hi |
```

Tokens:

```
WORD WORD OP_PIPE
```

✗ Reject.

After `OP_PIPE`, grammar requires:

```
CMD -> WORD ARG
```

So another `WORD` must follow.

Similarly:

```
echo hi ;
echo hi &
cat <
echo hi >
echo hi >>
```

all ❌.

3. Two operators next to each other

```
echo hi & &
```

```
WORD WORD OP_AMP OP_AMP
```

❌ Reject.

The first `&` can enter `BG`, but `BG` can't consume another `OP_AMP`.

Likewise:

```
echo hi | |
echo hi ; ;
echo hi > <
echo hi < >
```

all ❌.

4. Pipe followed by an operator

```
echo hi | ; echo
```

Tokens:

```
WORD WORD OP_PIPE OP_SEMI WORD
```

❌ Reject.

After `|`, you need:

```
CMD -> WORD ARG
```

but you got `OP_SEMI`.

Same idea:

```
echo hi | >
echo hi | &
echo hi | <
```

all ❌.

5. Redirection without a target

```
echo hi >
```

Tokens:

```
WORD WORD OP_GT
```

✗ Reject because:

```
OP_GT TGT
```

requires a target `WORD`.

Same for:

```
echo hi <
echo hi >>
```

6. Redirection followed by another operator

```
echo hi > |
```

Tokens:

```
WORD WORD OP_GT OP_PIPE
```

✗ Reject.

The `>` requires a `TGT`, which ultimately starts with `WORD`, but gets `OP_PIPE`.

7. Semicolon followed by nothing

```
echo hi ;
```

✗ Reject.

Your grammar says:

```
OP_SEMI CMD
```

and `CMD` requires a `WORD`.

This is why the assignment's example explicitly has:

```
echo hi ;
cshell: invalid syntax
```

8. Background operator followed by another operator

```
echo hi & |
```

Tokens:

```
WORD WORD OP_AMP OP_PIPE
```

✗ Reject.

After `&`, `BG` is either:

```
ε
```

or:

```
WORD ARG
```

It can't start with `|`.

9. Two commands without an operator

```
echo hi echo bye
```

This one is **actually valid** under your grammar.

It becomes:

```
WORD("echo")  
WORD("hi")  
WORD("echo")  
WORD("bye")
```

because `ARG` can contain unlimited `WORD`s.

That's a useful test because you don't want to accidentally impose a more restrictive grammar than the assignment gives you.

10. A particularly sneaky one

```
echo hi > out.txt more
```

This is also **valid according to the supplied grammar**.

After:

```
OP_GT TGT
```

you have:

```
TGT -> WORD ARG
```

so `more` can be another argument.

Therefore:

```
WORD WORD OP_GT WORD WORD
```

is valid.

This is worth testing because your intuition might say “redirection target must be the last thing,” but **their grammar doesn't say that**.

The general parser test

Whenever you want to test the parser, deliberately construct a token sequence where:

```
lexer: "Yep, every token is valid."  
parser: "Nope, these tokens can't occur in this order."
```

Some excellent ones:

```
| sort  
echo hi |  
echo hi ;  
echo hi &  
cat <  
echo hi >  
echo hi >>  
echo hi & &  
echo hi | |  
echo hi > <  
echo hi | ; echo  
echo hi & | echo
```

None of those require the lexer to complain. **The lexer did its job successfully; the parser is where the rejection happens.**



Part B: Shell Intrinsic [40]

These are commands any shell worth its salt supports.

Banned syscalls for Part B: `exec*` (any syscall that starts with `exec`)

B1: hop [10]

Syntax: `hop ((~ | . | .. | - | name)*)?`

Purpose: The `hop` command changes the shell's current working directory. Unlike a plain `cd`, `hop` falls back to a frequency lookup when a name doesn't resolve directly, so you can jump to directories you visit often without typing the full path.

Requirements

Execute the following sequentially for each passed argument:

1. `~` or no arguments: change the CWD to the shell's home directory.
2. `.`: do nothing, stay in the same CWD.
3. `..`: change the CWD to the parent directory of the CWD, or do nothing if the CWD has no parent.
4. `-`: change the CWD to the previous CWD, or do nothing if there was no previous CWD.
5. `name`: if this resolves to a valid relative or absolute path from the CWD, change to it directly.
6. If `name` does not resolve directly, fall back to a frequency lookup as defined below.
7. If neither a direct match nor a frequency match is found, output `hop: no such directory`
8. `hop` should not restrict directory changes to just the shell's home directory or its subdirectories. Valid absolute and relative paths that resolve outside the home directory must be allowed.

Frequency

Keep a persistent record of directories hopped into, across shell sessions.

1. Every successful hop updates that directory's standing.
2. Standing depends on both frequency and recency of visits.
3. When `name` doesn't resolve directly, hop into the highest ranked directory whose path contains `name` as a substring.
4. If the top match no longer exists on disk, skip it and try the next best.

Scoring, decay, and storage are up to you, but must be deterministic.

****Resource****: same idea behind
zoxide's ``cd`` replacement: <https://github.com/ajeetsouza/zoxide/wiki/Algorithms>

****Example****

```
```  
<goatam@iiit:~/osnmp1> hop ~
<goatam@iiit:~> hop ..
<goatam@iiit:/home/> hop goatam/osnmp1 .. -
<goatam@iiit:~/osnmp1> hop osn
<goatam@iiit:~/osnmp1>
```

what does "do nothing if the CWD has no parent" actually mean at the filesystem root? would `chdir("..")` already leave me at `/`, or should i explicitly detect that case?

```
```
```



At the filesystem root, `/`, `chdir("..")` **already leaves you at** `/`. POSIX treats the root as its own parent in this sense.

So if you're at:

```
/
```

and do:

```
chdir("..");
```

you remain at:

```
/
```

So what does “do nothing if the CWD has no parent” mean?

It's basically specifying the **observable behavior**:

If you're at a directory where moving to `..` doesn't actually take you somewhere new, don't change anything.

At `/`, that's naturally satisfied by the filesystem:

```
CWD = /  
chdir("..")  
CWD = /
```

You **don't need to explicitly detect** `/` just to satisfy this requirement.

For example, your `hop ..` implementation can simply do:

```
if (strcmp(arg, "..") == 0) {
    chdir("..");
}
```

At `/`, `chdir()` succeeds and you're still at `/`.

But there's a subtlety with your `previous CWD`

Because your shell tracks `-`, you should think about whether a "successful" `hop ..` at `/` counts as a directory change.

The wording says:

Every successful hop updates that directory's standing.

But for `..`:

change to the parent directory ... or do nothing if the CWD has no parent.

At `/`, I'd interpret this as **no actual directory change**, so you shouldn't update `previous_dir` from `/` to `/` just because `chdir("..")` technically succeeds.

A clean approach is to compare before/after:

```
char *old_cwd = getcwd(NULL, 0);

if (chdir("..") == 0) {
    char *new_cwd = getcwd(NULL, 0);

    if (strcmp(old_cwd, new_cwd) != 0) {
        // actual directory change
        update_previous_dir(old_cwd);
    }

    free(new_cwd);
}

free(old_cwd);
```

Although you can make this simpler by explicitly checking for `/` if you want the semantics to be crystal clear.

One more important point

Don't use:

```
chdir("..");
```

and then assume that means the new path is literally the string obtained by removing the last component. The filesystem handles things like symlinks and mount points.

For your shell, `chdir()` **should be the authority on whether the directory change actually succeeded**, and `getcwd()` can tell you where you ended up.

So the short answer:

At `/`, `chdir("../")` is already a no-op. You don't have to special-case it unless you need the explicit distinction for your `previous_dir` bookkeeping.



if one argument in `hop a b c` fails, should `b` and `c` still be attempted? the wording says execute sequentially, so i'm assuming yes, but i want to make sure a failed hop doesn't stop the rest.



Yes — **that's the correct interpretation.** A failed argument should **not stop processing the remaining arguments.**

The wording:

"Execute the following sequentially for each passed argument"

means each argument is handled independently, in order.

So:

```
hop a b c
```

means roughly:

```
process "a"
  ↓
process "b"
  ↓
process "c"
```

If `a` fails:

```
hop a
→ hop: no such directory
```

you **still process** `b`, then `c`.

Example

Suppose you're currently in:

```
/home/user/project
```

and:

```
hop does-not-exist src ..
```

Then:

1. `does-not-exist`
 - direct lookup fails

- frecency lookup fails
- print:

```
hop: no such directory
```

- **stay where you are**

2. `src`

- resolves
- `chdir("src")`

3. `..`

- resolves
- `chdir("../")`

So after the entire command, you could be back at:

```
/home/user/project
```

even though the first argument failed.

Implementation-wise

I'd make your builtin loop look conceptually like:

```
for (int i = 1; i < argc; i++) {
    if (strcmp(argv[i], "~") == 0) {
        ...
    } else if (strcmp(argv[i], ".") == 0) {
        ...
    } else if (strcmp(argv[i], "..") == 0) {
        ...
    } else if (strcmp(argv[i], "-") == 0) {
        ...
    } else {
        if (!try_direct(argv[i]) &&
            !try_frecency(argv[i])) {
            printf("hop: no such directory\n");
        }
    }
}
```

Notice there's **no** `return` **or** `break` **when one fails**.

One subtle consequence: **the later arguments operate relative to wherever the earlier arguments left you**. That's what “sequentially” implies.

So:

```
hop dir1 dir2
```

means:

```
CWD
↓
dir1
↓
dir2 ← resolved from dir1, not the original CWD
```

That detail is probably quite important for your implementation.



if `hop -` is used before i've successfully changed directories at all, what exactly should happen? should it silently do nothing as specified, or should there be an error?



It should **silently do nothing**.

The requirement explicitly says:

```
"-": change to the previous CWD, or do nothing if there was no previous CWD.
```

So if the shell starts at `/a/b` and you immediately do:

```
hop -
```

there is no previous directory yet, so:

```
CWD: /a/b
↓
hop -
↓
CWD: /a/b
```

No error message.

And importantly, this is different from a failed `name` lookup. For example:

```
hop nonexistent
```

should produce:

```
hop: no such directory
```

because the requirement explicitly says to print that when neither direct nor frequency lookup succeeds.

I'd model the two cases separately:

```
hop -
↓
previous_dir exists?
├─ yes → chdir(previous_dir)
└─ no  → do nothing

hop name
↓
direct match?
├─ yes → chdir(...)
└─ no
↓
```

```
frequency match?  
└─ yes → chdir(...)  
└─ no  → print "hop: no such directory"
```

One thing to be careful about: **a failed** `hop name` **should not create or modify** `previous_dir`, since no directory change occurred.

Likewise, I'd treat `hop .` at the current directory as a no-op rather than updating `previous_dir`, because the requirement describes it as “do nothing.”



i'm less sure about the frequency part because the scoring is left up to us. what information would i actually need to persist for each directory so that frequency and recency can both affect the ranking?



Yeah — since **the scoring/decay/storage are explicitly left to you**, you don't need to reproduce zoxide's exact algorithm. You just need a deterministic scheme where **both frequency and recency matter** and the record survives shell sessions.

For each directory, I'd persist at least:

```
path
visit_count
last_visited
```

For example:

```
/path/to/project      7      2026-08-25T01:30:00
/path/to/downloads    3      2026-08-24T18:20:00
/path/to/foo          1      2026-08-25T01:40:00
```

Then you can define a deterministic score.

A simple scoring scheme

You could use something like:

```
score = frequency / (1 + age)
```

where:

- `frequency` = number of successful hops into that directory
- `age` = time since its last visit

For example, if you measure age in hours:

```
score = visit_count / (1 + hours_since_last_visit)
```

Then:

```
A: visits = 10, age = 20 hours
   score = 10 / 21 ≈ 0.476
```

```
B: visits = 3, age = 1 hour
   score = 3 / 2 = 1.5
```

So B wins despite being visited fewer times because it's much more recent.

That satisfies the requirement that **both frequency and recency influence ranking**.

What happens on a successful hop?

Suppose you successfully hop into:

```
/home/user/projects
```

You update:

```
path          = /home/user/projects
visit_count += 1
last_visited = now
```

If the entry doesn't exist yet, create it:

```
/home/user/projects    1    <timestamp>
```

Then save the database.

For lookup

Suppose the user types:

```
hop proj
```

First try:

```
chdir("proj")
```

If that doesn't work, search your persisted records for paths containing `"proj"`:

```
/home/user/projects
/home/user/old-project
/tmp/project-backup
```

Then calculate the score for each and choose the highest.

But check that the directory still exists before accepting it, exactly as the assignment requires.

One thing I'd definitely persist: absolute paths

Don't store:

```
projects
```

Store:

```
/home/somsuta/projects
```

Otherwise the meaning of the entry changes depending on where the shell currently is.

You don't need to persist the calculated score

I'd actually store only:

```
struct FrecencyEntry {
    char *path;
    unsigned long visits;
    time_t last_visited;
};
```

and calculate:

```
score(path)
```

when doing a lookup.

That's cleaner because the score automatically reflects the current time.

So your persistent file might conceptually look like:

```
/home/user/projects      12  1756059000
/home/user/downloads     5   1756051000
/home/user/os             8   1756058500
```

where the last column is a Unix timestamp.

And there's a nice property here

You can explain your algorithm very simply in your submission:

Each successfully visited directory is stored with its absolute path, visit count, and timestamp of its most recent visit. When a direct lookup fails, matching entries are ranked using a deterministic score combining visit frequency and recency. The highest-scoring existing directory is selected.

That's completely defensible given that the assignment explicitly says **“Scoring, decay, and storage are up to you, but must be deterministic.”**

I wouldn't over-engineer this with a complicated zoxide clone. A small persistent `(path, count, last_visited)` database is plenty.



i'm confused about how `hop` should actually be implemented because unlike the other commands, changing the directory has to affect the shell itself. if i fork a child and call `chdir()` there, the parent shell obviously won't change its cwd, so does that mean `hop` has to be handled completely in the parent process? and then how should i structure the execution code so that `hop` is treated differently from external commands without making the whole executor messy?



Yes — **you've identified the key issue exactly.** `hop` must execute in the **shell process**, not in a child, because the shell's CWD is what needs to change.

If you do:

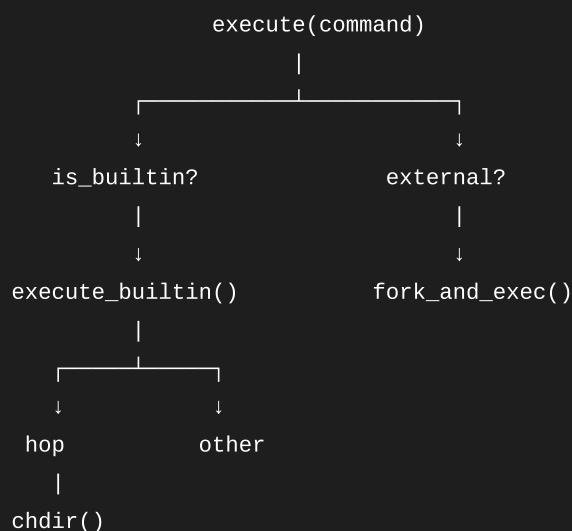
```
shell
|
fork()
|
child → chdir("/somewhere")
|
exits
```

the parent's CWD remains unchanged.

So I'd structure the executor around a deliberate distinction between **shell builtins** and **external commands**, rather than trying to make `hop` fit the `fork → exec` path.

A clean architecture

Think of your executor as having a dispatcher:



So something like:

```
void execute_command(ShellState *state, Command *cmd)
{
    if (is_builtin(cmd)) {
        execute_builtin(state, cmd);
    } else {
        execute_external(cmd);
    }
}
```

Then:

```
void execute_builtin(ShellState *state, Command *cmd)
{
    if (strcmp(cmd->argv[0], "hop") == 0) {
        builtin_hop(state, cmd);
    }
    // other builtins...
}
```

and separately:

```
void execute_external(Command *cmd)
{
    pid_t pid = fork();

    if (pid == 0) {
        execvp(cmd->argv[0], cmd->argv);
        _exit(127);
    }

    waitpid(pid, NULL, 0);
}
```

That keeps the special behavior **contained**.

Why `ShellState *` is particularly useful here

Your `hop` implementation needs access to things like:

```
typedef struct {
    char *home_dir;
    char *previous_dir;
    int should_exit;
    // frequency state...
} ShellState;
```

So:

```
builtin_hop(state, cmd);
```

can do:

```
current CWD
  ↓
resolve hop argument
  ↓
chdir()
  ↓
update state->previous_dir
  ↓
update frecency
```

The external executor doesn't need any of that.

There's another important consequence

You should generally execute **state-changing builtins in the parent**, but that doesn't mean *every* builtin necessarily has to be implemented differently.

For example, if you eventually have:

```
hop
exit
```

both need to affect the shell itself:

```
hop → changes ShellState/CWD
exit → changes ShellState/terminates shell loop
```

So both belong in the parent-side builtin path.

Whereas:

```
reveal
echo
```

if they're shell intrinsics but don't need to modify persistent shell state, can potentially be implemented directly in the parent too.

And since your assignment explicitly says Part B bans `exec*`, your builtins **shouldn't be going through `exec` anyway**.

What about pipes/background execution?

This is where you want to be careful not to over-generalize.

For a simple command:

```
hop /tmp
```

you can do:

```

parse
  ↓
builtin?
  ↓ yes
builtin_hop()    ← parent
  ↓
prompt

```

But if the grammar later allows:

```
hop /tmp | something
```

you can't simply execute `hop` in the parent as part of a pipeline, because pipeline commands need their own processes. A `chdir()` in a pipeline child wouldn't change the shell's CWD.

Whether your assignment permits/defines such combinations will determine the exact semantics. **Don't try to solve that complication until the relevant execution section requires it.**

For the normal standalone builtin case, the clean separation is enough.

So your `exec.c` doesn't need to become messy

I'd actually consider naming the module `executor.c` rather than `exec.c`, because you're going to have both:

```

executor.c
├─ execute_command()
├─ execute_builtin()
└─ execute_external()

builtins.c
├─ builtin_hop()
└─ other builtins

process.c (optional)
├─ fork
├─ exec
├─ wait
└─ pipe/redirection helpers

```

Then the responsibilities are very clear:

```

executor
├─ builtin → builtins.c → runs in shell process
└─ external → process.c → fork/exec/wait

```

That's **much cleaner than putting a special** `if (strcmp(..., "hop"))` **somewhere inside your generic fork/exec function.**

And your intuition about `hop` is exactly right: **the shell's working directory is process state, so the code that changes it has to run in the shell process itself.**



i'm trying to implement the frecency fallback now and i'm not sure what the simplest reasonable design is. the spec lets us choose the scoring formula and storage format, but it has to persist across shell sessions and consider both frequency and recency. would it make sense to keep a file containing each absolute directory path along with something like visit count and last-visited timestamp, calculate a deterministic score from those whenever a fallback lookup happens, and then pick the highest-scoring path containing the requested name?



Yes — **that's probably the simplest reasonable design**, and it fits the spec very cleanly. I wouldn't make it more complicated than that.

1. Persist exactly these three things

For every successfully visited directory:

```
absolute_path
visit_count
last_visited
```

For example, conceptually:

```
/home/somsuta/projects|12|1756059000
/home/somsuta/downloads|5|1756057000
/tmp/test|2|1756040000
```

Use an absolute path because the meaning of a frecency entry shouldn't depend on the shell's current directory.

A corresponding C structure could be:

```
typedef struct {
    char *path;
    unsigned long visits;
    time_t last_visited;
} FrecencyEntry;
```

You can keep an array/list of these in memory while the shell runs.

2. Update it only after a successful hop

Suppose:

```
hop /home/somsuta/projects
```

succeeds.

Then:

```
visits++;  
last_visited = time(NULL);
```

If the path isn't already in the database, create:

```
visits = 1  
last_visited = now
```

Then write the updated data back to the persistent file.

This means your database represents **successful directory visits**, which is exactly what the requirement asks for.

3. Use a very simple deterministic score

You need both frequency and recency to matter.

For example:

```
score = visits / (1 + age_in_hours)
```

where:

```
age_in_hours = (current_time - last_visited) / 3600.0
```

That's enough.

So:

```
A: visits = 10, age = 1 hour  
score = 10 / 2 = 5
```

```
B: visits = 3, age = 0.1 hour  
score = 3 / 1.1 ≈ 2.73
```

A wins.

But if B is much more recent:

```
A: visits = 10, age = 20 hours  
score ≈ 0.476
```

```
B: visits = 3, age = 0.1 hours  
score ≈ 2.73
```

B wins.

Thus **frequency and recency genuinely affect the result**.

You could use a different formula, but I'd avoid anything fancy. Your assignment explicitly gives you freedom here.

4. Fallback lookup becomes very straightforward

For:

```
hop osn
```

do:

```
try direct path "osn"
  |
  |— succeeds → done
  |
  |— fails
      ↓
    iterate frecency entries
      ↓
    path contains "osn"?
      ↓ yes
    directory still exists?
      ↓ yes
    calculate score
      ↓
    keep highest
      ↓
    chdir(best_path)
```

Important: **check that the path still exists before selecting it.**

The spec explicitly says stale entries should be skipped.

I'd use something like:

```
struct stat st;

if (stat(entry.path, &st) == 0 && S_ISDIR(st.st_mode)) {
    // candidate
}
```

Then you don't have to delete stale entries immediately. You can optionally clean them up later.

5. What does "contains name" mean?

The spec says:

whose path contains "name" as a substring

So this is a **literal substring check**, not a path-component check.

If the query is:

```
proj
```

then all of these potentially match:

```
/home/user/project
/home/user/projects
/home/user/myproject
/tmp/project-backup
```

You can simply use:

```
if (strstr(entry.path, name) != NULL)
```

That's actually simpler than your earlier home-directory ancestry check.

6. Handle ties deterministically

This is an easy thing to overlook.

If two directories have exactly the same score, don't let the result depend on some accidental ordering.

For example, define:

```
higher score wins
if scores equal:
    more recent wins
if still equal:
    lexicographically smaller path wins
```

Then the algorithm is completely deterministic.

You could even just use:

```
score equal → lexicographically smaller path
```

since `last_visited` is already part of your score.

7. Storage format

For a small assignment, **don't build a database**. A plain text file is perfectly reasonable.

Something like:

```
/path/one\t12\t1756059000
/path/two\t5\t1756057000
/path/three\t2\t1756040000
```

Using tabs as separators is convenient.

On startup:

```
frecency file
↓
load into memory
```

During execution:

```
successful hop
  ↓
update in-memory entry
  ↓
save file
```

You can either rewrite the whole file after every successful hop or save periodically. For this assignment, **rewriting the tiny file after every successful hop is probably the simplest and safest.**

One design I'd recommend

Keep frecency separate from `ShellState` if it starts getting substantial:

```
typedef struct {
    char *home_dir;
    char *previous_dir;
    int should_exit;
} ShellState;

typedef struct {
    char *path;
    unsigned long visits;
    time_t last_visited;
} FrecencyEntry;

typedef struct {
    FrecencyEntry *entries;
    size_t count;
    size_t capacity;
} FrecencyDB;
```

Then:

```
hop
|
├─ ShellState → home_dir / previous_dir
|
└─ FrecencyDB → persistent directory history
```

And your functions can have very clear responsibilities:

```
frecency_load()
frecency_save()
frecency_record(path)
frecency_find(query)
frecency_score(entry)
```

Then `hop` itself mostly orchestrates things.

So yes: your proposed (absolute path, visit count, last-visited timestamp) + deterministic score + persistent file **design is exactly where I'd start.** It's simple enough to implement reliably, while satisfying every part of the frequency requirement without trying to recreate zoxide.



i'm trying to make sure the direct path and frecency logic interact correctly. my current idea is to first try the argument directly with ``chdir()``, and only if that fails do i search the stored frecency entries for paths containing that name. but i'm unsure what should happen with things like absolute paths, paths that exist but aren't directories, and paths where ``chdir()`` fails for some reason other than not existing. should every failed direct ``chdir()`` trigger the frecency fallback, or should i distinguish between different failure cases?



I'd keep the rule **very simple: try the argument directly, and if the direct `chdir()` fails, perform the frecency fallback**. You don't need to distinguish every possible `errno` for this assignment.

The important distinction is that **frecency is only for the "name" case**, not for special arguments like `\~`, `.`, `..`, or `-`.

For a normal `name`

Suppose:

```
hop projects
```

Do:

```
"projects"
  ↓
chdir("projects")
  ↓
success? — yes → done
          |
          no
          ↓
search frecency entries containing "projects"
```

That naturally handles all of these:

Absolute path

```
hop /home/user/projects
```

`chdir()` understands it directly.

- Exists and is a directory → success.
- Doesn't exist → fallback search for the **literal string** `/home/user/projects` would probably find nothing.
- Is a file → `chdir()` fails; fallback is technically allowed by your simple rule, but likely finds nothing.

There's no need to special-case absolute paths.

Relative path

```
hop ../projects
hop src
hop ./src
```

Same thing: let `chdir()` decide whether it resolves.

What about a path that exists but isn't a directory?

For:

```
hop notes.txt
```

if `notes.txt` is a regular file, `chdir()` fails with `ENOTDIR`.

At that point, I'd still follow the assignment's basic rule:

If “name” does not resolve directly, fall back to a frecency lookup.

So you can search the frecency database.

It probably won't find a useful directory anyway, because your database should contain **directories that were successfully hopped into**, not files.

What about permission errors?

Suppose:

```
hop secret
```

and `secret` exists but you don't have permission to enter it.

`chdir()` fails with something like `EACCES`.

I would **still treat that as “direct resolution failed” and attempt frecency**, rather than trying to build a complicated distinction around `errno`.

The assignment doesn't say:

Fall back only if the path doesn't exist.

It says:

If “name” does not resolve directly, fall back to a frecency lookup.

So your implementation can simply use the return value:

```
if (chdir(name) == 0) {
    // direct success
    record_successful_hop(...);
} else {
```

```

    // direct resolution failed
    // try frecency
}

```

One subtle issue: don't use `chdir()` as your *only* test if you need `errno`

You might want to save `errno` immediately if you're going to inspect it:

```

if (chdir(name) != 0) {
    int err = errno;

    // frecency fallback
}

```

But unless the assignment requires a different error for permission failures, you don't need to do anything with `err`.

I'd structure `hop` like this

```

hop(argument)
├── |
├── ├── "~" → home
├── ├── "." → no-op
├── ├── ".." → parent
├── ├── "-" → previous directory
├── |
├── └── name
│   ├── |
│   ├── ├── chdir(name)
│   ├── │   ├── |
│   ├── │   └── success → record + done
│   ├── |
│   └── └── failure
│       ├── |
│       ├── ├── search frecency for paths containing name
│       ├── |
│       ├── ├── candidate exists and is directory?
│       ├── │   ├── yes → chdir(candidate)
│       ├── │   └── no → next candidate
│       ├── |
│       └── └── none → "hop: no such directory"

```

That also handles a nice case:

```
hop nonexistent
```

If there's a frecency entry:

```
/home/user/projects/nonexistent-old
```

and it still exists, you can jump there if it contains `"nonexistent"`.

One thing I would not do

Don't make the fallback behavior depend on whether the argument *looks like* a path:

```
if (strchr(name, '/') != NULL)
    // don't fallback
```

That's unnecessary and could conflict with the spec. The specification deliberately says “**name**” can be a relative or absolute path for the direct lookup, and then says to fall back if it doesn't resolve.

So the clean abstraction is simply:

Direct lookup gets first priority. If it fails, freecy gets a chance.

That keeps the implementation deterministic and avoids guessing why `chdir()` failed.



B2: reveal [10]

Syntax: ``reveal` (-a|t)* (~ | . | .. | - | name)?``

Purpose: The reveal command lists the files and directories in a directory.

Requirements

1. If no argument is passed, ``reveal`` lists the contents of the current working directory.
2. "a": reveal all files and directories, including hidden ones (starting with ``.``). Default is to hide them.
3. "t": recursively reveal the contents of all subdirectories. the content of the subdirectory must be displayed after the directory itself (refer example below).
4. When both flags are set, recursively show all files, hidden included, in the format given below.
5. When neither flag is set, print entries in the format of ``ls``.
6. The argument passed resolves the same way as hop's ``~``, ``.``, ``.``, ``.``, ``.``, and direct path cases, except reveal lists directory contents instead of changing the CWD. There is no frequency fallback: if the argument does not resolve to an existing directory, print ``reveal: no such directory``.
7. Entries are always listed in lexicographic order, sorted by ASCII value. When the ``-t`` flag is passed, entries within each directory must also be listed in lexicographic order.
8. If reveal is passed too many arguments, print: ``reveal: invalid syntax``
9. If reveal is passed an invalid flag, print: ``reveal: invalid syntax``
10. If ``reveal -`` is run before any hop has been run since shell start, print: ``reveal: no such directory``

Example

```
```
<goatam@iiit:~> reveal ~
osnmp1
<goatam@iiit:~> hop ..
<goatam@iiit:/home> reveal
goatam
<goatam@iiit:/home> hop goatam/osnmp1
<goatam@iiit~/osnmp1> reveal -t
Makefile
README.md
include/
include/parser.h
include/shell.h
shell.out
src/
```

```

src/main.c
src/parser.c
<goatam@iiit:~/osnmp1> reveal -ta
Makefile
README.md
include/
include/parser.h
include/shell.h
shell.out
src/
src/main.c
src/parser.c
<goatam@iiit:~/osnmp1> reveal -ta -tttttaaaaaaaaaatttt -aaaa -tttt
Makefile
README.md
include/
include/parser.h
include/shell.h
shell.out
src/
src/main.c
src/parser.c
` `

```

for the sorting, `readdir()` obviously doesn't guarantee the order i want, so i'm thinking of storing all the directory entries in an array first and then using `qsort()` with a comparator based on `strcmp()`. i'm a little unsure about the memory management though — since every `readdir()` result points into internal directory storage, do i need to copy the filenames into my own allocated strings before closing the directory? and if i'm recursively calling the same function for subdirectories, how should i manage/free all those entries without leaking memory?



Yep — **copy the filenames into your own storage**. Your understanding of `readdir()` is right.

`readdir()` returns a pointer to a `struct dirent` whose contents are managed by the `DIR *` stream. You should **not keep those pointers around after subsequent `readdir()` calls or after `closedir()`**.

For a small shell, i'd make each directory listing own its own strings.

## A simple design

Something like:

```

typedef struct {
 char *name;
 bool is_dir;
} Entry;

```

Then while reading:

```
struct dirent *de;

while ((de = readdir(dir)) != NULL) {
 if (!show_hidden && de->d_name[0] == '.')
 continue;

 entries[count].name = strdup(de->d_name);
 entries[count].is_dir = (de->d_type == DT_DIR);
 count++;
}
```

Then:

```
closedir(dir);

qsort(entries, count, sizeof(Entry), compare_entries);
```

Now the array is completely independent of `readdir()`'s internal storage.

## Memory ownership becomes very clean

Your listing function owns:

```
entries
├─ entries[0].name → malloc'd
├─ entries[1].name → malloc'd
├─ entries[2].name → malloc'd
└─ ...
```

So after you're done with that directory:

```
for (size_t i = 0; i < count; i++)
 free(entries[i].name);

free(entries);
```

That's it.

I'd actually make a helper:

```
void free_entries(Entry *entries, size_t count)
{
 for (size_t i = 0; i < count; i++)
 free(entries[i].name);

 free(entries);
}
```

Then every call that allocates an entry array has one obvious cleanup point.

## For recursion, don't share the array

This is the part that makes recursion much easier.

Suppose:

```
project/
├─ Makefile
├─ include/
│ ├─ parser.h
│ └─ shell.h
└─ src/
 ├─ main.c
 └─ parser.c
```

Your recursive function can conceptually do:

```
reveal_directory("project/")
|
├─ allocate entries for project
├─ read + sort entries
├─ print entries
|
├─ reveal_directory("project/include/")
| ├─ allocate entries for include
| ├─ read + sort
| ├─ print
| └─ free include entries
|
├─ reveal_directory("project/src/")
| ├─ allocate entries for src
| ├─ read + sort
| ├─ print
| └─ free src entries
|
└─ free project entries
```

Each recursive invocation has **its own local** `entries` **array**.

So you don't have to maintain one giant global collection of every file you've encountered.

## Something like

```
void reveal_directory(const char *path, bool show_hidden, bool recursive)
{
 Entry *entries = NULL;
 size_t count = 0;

 // read entries
 // sort entries
 // print entries
```

```

if (recursive) {
 for (size_t i = 0; i < count; i++) {
 if (entries[i].is_dir) {
 // construct child path
 reveal_directory(child_path, show_hidden, true);
 }
 }
}

free_entries(entries, count);
}

```

The crucial thing is that **you free the current directory's entries after all recursive calls that need them have finished.**

### One important wrinkle: `d_type`

I'd be slightly careful about:

```
de->d_type == DT_DIR
```

On some filesystems, `d_type` can be `DT_UNKNOWN`. So if your assignment expects robust behavior, use `stat()` / `lstat()` when necessary to determine whether an entry is actually a directory.

For example:

```

d_type == DT_DIR
 → definitely directory

d_type == DT_UNKNOWN
 → stat(path) to determine

```

Also, don't blindly recurse into `.` and `..`. You're already filtering hidden files depending on `-a`, but `.` and `..` are special and should **not** cause recursive traversal.

### `strcmp()` vs ASCII sorting

Your instinct here is also good.

If they literally want lexicographic order by ASCII value, a comparator based on `strcmp()` is appropriate:

```

int compare_entries(const void *a, const void *b)
{
 const Entry *ea = a;
 const Entry *eb = b;

 return strcmp(ea->name, eb->name);
}

```

`strcmp()` compares the unsigned-character values in lexicographic order, so things like:

```
A
B
a
b
```

will sort according to their character codes rather than case-insensitively.

## One design choice I'd make

Don't store full paths in every `Entry`. Just store:

```
name
is_directory
```

The recursive function already knows:

```
current_directory = /home/user/project/src
```

so when it needs to recurse into `parser.c`'s parent directory etc., construct the child path temporarily:

```
current_path + "/" + entry.name
```

That keeps your entry structure tiny.

So overall:

```
readdir()
↓
copy d_name with strdup()
↓
Entry[]
↓
qsort()
↓
print
↓
recursively process directories
↓
free every name
↓
free array
```

That's a very manageable ownership model, and **each recursive call cleans up exactly the memory it allocated**. That is much less error-prone than trying to keep all `readdir()` pointers alive across recursion.



for recursive reveal, i'm confused about how to distinguish a directory entry from something that just happens to look like a directory. i'm guessing i should use `lstat()` / `stat()` on the constructed full path after getting each entry from `readdir()`, but i'm not sure which one makes more sense if there are symbolic links. also, when recursively building paths like `parent/subdir/file`, what's a safe way to construct the path without getting the same `snprintf` truncation warnings i had earlier?



Yeah — for recursive `reveal`, i'd use `lstat()` **rather than** `stat()` if you want symlink behavior to be predictable.

`stat()` **VS** `lstat()`

The distinction is:

```
stat("foo")
 ↓
follows symlink → tells you what foo ultimately points to

lstat("foo")
 ↓
doesn't follow symlink → tells you that foo itself is a symlink
```

So imagine:

```
project/
├─ real_dir/
└─ link → real_dir/
```

With `stat("project/link")`:

```
S_ISDIR(...) == true
```

because `link` points to a directory.

With `lstat("project/link")`:

```
S_ISLNK(...) == true
```

because the entry itself is a symlink.

**Which should you use for `reveal -t`?**

i'd use `lstat()` and only recurse when:

```
S_ISDIR(st.st_mode)
```

is true.

That means:

```
real_dir/ → recurse
link → don't recurse
```

This is generally safer because otherwise a symlink to a directory could cause recursive traversal outside the directory tree you're currently displaying, and symlink cycles can make recursion nasty:

```
a/
└─ b → ../a
```

With `stat()`, you could end up recursively following that forever.

So your logic can be:

```
struct stat st;

if (lstat(full_path, &st) == -1) {
 // couldn't inspect it
} else if (S_ISDIR(st.st_mode)) {
 // safe candidate for recursion
}
```

You can still use `de->d_type == DT_DIR` as a quick optimization, but `lstat()` gives you consistent behavior when `d_type == DT_UNKNOWN`.

---

## Constructing recursive paths

And yes, this is where blindly doing:

```
char path[1024];

snprintf(path, sizeof(path), "%s/%s", parent, name);
```

can trigger the truncation warning.

The cleanest approach for your shell is probably **dynamic allocation based on the actual lengths**.

```
size_t len = strlen(parent) + 1 + strlen(name) + 1;

char *child_path = malloc(len);

if (child_path == NULL) {
 // handle allocation failure
}

snprintf(child_path, len, "%s/%s", parent, name);
```

Here the compiler can see that the buffer is large enough because you've explicitly calculated the required size.

You can actually avoid `snprintf()` entirely:

```
size_t parent_len = strlen(parent);
size_t name_len = strlen(name);

char *child_path = malloc(parent_len + 1 + name_len + 1);

if (child_path == NULL) {
 // handle error
}

memcpy(child_path, parent, parent_len);
child_path[parent_len] = '/';
memcpy(child_path + parent_len + 1, name, name_len);
child_path[parent_len + 1 + name_len] = '\\0';
```

But honestly, that's more code than you need.

## I'd use a helper

Since you're going to construct paths **everywhere** in `reveal`, make it one utility:

```
char *join_path(const char *parent, const char *name)
{
 size_t len = strlen(parent) + 1 + strlen(name) + 1;

 char *result = malloc(len);
 if (result == NULL)
 return NULL;

 snprintf(result, len, "%s/%s", parent, name);
 return result;
}
```

Then recursion becomes much cleaner:

```
char *child_path = join_path(path, entries[i].name);

if (child_path == NULL) {
 // handle allocation failure
 continue;
}

struct stat st;

if (lstat(child_path, &st) == 0 && S_ISDIR(st.st_mode)) {
 reveal_directory(child_path, show_hidden, true);
}
```

```
free(child_path);
```

And the ownership is very obvious:

```
join_path()
 ↓
malloc
 ↓
caller owns child_path
 ↓
recursive call
 ↓
free(child_path)
```

## One small edge case

If `parent` can be `/`, blindly doing:

```
"/" + "/" + "foo"
```

gives:

```
//foo
```

which usually works on Unix, but is ugly and can matter in string comparisons.

So I'd make `join_path()` handle that:

```
char *join_path(const char *parent, const char *name)
{
 bool root = strcmp(parent, "/") == 0;
 size_t len = strlen(parent) + (root ? 0 : 1) + strlen(name) + 1;

 char *result = malloc(len);
 if (result == NULL)
 return NULL;

 if (root)
 snprintf(result, len, "%s", name);
 else
 snprintf(result, len, "%s/%s", parent, name);

 return result;
}
```

Then:

```
join_path("/home/user", "src")
→ /home/user/src
```

```
join_path("/", "home")
→ /home
```

So I'd go with `lstat()` + **dynamically allocated** `join_path()`. It keeps the recursion safe around symlinks and avoids fixed-size path buffers entirely.



i'm trying to handle ``-a``, ``-t``, and combinations like ``-ta -aaaa -tttt`` without making the reveal argument parsing messy. my current idea is to scan every argument starting with ``-``, then scan each character after it and just set boolean flags when i see ``a`` or ``t``. but then i need to distinguish that from an actual ``-`` argument, which means ``reveal -`` should be treated as the previous-directory target rather than a flag. is there a clean way to structure that parsing so repeated flags don't cause problems and invalid characters like ``-x`` can be rejected?



Yes — your approach is good. The cleanest thing is to treat `-` **as a special non-flag argument first**, and only treat an argument as a flag bundle if it starts with `-` **and has at least one character after it**.

Given the syntax:

```
reveal (-(a|t)*)* (\~ | . | .. | - | name)?
```

I'd parse it in two phases.

## 1. Scan all leading flag arguments

For each argument:

```
arg == "-"
 → this is the target "previous directory"
 → stop interpreting flags

arg starts with "-" and length > 1
 → flag bundle
 → inspect every character after '-'

otherwise
 → this is the target
```

So:

```
reveal -ta -aaaa -tttt
```

becomes:

```
-ta → a=true, t=true
-aaaa → a=true
-tttt → t=true
```

Repeated flags simply do nothing extra because you're assigning booleans:

```
show_all = true;
```

```
recursive = true;
```

You don't need counters.

## 2. Validate each flag character

Something like:

```
for (size_t j = 1; arg[j] != '\0'; j++) {
 if (arg[j] == 'a') {
 show_all = true;
 } else if (arg[j] == 't') {
 recursive = true;
 } else {
 printf("reveal: invalid syntax\n");
 return;
 }
}
```

So:

```
-aaa → valid
-ttttt → valid
-atatata → valid
-a → valid
-t → valid

-x → invalid
-ax → invalid
-- → invalid
-aX → invalid
```

## The - case

This is the important special case:

```
if (strcmp(arg, "-") == 0) {
 // target = previous directory
} else if (arg[0] == '-') {
 // flag bundle
} else {
 // target
}
```

Notice that "-" is checked **before** `arg[0] == '-'`.

Then:

```
reveal -
```

means:

```
flags: none
target: previous directory
```

while:

```
reveal -a
```

means:

```
flags: a
target: current directory
```

And:

```
reveal -ta -
```

means:

```
flags: t + a
target: previous directory
```

## One more useful observation from the grammar

Your syntax has:

```
(-(a|t)*)*
```

which means **all flag bundles come before the optional target**.

So I'd actually stop flag parsing as soon as you encounter a non-flag target.

For example:

```
reveal -ta src
```

→ flags `a,t`, target `src`.

But:

```
reveal src -ta
```

should **not** interpret `-ta` as another flag bundle after the target. It should be an additional argument and therefore trigger:

```
reveal: invalid syntax
```

That's an easy mistake if you just scan every argument independently.

So I'd structure the parser roughly:

```

i = 1

while i < argc:
 if argv[i] == "-":
 target = "-"
 i++
 break

 if argv[i] starts with "-":
 validate flag characters
 set flags
 i++
 continue

 target = argv[i]
 i++
 break

if i != argc:
 invalid syntax

```

This naturally enforces:

- arbitrary repeated `-a / -t` bundles
- only `a` and `t` are accepted
- `-` is a target, not a flag
- at most one target
- flags must occur before the target

And it matches the examples like:

```
reveal -ta -ttttttaaaaaaaaaatttt -aaaa -tttt
```

perfectly: all four arguments are flag bundles, so you end with `a=true`, `t=true`, and no target → current directory.



i'm trying to deal with hidden files and recursion together, and i'm not sure where the ``-a`` check should happen. should i filter out entries beginning with ``.`` while building the sorted list, or should i keep everything in the list and decide whether to print/recurse into each entry later? especially with ``-ta``, i want hidden directories to actually be traversed too, not just hidden files printed, so i'm wondering what the cleanest place for that condition is.



Yeah — I would **keep the entries in the list and make the `-a` decision when you're processing them**, rather than filtering them out during collection.

That's especially important for `-ta`, because a hidden directory must be available for recursion.

Think of it as three separate stages:

```
readdir()
 ↓
collect ALL entries
 ↓
sort ALL entries
 ↓
for each entry:
 should this entry be shown?
 should this entry be recursed into?
```

## Why filtering during collection is awkward

Suppose:

```
project/
├─ visible/
│ └─ a.c
└─ .hidden/
 └─ secret/
 └─ x.txt
```

With:

```
reveal -ta
```

you want:

```
.hidden/
.hidden/secret/
.hidden/secret/x.txt
```

If you filtered `.hidden` out immediately because it starts with `.`, you've lost the thing you need to recurse into.

You'd end up needing special logic like:

*"Don't show this hidden entry, but remember it anyway because maybe I need to recurse."*

Much cleaner to just keep it.

---

## Separate "print?" and "recurse?"

For each entry, think:

```
bool hidden = entry.name[0] == '.';

bool should_print = show_all || !hidden;
```

Then for recursion:

```
if (recursive && is_directory && !is_dot_or_dotdot) {
 recurse(...);
}
```

**Notice that `show_all` doesn't necessarily belong in the recursion condition.**

If `-t` is set, you recurse into directories.

If `-a` is additionally set, hidden directories are also eligible for recursion.

So:

```
-t:
 visible directories → recurse
 hidden directories → don't recurse

-ta:
 visible directories → recurse
 hidden directories → recurse
```

while:

```
-a:
 hidden files/directories → print
 no recursion
```

## One special exception: `.` and `..`

This is the important bit.

`readdir()` can return:

```
.
..
```

They are hidden by the usual `.` rule, but **you absolutely don't want** `reveal -ta` **to recurse into them.**

Otherwise:

```
.
└─ .
 └─ .
 └─ ...
```

So distinguish:

```
bool dot_entry =
 strcmp(entry.name, ".") == 0 ||
 strcmp(entry.name, "..") == 0;
```

Then your logic becomes roughly:

```
for each sorted entry {

 bool hidden = entry.name[0] == '.';
 bool dot_entry = strcmp(entry.name, ".") == 0 ||
 strcmp(entry.name, "..") == 0;

 if (show_all || !hidden)
 print_entry(entry);

 if (recursive && is_directory && !dot_entry) {
 recurse_into(entry);
 }
}
```

That gives you the desired behavior naturally.

**One subtle question: should hidden directories be traversed without `-a`?**

Based on the requirement:

*“t: recursively reveal the contents of all subdirectories.”*

and:

*“a: reveal all files and directories, including hidden ones.”*

I'd interpret `-t` as **respecting the normal visibility rule at every level**, so a hidden directory isn't traversed unless `-a` is present.

Thus:

```
reveal -t
```

doesn't enter `.git/`.

But:

```
reveal -ta
```

does enter `.git/` and shows its contents.

That also matches the example's statement:

*“When both flags are set, recursively show all files, hidden included...”*

So I'd implement `-a` as affecting **both visibility and recursive traversal**, while keeping the two decisions as separate conditions in your code.

And sorting **before** applying the visibility filter is nice because your recursion always operates on a consistently sorted list, which makes the `-t` output deterministic.



for the `reveal -` case, i don't want to actually call `hop -` because reveal isn't supposed to change the cwd. i'm thinking i should reuse whatever previous-directory state `hop` maintains and get the corresponding path directly, then call `opendir()` on it. but i'm not sure how to share that state cleanly between `hop.c` and `reveal.c`, especially when no hop has happened yet and there isn't a previous directory.



Yes — **don't call** `hop -` **from** `reveal`. Your instinct is right. `hop -` has a side effect (`chdir()`), whereas `reveal -` only needs to **resolve the same target**.

The cleanest design is to make the previous-directory path part of your shared `ShellState`.

```
typedef struct {
 char *home_dir;
 char *previous_dir;
 int should_exit;
} ShellState;
```

Then both builtins receive the same state:



`hop -`

`hop` does:

```
if (state->previous_dir == NULL) {
 // do nothing
} else {
 chdir(state->previous_dir);
}
```

When a **real directory change** occurs, it updates `previous_dir`.

For example:

```
CWD = /a
previous = NULL

hop /b
```

```
CWD = /b
previous = /a
```

Then:

```
reveal -
```

can simply look at:

```
state->previous_dir
```

without changing anything.

```
reveal -
```

Its logic becomes:

```
previous_dir exists?
|
├─ no → "reveal: no such directory"
|
└─ yes
 ↓
 opendir(previous_dir)
 ↓
 list it
```

So:

```
void reveal_previous(const ShellState *state, ...)
{
 if (state->previous_dir == NULL) {
 printf("reveal: no such directory\n");
 return;
 }

 reveal_directory(state->previous_dir, ...);
}
```

Notice the `const` :

```
const ShellState *state
```

That's a nice signal that `reveal` is **only reading** shell state.

## Don't make `previous_dir` a global

You could technically have:

```
char *previous_dir;
```

shared between `hop.c` and `reveal.c`, but that's exactly the sort of global state you've been trying to avoid. Instead:

```
// shell.h

typedef struct {
 char *home_dir;
 char *previous_dir;
 ...
} ShellState;
```

Then:

```
// hop.c
void builtin_hop(ShellState *state, ...);
```

and:

```
// reveal.c
void builtin_reveal(const ShellState *state, ...);
```

Both operate on the **same instance created in** `main()`.

---

### One subtle point: what counts as "previous"?

You want `previous_dir` to mean:

***the directory I was in immediately before the last successful directory change***

not:

***the last directory that was ever visited.***

So if you're at:

```
/a
```

and do:

```
hop /b
```

you get:

```
CWD = /b
previous = /a
```

Then:

```
hop .
```

shouldn't turn `previous_dir` into `/b`, because `.` didn't actually change the directory.

Likewise, `hop ..` at `/` shouldn't manufacture a previous directory of `/`.

That makes `reveal -` behave consistently too.

## And this gives you a nice shared abstraction

You could even have a helper:

```
const char *get_previous_dir(const ShellState *state);
```

but honestly, for a project this size, directly accessing:

```
state->previous_dir
```

is probably clearer.

The important architectural principle is:

`hop` **owns the logic for updating the state;** `reveal` **consumes the state without invoking** `hop` .

That way `reveal -` remains a pure “resolve and list” operation and never accidentally changes the shell's CWD.



for multiple files with ``-r``, i'm a little unsure where the separation should happen. if i do ``peek -r file1 file2``, i need to reverse file1, print it, then separately reverse file2, rather than concatenating both files and reversing the whole thing. so should the main function basically loop over each filename and call the reverse-reading function independently, while the numbering state for ``-n`` is either reset or continued depending on what the assignment means?



Yes — **that separation is the cleanest design**. For something like:

```
peek -r file1 file2
```

you generally want:

```
parse arguments
 ↓
for each file:
 open file
 process/reverse that file
 print its result
 close file
```

rather than:

```
open file1
open file2
concatenate contents
reverse everything
```

because `-r` is a transformation of **each input file**, not of the combined input stream.

So conceptually:

```
for (each filename) {
 process_file(filename, flags);
}
```

and inside:

```
process_file(...)
{
 if (reverse)
 reverse_file(...);
 else
 normal_file(...);
}
```

## About `-n`

This is the one thing I'd **not guess** without seeing the exact `peek` specification.

There are two possible semantics:

### Reset numbering per file:

```
peek -n file1 file2
```

```
file1:
```

```
1 ...
```

```
2 ...
```

```
file2:
```

```
1 ...
```

```
2 ...
```

### or **continue numbering across files:**

```
file1:
```

```
1 ...
```

```
2 ...
```

```
file2:
```

```
3 ...
```

```
4 ...
```

The right choice depends entirely on how the assignment defines `-n` and multiple input files.

Architecturally, though, you can make either behavior easy by having the caller own the numbering state:

```
int line_number = 1;

for each file {
 process_file(file, ..., &line_number);
}
```

for continuous numbering, or:

```
for each file {
 int line_number = 1;
 process_file(file, ..., &line_number);
}
```

for resetting.

And for `-r`, I'd keep the reversal logic **inside** `process_file()`, so the outer loop doesn't need to care whether you're reading normally or backwards:

```
process_file(file1)
 ↓
reverse file1
```

```
↓
print

process_file(file2)
↓
reverse file2
↓
print
```

That separation will also make combinations like `-rn` much easier later.

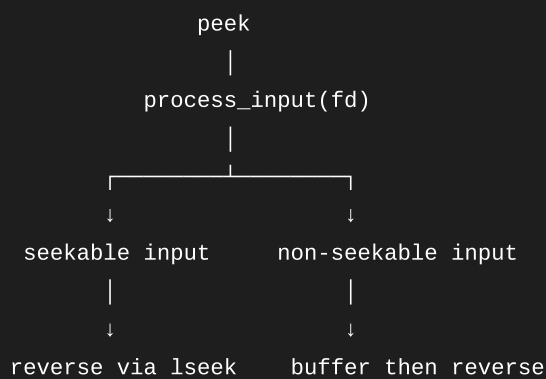


i'm trying to keep the implementation consistent for regular files, stdin, and pipes. for a regular file i need to use `lseek` for `-r`, but stdin isn't seekable, so the spec allows buffering it completely. would it be cleaner to have two separate implementations — one that reads a seekable fd backwards and another that buffers non-seekable input — and then have the main `peek` logic just decide which one to call based on whether `lseek` works?



Yes — **that's a good design**, and I'd actually recommend it. The important thing is to separate the “**what am I processing?**” decision from the “**how do I reverse this input?**” implementation.

Think of it as:



## For `-r`

For a regular file:

```
fd
↓
lseek(fd, ...)
↓
read backwards
↓
print lines in reverse
```

You don't need to buffer the whole file.

For stdin/a pipe:

```
fd
↓
lseek(fd, ...)
↓
fails with ESPIPE
↓
read entire input into memory
```

```
↓
reverse the buffered data
↓
print
```

That gives you two clean implementations:

```
void reverse_seekable(int fd, ...);
void reverse_buffered(int fd, ...);
```

Then something like:

```
off_t pos = lseek(fd, 0, SEEK_CUR);

if (pos != (off_t)-1) {
 reverse_seekable(fd, ...);
} else {
 reverse_buffered(fd, ...);
}
```

**But there's one subtle issue:** don't permanently alter the file offset just to test whether it's seekable.

A cleaner test is:

```
off_t pos = lseek(fd, 0, SEEK_CUR);

if (pos == (off_t)-1 && errno == ESPIPE) {
 // non-seekable
} else {
 // seekable
}
```

For a regular file, `SEEK_CUR` succeeds and leaves the offset unchanged.

For a pipe/stdin, it fails with `ESPIPE`.

## Even better abstraction

You could make the higher-level code completely unaware of the difference:

```
void process_fd(int fd, const PeekOptions *opts)
{
 if (opts->reverse) {
 if (is_seekable(fd))
 reverse_seekable(fd, opts);
 else
 reverse_buffered(fd, opts);
 } else {
 read_forward(fd, opts);
 }
}
```

Then all your input sources converge at the same point:

```
regular file ─┐
stdin ─────────┤──> fd ──> process_fd()
pipe ─────────┘
```

That's particularly nice because **a pipe doesn't need special treatment in the rest of** `peek`.

## One thing I'd avoid

Don't detect the source based on things like:

```
if (fd == STDIN_FILENO)
```

because stdin could itself be redirected:

```
peek -r < file.txt
```

or:

```
cat file.txt | peek -r
```

In both cases `STDIN_FILENO` is still `0`, but its underlying object is different.

Instead, **ask the file descriptor whether it is seekable**.

That's a much cleaner abstraction.

So yes: **two reverse implementations + one dispatch layer based on seekability** is exactly how I'd structure it. It also keeps your `-r` logic from becoming a giant function full of `if (is_pipe)`, `if (is_stdin)`, `if (is_regular_file)`, etc.



i'm trying to decide how much of `peek` should use low-level `read()`/`write()` versus stdio functions like `fgets()`. the reverse case seems to naturally require file descriptors and `lseek`, but normal mode and stdin would be much easier with buffered input. would mixing the two approaches cause any problems, especially when `peek` is later used inside a pipe where the input is no longer seekable?



Mixing them is totally fine. In fact, for `peek`, I'd **use the abstraction that fits each mode** rather than forcing everything through either `read()` or stdio.

The main thing to watch is **not mixing stdio buffering with raw `read()` on the same underlying file descriptor at the same time.**

## A clean split

I'd structure it like:

```
peek
|
├─ normal mode
| └─ read lines with read()/fgets()
|
└─ -r
 |
 ├─ seekable fd → lseek + read
 |
 └─ non-seekable fd → read/buffer + process
```

Honestly, I'd lean toward **using `read()` throughout the low-level implementation**, even for normal mode. Then you have one consistent input abstraction:

```
fd → read bytes → identify lines → print
```

and you don't have to worry about stdio buffering at all.

But if `fgets()` makes your normal-mode implementation substantially simpler, there's nothing inherently wrong with:

```
FILE *fp = fdopen(fd, "r");
fgets(...);
```

for normal mode and raw `read()` / `lseek()` for reverse mode.

## The important danger

Don't do this on the same fd:

```
FILE *fp = fdopen(fd, "r");

fgets(..., fp);

/* then */
read(fd, buffer, ...);
```

because `fgets()` may have buffered **more data than you consumed** from the kernel fd.

You could end up with:

```
kernel fd position
 ↓
 [already advanced]

stdio buffer
 ↓
 [contains data you haven't seen yet]
```

Then `read(fd, ...)` sees a different position from what you might expect.

So if you use stdio, keep the **entire processing of that fd** in stdio land.

## Pipes don't fundamentally change anything

For:

```
cat file.txt | peek
```

`peek` gets stdin as a pipe fd.

`fgets()` works perfectly on a pipe because it doesn't require seeking.

`lseek()` fails with `ESPIPE`, which is how you detect that `-r` needs the buffering implementation.

So:

```
normal:
pipe → fgets → output

-r:
pipe → lseek fails → read everything → reverse → output
```

works fine.

## One thing I'd recommend

Since you're already dealing with:

- regular files
- stdin
- pipes

- `-r`
- line numbering

I'd probably make your **core processing fd-based** and avoid `FILE *` entirely:

```
process_fd(fd, options)
|
|— normal → read lines
|
|— reverse
 |— seekable → lseek/read
 |— non-seekable → buffer/read
```

Then `fgets()` isn't necessary.

You can write a small `read_line_fd()` helper that accumulates bytes until `\n`. That sounds like more work initially, but it gives you one consistent representation for **file, stdin, and pipe**, and avoids the stdio/raw-FD interaction entirely.

But if the assignment doesn't make you implement your own line reader, **using `fgets()` for normal mode and `read()` for the separate reverse implementation is completely acceptable**. Just don't switch between them on the same fd during one operation.

So the rule I'd keep in your head is:

***Mixing APIs across different code paths is fine; mixing buffered stdio and raw `read()` on the same fd without carefully synchronizing them is what causes trouble.***



### B4: locate [10]

**Syntax:** ``locate filename+``

**Purpose:** The ``locate`` command returns the pathnames of the files (or links) which would be executed in the current environment, had its arguments been given as commands in a strictly POSIX-conformant shell.

**Requirements:**

1. For each filename, first check the current working directory. If a match exists there, print its absolute path first. Then search each directory listed in ``PATH``, in order, and print the absolute path of every match found. This means a name matching in both the cwd and two ``PATH`` entries prints three lines, in that order. Do not recursively search within the directories listed in ``PATH``.
2. The absolute path to the executable must be printed (even if the executable exists in the current working directory).
3. If no arguments are provided, print ``locate: invalid syntax``
4. If multiple arguments are provided, print the path for each one of them, in the order specified by the user.
5. If multiple matching executable paths are found, print all the paths.
6. If ``filename`` is not a valid executable, print: ``locate: command not found (commandname)`` and continue processing the remaining arguments.
7. You may assume that file paths will not be passed as arguments.

**Example:**

```
```\n<goatam@iiit:~> locate java\n/usr/bin/java\n<goatam@iiit:~> locate man java python nada\n/usr/bin/man\n/usr/bin/java\n/usr/bin/python\nlocate: command not found (nada)\n<goatam@iiit:~> locate sudo\n/usr/bin/sudo\n/bin/sudo\n```\n
```

bro i'm actually confused about the lookup order here. for every command am i supposed to manually check the current directory first, and only if that isn't an executable then split ``PATH`` and check every directory one by one? because i initially thought i could just use ``access(command, X_OK)`` and then do a PATH lookup, but the requirement that cwd has to come first and that i have to print every matching executable means i feel like i need to implement the whole

lookup myself. what would be the cleanest way to structure this without duplicating all the path checking logic?



Yeah — **you've caught an important difference between ordinary command lookup and what this `locate` requirement is asking you to do.**

You **cannot** just do:

```
access(command, X_OK)
```

and then stop at the first PATH match. `locate` explicitly wants **every executable match**, with the CWD checked **before** `PATH`.

The cleanest design is to separate:

1. **“Is this particular path an executable file?”**
2. **“Given a directory + filename, construct the candidate path and test it.”**
3. **“For one command, enumerate CWD then every PATH directory.”**

For each filename

Do exactly this:

```
locate java
|
├─ 1. current directory
|   └─ ./java
|       └─ executable? → print absolute path
|
└─ 2. PATH entries, in order
    └─ /usr/local/bin/java → executable? → print
        └─ /usr/bin/java    → executable? → print
            └─ /bin/java     → executable? → print
                └─ ...
```

Do not stop after finding one.

And importantly, the CWD check is **not conditional on PATH failing**. You always check it first, even if PATH will also contain matches.

I'd make one reusable helper

Something like:

```
bool is_executable(const char *path)
{
    struct stat st;
```

```

    if (stat(path, &st) != 0)
        return false;

    return S_ISREG(st.st_mode) && access(path, X_OK) == 0;
}

```

Then you don't duplicate the actual validity test.

The `S_ISREG()` part matters because you want an **executable file**, not merely something for which `access(..., X_OK)` happens to succeed.

Then another helper can construct and test a candidate:

```

check_candidate(directory, filename)
    ↓
directory + "/" + filename
    ↓
is_executable(path)?
    ↓ yes
make absolute path
print

```

CWD needs special handling

Suppose:

```

cwd = /home/somsuta/project
filename = myprog

```

The candidate is:

```

/home/somsuta/project/myprog

```

If it's executable, print:

```

/home/somsuta/project/myprog

```

not:

```

./myprog

```

That's why I'd get the absolute CWD once with `getcwd()` and construct the candidate from that.

So:

```

cwd_abs + "/" + filename

```

rather than relying on the relative string `filename`.

Then PATH is just another sequence of directories

Get:

```
char *path = getenv("PATH");
```

and split it on `:`.

For:

```
PATH=/usr/local/bin:/usr/bin:/bin
```

you check:

```
/usr/local/bin/java
/usr/bin/java
/bin/java
```

in exactly that order.

If all three are executable, print all three.

This is why your instinct that you need to implement the enumeration yourself is correct.

One subtle POSIX detail: empty PATH components

If you want to be genuinely POSIX-conformant, don't forget that an empty component in `PATH` represents the current directory.

For example:

```
PATH=/usr/bin:./bin
```

means:

```
/usr/bin
current directory
/bin
```

So you shouldn't blindly construct:

```
":" + "/" + filename
```

for an empty component.

Treat an empty PATH component as the CWD.

This also makes the explicit CWD-first rule interesting: if the CWD is also represented by an empty PATH component, **you should print it twice** if the requirement literally says to check CWD first and then every PATH entry.

The assignment says:

"If a match exists there, print its absolute path first. Then search each directory listed in `PATH`, in order, and print the absolute path of every match found."

So yes, **don't deduplicate matches** unless they explicitly tell you to. Their example:

```
/usr/bin/sudo
/bin/sudo
```

is demonstrating exactly that multiple matches are expected.

I'd structure `locate` roughly like this

```
locate(argc, argv)
|
|— for each filename
|
|   |— check cwd
|   |   └─ print if executable
|   |
|   |— split PATH
|   |
|   |— for each PATH directory
|   |   └─ check candidate
|   |       └─ print if executable
|   |
|   └─ if nothing printed
|       └─ command not found
|
└─ next filename
```

And the reusable pieces:

```
is_executable(path)
join_path(directory, filename)
print_absolute_path(path)
```

That keeps `locate()` itself pretty readable.

One thing I'd *not* do

Don't implement:

```
execvp(filename, ...)
```

to discover what would run.

Apart from the assignment's Part B `exec*` restriction, `execvp()` gives you **one selected executable**, whereas `locate` needs **all matching executables**.

Likewise, don't do:

```
access(filename, X_OK)
```

and interpret success as “found in PATH.” `access()` doesn't perform PATH searching for you; it checks the **specific path you give it**.

So your mental model should be:

`access()` *is the test; PATH traversal is my job.*

And because `locate` explicitly requires CWD → PATH[0] → PATH[1] → ... → all matches, implementing that enumeration yourself is exactly the right approach.



i'm stuck on what exactly counts as a matching executable. say there is a file called `foo` in the current directory but it doesn't have execute permission, while `/usr/bin/foo` does. should i completely ignore the cwd file and continue searching PATH? and if a file exists in one of the PATH directories but isn't executable, should that also just be skipped? i'm thinking `access(path, X\_OK)` might be what i want, but i'm not sure if there are any gotchas with using that instead of checking the permission bits myself.



Yes — your interpretation is right:

A candidate only counts if it is actually executable.

So in your example:

```
cwd/foo      exists, but NOT executable
/usr/bin/foo  exists, and executable
```

`locate foo` should **skip the CWD candidate** and continue searching `PATH`, eventually printing:

```
/usr/bin/foo
```

It should **not** stop just because something named `foo` exists in the CWD.

`access(path, X_OK)` **is the right tool**

I'd use:

```
if (access(path, X_OK) == 0) {
    // candidate is executable
}
```

But there's one important gotcha: `access(X_OK)` **tells you about execute permission/access, not necessarily that the path is a regular file.**

For example, you probably don't want a directory named `foo` to count as an executable command just because the process has execute/search permission on that directory.

So I'd combine it with `stat()`:

```
bool is_executable(const char *path)
{
    struct stat st;

    if (stat(path, &st) == -1)
        return false;
```

```

    if (!S_ISREG(st.st_mode))
        return false;

    return access(path, X_OK) == 0;
}

```

Then:

```

foo in cwd, non-executable
  ↓
is_executable() → false
  ↓
continue PATH

/usr/bin/foo, executable
  ↓
is_executable() → true
  ↓
print /usr/bin/foo

```

Why not check permission bits yourself?

You *could* do:

```
st.st_mode & S_IXUSR
```

etc., but that's actually less correct for determining whether **the current process can execute the file**.

For example, whether you can execute something depends on:

- your effective UID
- group membership
- owner/group/other permission bits
- potentially ACLs and other filesystem permission mechanisms

`access(path, X_OK)` asks the OS whether the file is accessible for execution under the relevant access-check semantics.

So for this assignment, **use** `access()` **rather than manually interpreting the** `x` **bits.**

And don't stop on a non-executable match

This is important for your PATH loop:

```

/usr/local/bin/foo  → exists, not executable → skip
/usr/bin/foo        → executable → print
/bin/foo            → executable → print

```

You print **both** executable matches.

Your algorithm is therefore:

```
for each candidate path:
  does it exist?
    no → skip
    yes
      ↓
    is it a regular file?
      no → skip
      yes
        ↓
    access(path, X_OK)?
      no → skip
      yes → print absolute path
```

And only after checking **all** candidates do you decide whether to print:

```
locate: command not found (foo)
```

if you found zero executable matches.

One small distinction from `hop`: for `locate`, **a non-executable `foo` in the CWD does not “block” `/usr/bin/foo`**. You're enumerating all executable matches, not implementing the shell's “first executable wins” command resolution.



bro the absolute path requirement is confusing me more than it should. if i find `ls` in the current directory, the candidate i started with is basically just `./ls` or `ls`, but the output has to be something like `/mnt/.../ls`. should i just get the cwd with `getcwd()` and concatenate the filename myself, or is there a proper POSIX way to resolve a candidate into an absolute path? i'm also wondering what happens if a PATH entry itself is relative, because then just concatenating the filename wouldn't necessarily give me an absolute path.



Yeah, this is one of those cases where **"absolute path" sounds more complicated than it actually is.**

For your assignment, I would **not try to invent an absolute-path resolver yourself**. POSIX gives you a useful tool: `realpath()`.

For the CWD case

If you find:

```
./ls
```

and `getcwd()` gives:

```
/mnt/projects/shell
```

you *could* construct:

```
/mnt/projects/shell/ls
```

with `getcwd() + "/" + filename`.

That's perfectly reasonable **if the candidate is relative to the current directory.**

But `realpath()` is cleaner:

```
char *absolute = realpath(candidate, NULL);
```

For:

```
realpath("./ls", NULL)
```

you'd get something like:

```
/mnt/projects/shell/ls
```

The `NULL` tells POSIX to allocate the result for you, so:

```
char *absolute = realpath(candidate, NULL);
```

```
if (absolute != NULL) {
    printf("%s\n", absolute);
    free(absolute);
}
```

But there's a subtlety with `realpath()`

`realpath()` resolves symlinks and canonicalizes the path.

Your assignment says:

“files (or links) which would be executed”

So suppose:

```
/usr/bin/python -> /usr/bin/python3
```

and the PATH candidate is:

```
/usr/bin/python
```

If you use `realpath()`, you may print:

```
/usr/bin/python3
```

instead of:

```
/usr/bin/python
```

Those are **not the same pathname**, even though one is a symlink to the other.

Since the assignment says to print the **absolute path to the executable** and explicitly mentions files **or links**, I'd actually lean toward **constructing the absolute candidate path yourself**, rather than canonicalizing it with `realpath()`.

The PATH-relative case

This is the interesting part you noticed.

Suppose:

```
PATH=./bin:/usr/bin
```

and your CWD is:

```
/mnt/project
```

For:

```
locate foo
```

the first PATH candidate is conceptually:

```
./bin/foo
```

If it's executable, the output should be:

```
/mnt/project/bin/foo
```

So you need to turn the **PATH directory itself** into an absolute path before constructing the output.

You can do that with `getcwd()` when the PATH component is relative.

Conceptually:

| PATH entry | candidate | output |
|------------|--------------|----------------------|
| ----- | | |
| ./bin | ./bin/foo | /mnt/project/bin/foo |
| /usr/bin | /usr/bin/foo | /usr/bin/foo |

A nice helper

I'd make one function responsible for this:

```
char *make_absolute_candidate(const char *dir, const char *filename);
```

Its logic:

```
dir starts with '/'
↓
already absolute
→ dir + "/" + filename

dir is relative
↓
getcwd()
→ cwd + "/" + dir + "/" + filename
```

Then your lookup code doesn't care whether the PATH entry was absolute or relative.

For example:

```
char *make_absolute_candidate(const char *dir, const char *name)
{
    char *cwd = NULL;
    char *result = NULL;

    if (dir[0] == '/') {
        // construct dir/name
    } else {
        cwd = getcwd(NULL, 0);
        // construct cwd/dir/name
        free(cwd);
    }
}
```

```

    }

    return result;
}

```

You can use your dynamic `join_path()` helper from `reveal` here too.

And what about the CWD candidate?

Treat the CWD as just another directory:

```

directory = current absolute CWD
filename = ls

```

So if:

```

cwd = /mnt/project

```

then:

```

make_absolute_candidate("/mnt/project", "ls")

```

gives:

```

/mnt/project/ls

```

That means your lookup code can become very uniform:

```

CWD
↓
make_absolute_candidate(cwd, filename)
↓
check_executable
↓
print

PATH[0]
↓
make_absolute_candidate(PATH[0], filename)
↓
check_executable
↓
print

PATH[1]
↓
...

```

One more subtle POSIX case

An **empty PATH component** means the current directory:

```
PATH=/usr/bin:./bin
```

So the middle entry behaves like:

```
.
```

You can normalize an empty component to your absolute CWD before constructing the candidate.

So I'd recommend:

Don't use `realpath()` for the output path unless the assignment explicitly wants canonicalized paths.

Instead:

```
getcwd() once
    ↓
absolute CWD

for each search directory:
    ↓
if absolute → use it
if relative → resolve relative to CWD
    ↓
append filename
    ↓
test executable
    ↓
print that constructed absolute path
```

That preserves the exact candidate you actually found, including symlinks, while guaranteeing that every printed path is absolute.



i think i understand why normal command lookup stops after the first executable, but this assignment specifically wants every matching executable printed, so i'm not sure whether i should even think of this as normal command lookup anymore. should my function just keep scanning all of PATH even after finding a match, and then print everything in exactly the order the directories appear in PATH? and if the same directory happens to appear twice in PATH, would i literally print the same executable twice because those are technically two matching PATH entries?



Exactly. For this assignment, **stop thinking of** `locate` **as normal command resolution**. It is more like:

"Enumerate every executable candidate that a POSIX shell could find, in the specified search order."

So yes:

```
CWD
↓
PATH[0]
↓
PATH[1]
↓
PATH[2]
↓
...
```

and **you never stop after the first match**.

Duplicate PATH entries

Yes — if the same directory appears twice in `PATH`, I would **print the match twice**.

For example:

```
PATH=/usr/bin:/bin:/usr/bin
```

and `/usr/bin/foo` is executable:

```
locate foo
```

should produce:

```
/usr/bin/foo
/bin/foo
/usr/bin/foo
```

Why? Because the requirement says:

"Then search each directory listed in `PATH`, in order, and print the absolute path of every match found."

A `PATH` entry is an individual search location. It doesn't say to deduplicate them.

This is also consistent with the assignment's example:

```
locate sudo
/usr/bin/sudo
/bin/sudo
```

They're explicitly demonstrating that **multiple matching locations are all printed**.

So your function should look conceptually like

```
bool found = false;

check_current_directory(filename, &found);

for (each PATH entry in order) {
    if (is_executable(candidate)) {
        printf("%s\n", candidate);
        found = true;
    }
}

if (!found) {
    printf("locate: command not found (%s)\n", filename);
}
```

Notice there's **no** `break` in that `PATH` loop.

And the outer loop handles multiple arguments:

```
locate foo bar baz
  |   |   |
  ↓   ↓   ↓
all  all  all
matches matches matches
```

with the entire set of `foo` results appearing before the `bar` results, because the requirement says arguments are processed in the order specified.

So the complete ordering is:

```
argument 1:
  CWD
  PATH[0]
  PATH[1]
  PATH[2]
  ...

argument 2:
```

```
CWD
PATH[0]
PATH[1]
PATH[2]
...
```

```
argument 3:
```

```
...
```

And **duplicates are preserved**. Don't deduplicate the PATH array or the resulting executable paths unless the assignment explicitly tells you to.



Part C: File Redirection and Pipes [50]

For this part, when processing commands with sequential (`;`) or background (`&`) operators, you should only execute the first command group and ignore the rest.

Banned syscalls for Part C: You are not allowed to use `system()` and `popen()` for file IO and piping.

C1: Command Execution [8]

You must allow execution of arbitrary commands, found either as a relative/absolute path to an executable, or by name.

Requirements

1. If the command name contains a `/`, treat it as a literal path and execute it directly if it exists and is executable.
2. If the command name does not contain a `/`, first check whether an executable with that name exists in the current working directory, and run it if so.
3. If no such executable exists in the current working directory, search `PATH` as usual.
4. To skip the current directory check and search `PATH` directly, prefix the command name with `%`. `%name` must always resolve via `PATH`, even if a matching executable exists in the current directory.
5. If no executable is found by any of the above, output `cshell: command not found (commandname)`.

Example

```
...
<goatam@iiit:~/proj> echo hello
hello
<goatam@iiit:~/proj> ls
build.sh
<goatam@iiit:~/proj> build.sh
running local build script
<goatam@iiit:~/proj> %build.sh
cshell: command not found (build.sh)
<goatam@iiit:~> dosakdaoskdos
cshell: command not found (dosakdaoskdos)
...
```

bro i'm trying to implement the actual command lookup now and i'm getting confused about where the different cases should split. if the command contains `/`, i know i'm supposed to treat it literally and not search PATH, but for something like `ls` i first have to check cwd and then PATH, while `%ls` skips cwd completely. should i basically

normalize the command first by checking whether it starts with `%`, then whether it contains `/`, and then have three separate lookup paths? or is there a cleaner way to structure this without making the execution function a giant chain of conditions?



Yeah — I wouldn't make the executor a giant `if/else` chain. The cleanest way is to separate **"what kind of lookup is this?"** from **"how do I perform a lookup?"**

You essentially have **three lookup modes**:

```
command
|
|— contains '/' —————> LITERAL PATH
|
|— starts with '%' —————> PATH ONLY
|
|— ordinary name —————> CWD THEN PATH
```

The `%` is a little special because it isn't actually part of the command name. So strip it first.

Step 1: normalize `%`

Given:

```
ls
%ls
/bin/ls
./build.sh
%build.sh
```

turn them into something like:

```
typedef enum {
    LOOKUP_CWD_THEN_PATH,
    LOOKUP_PATH_ONLY,
    LOOKUP_LITERAL
} LookupMode;
```

Then:

```
ls          → name="ls",      mode=CWD_THEN_PATH
%ls         → name="ls",      mode=PATH_ONLY
/bin/ls      → name="/bin/ls", mode=LITERAL
./build.sh   → name="./build.sh", mode=LITERAL
%build.sh    → name="build.sh", mode=PATH_ONLY
```

The important ordering is:

check `%` first, then check `/` on the resulting name.

I'd actually make the rule:

```
if original starts with '%':
    remove %
    mode = PATH_ONLY
else if name contains '/':
    mode = LITERAL
else:
    mode = CWD_THEN_PATH
```

This means `%` takes precedence.

Step 2: have one function for each lookup operation

You already have the pieces from `locate`. Reuse them.

Conceptually:

```
char *find_executable(const char *name, LookupMode mode);
```

Then internally:

```
LOOKUP_LITERAL
    ↓
    check exactly `name`
    ↓
    return executable path or NULL

LOOKUP_CWD_THEN_PATH
    ↓
    check cwd/name
    ↓ success → return
    ↓ failure
    check PATH[0]/name
    check PATH[1]/name
    ...
    ↓
    return first executable or NULL

LOOKUP_PATH_ONLY
    ↓
    check PATH[0]/name
    check PATH[1]/name
    ...
    ↓
    return first executable or NULL
```

Notice the big difference from `locate`:

`locate` **prints every match.**

command execution stops at the first match.

So you can potentially share a lower-level helper:

```
bool is_executable(const char *path);
```

and your PATH traversal logic, while having different policies about whether to stop.

One particularly clean approach

Have the lookup function return the **actual path to execute**:

```
char *resolve_command(const char *command);
```

Then your executor becomes almost boring:

```
char *path = resolve_command(cmd->argv[0]);

if (path == NULL) {
    printf("cshell: command not found (%s)\n", cmd->argv[0]);
    return;
}

execute_path(path, cmd->argv);
free(path);
```

And `resolve_command()` deals with all the lookup weirdness.

For `%ls`, make sure the error message uses the **original command name without** `%`, because the example says:

```
%build.sh
cshell: command not found (build.sh)
```

So it may be useful to keep:

```
original = "%build.sh"
name      = "build.sh"
```

separate.

One subtle case

Suppose someone gives:

```
%./foo
```

After removing `%`, the name is:

```
./foo
```

but because `%` means:

must always resolve via PATH

you should **not** suddenly treat `./foo` as a literal path.

So don't do:

```
strip %  
then see '/' → literal
```

Instead, `%` establishes the lookup mode first:

```
%./foo  
↓  
PATH_ONLY  
name = ./foo  
↓  
search PATH directories for a file literally named "./foo"
```

Whether such a PATH match is useful is another matter, but the key is that `%` says **don't perform the CWD/literal-path behavior**.

I would structure your modules roughly like this

```
executor.c  
    execute_command()  
    resolve_command()  
  
lookup.c  
    is_executable()  
    lookup_literal()  
    lookup_cwd()  
    lookup_path()  
  
process.c  
    fork()  
    exec*()  
    waitpid()  
    redirection/pipes later
```

Then:

```
execute_command()  
    |  
    ↓  
resolve_command()  
    |  
    ├── literal  
    ├── cwd → path  
    └── path only  
    |  
    ↓
```

```
actual executable path
  |
  ↓
fork + exec
```

That separation is especially useful because **C1's lookup semantics are independent of process creation**. Once `resolve_command()` gives you `/usr/bin/ls`, the executor doesn't care whether it came from CWD, PATH, or a literal path.

And later, when you add redirection/pipes, you can keep that same resolver and put the resolved executable into the child process's `exec` call.



i'm stuck on the `%` case because the user actually types `%echo hello`, but obviously the program i'm trying to execute is called `echo`, not `%echo`. so should i remove the `%` before doing any lookup and keep the original command around separately just for error messages? and if the local directory contains an executable called `echo`, `%echo` should completely ignore it and search PATH, even though normal `echo` would pick the local one first, right?



Exactly. That's the clean way to handle `%`.

Treat `%` as **syntax controlling lookup**, not as part of the executable's name.

For:

```
%echo hello
```

you want:

```
original command = "%echo"
lookup name      = "echo"
lookup mode      = PATH_ONLY
argv[0]          = "echo"
```

So yes, keep the original string around if you need it for diagnostics, but strip `%` before doing any actual executable lookup.

The important behavior

Suppose your current directory contains:

```
./echo      ← executable
```

and PATH contains:

```
/usr/bin/echo
```

Then:

```
echo hello
```

does:

```
CWD/echo
↓
found
↓
execute ./echo
```

But:

```
%echo hello
```

does:

```
strip %  
  ↓  
"echo"  
  ↓  
SKIP CWD  
  ↓  
search PATH  
  ↓  
/usr/bin/echo  
  ↓  
execute /usr/bin/echo
```

So **yes**, `%echo` **completely ignores the local executable.**

I'd represent this explicitly

For example:

```
typedef enum {  
    LOOKUP_CWD_THEN_PATH,  
    LOOKUP_PATH_ONLY,  
    LOOKUP_LITERAL  
} LookupMode;  
  
typedef struct {  
    const char *original;  
    const char *name;  
    LookupMode mode;  
} CommandLookup;
```

Then when you receive the command:

```
%echo
```

you construct the lookup information:

```
original → "%echo"  
name     → "echo"  
mode     → LOOKUP_PATH_ONLY
```

For:

```
echo
```

you get:

```
original → "echo"
name      → "echo"
mode      → LOOKUP_CWD_THEN_PATH
```

For:

```
./echo
```

you get:

```
original → "./echo"
name      → "./echo"
mode      → LOOKUP_LITERAL
```

Then `resolve_command()` doesn't need to know anything about `%` itself. It just receives:

```
resolve_command("echo", LOOKUP_PATH_ONLY);
```

and searches PATH.

One small detail about `argv`

You probably want the `%` removed from the argument that eventually goes to the program too.

So:

```
%echo hello
```

should result in the child getting approximately:

```
argv[0] = "echo"
argv[1] = "hello"
```

not:

```
argv[0] = "%echo"
```

Otherwise the executed program would see `%echo` as its name.

So the pipeline is:

```
user input
↓
"%echo", "hello"
↓
recognize % syntax
↓
"echo", "hello"
↓
PATH-only resolution
↓
```

```
"/usr/bin/echo"
```

↓

```
exec("/usr/bin/echo", ["echo", "hello", NULL])
```

And for the error:

```
%doesnotexist
```

you'd print:

```
cshell: command not found (doesnotexist)
```

matching the assignment's example for `%build.sh`.

So yes: **strip** `%` **early, preserve the stripped name as the command name, and separately retain the original token only if you need it for diagnostics.**



i'm implementing the current-directory priority and i'm wondering if i should use ``access()`` to check whether ``./command`` exists and is executable, and then pass that path into ``execv()``. but what if the file exists and is executable but ``execv()`` still fails for some other reason, like it's not actually a valid executable format? should that be treated as command not found, or should i let the actual exec error through? the spec only seems to care about whether an executable was found.



Right — I'd distinguish **“found a candidate”** from **“successfully executed the candidate.”** They're two different stages.

For C1, your lookup can reasonably do:

```
if (is_executable(path)) {  
    // found it  
    execv(path, argv);  
}
```

where `is_executable()` might be:

```
bool is_executable(const char *path)  
{  
    struct stat st;  
  
    if (stat(path, &st) == -1)  
        return false;  
  
    return S_ISREG(st.st_mode) && access(path, X_OK) == 0;  
}
```

What if `execv()` then fails?

Suppose:

```
./foo
```

exists, is a regular file, and has execute permission, but contains garbage rather than a valid executable format.

Then:

```
execv("./foo", argv);
```

fails, typically with:

```
errno == ENOEXEC
```

I would **not turn that into**:

```
cshell: command not found (foo)
```

because you *did* find `foo`. The lookup succeeded; execution failed.

For the assignment, the clean separation is:

```
lookup
  ↓
candidate exists + executable?
├─ no → keep searching
└─ yes
    ↓
    FOUND
    ↓
    fork
    ↓
    child: execv()
    ↓
    exec succeeds → program runs
    exec fails    → execution error
```

And this matters for PATH

Suppose:

```
./foo      → executable but invalid format
/usr/bin/foo → valid executable
```

Your C1 lookup says **CWD has priority**, so you shouldn't silently skip `./foo` just because `execv()` would fail and then run `/usr/bin/foo`.

The first executable candidate has been found. The execution attempt should fail rather than changing the lookup semantics underneath it.

That's another reason to keep:

```
resolve_command()
```

separate from:

```
execute_command()
```

`resolve_command()` answers:

"Which executable path should win?"

`execute_command()` answers:

"Can I actually start that executable?"

One caveat

`access(path, X_OK)` is not a guarantee that `execv()` will succeed. That's normal. There can be things such as:

- invalid executable format (`ENOEXEC`)
- missing interpreter for a script (`ENOENT`)
- permissions changing between the check and `execv()` (`EACCES`)
- other execution failures

This is the classic **TOCTOU** issue: the check and the actual execution are separate operations.

For your assignment, don't try to eliminate that complexity. Use the check for **lookup**, then let `execv()` be the authority for **execution**.

So I'd implement the child roughly as:

```
execv(path, argv);

/* only reached if execv failed */
perror("cshell");
_exit(126);
```

The exact error message/status should follow whatever later C1 requirements specify, but **don't report "command not found" merely because `execv()` failed**. "Command not found" belongs to the case where your lookup couldn't find any executable candidate.



one thing that's confusing me is the difference between checking if something is executable and actually executing it. if i use ``access(path, X_OK)`` and it says yes, then fork and ``execv()`` fails anyway, do i have to handle that separately? i'm wondering if checking with ``access()`` is actually required or if i should just try ``execv()`` and use its failure to decide whether the command exists.



Yes — you should handle those as **two separate questions**.

1. `access()` answers: “Does this candidate qualify?”

For your C1 lookup:

```
access(path, X_OK) == 0
```

means the current process has execute permission for that path. You can use that to decide:

“This is a matching executable candidate, so this is the path that wins the CWD/PATH lookup.”

You might combine it with `stat()` to ensure it's a regular file:

```
struct stat st;

if (stat(path, &st) == 0 &&
    S_ISREG(st.st_mode) &&
    access(path, X_OK) == 0) {
    // found a candidate
}
```

Then **stop searching** because C1 wants the first matching executable.

2. `execv()` answers: “Can this candidate actually be executed?”

After you've selected the path:

```
pid_t pid = fork();

if (pid == 0) {
    execv(path, argv);

    // Only reached if execv() failed
    perror("cshell");
    _exit(126);
}

waitpid(pid, NULL, 0);
```

So:

```
access()
  ↓
"Looks like an executable candidate"
  ↓
choose this path
  ↓
fork()
  ↓
execv()
  ↓
"Can the OS actually execute it?"
```

Why not just try `execv()` to discover whether it exists?

Because **you need to search before you fork/exec.**

Suppose:

```
./foo          exists but isn't executable
/usr/bin/foo    is executable
```

You need to skip `./foo` and find `/usr/bin/foo`.

You can't meaningfully do:

```
fork ./foo
execv ./foo → fails
then try /usr/bin/foo
```

because you've now mixed **lookup semantics** with **execution semantics**, and you'd also potentially produce unwanted errors from candidates that shouldn't count.

Instead:

```
./foo
  ↓
access(X_OK) → no
  ↓
continue PATH

/usr/bin/foo
  ↓
access(X_OK) → yes
  ↓
THIS is the selected command
  ↓
fork + execv()
```

And yes, `execv()` can still fail

For example:

```
./foo
```

is a regular file with execute permission but contains invalid executable data.

Then:

```
access("./foo", X_OK)
  ↓
  yes
  ↓
execv("./foo", ...)
```

↓

fails

That's **not a lookup failure**. You've found the command; execution itself failed.

So don't do:

```
if (execv(...) == -1)
    printf("command not found");
```

because that conflates two different situations.

The clean architecture

I'd make the stages:

```
resolve_command()
|
├─ CWD candidate
├─ PATH candidate 1
├─ PATH candidate 2
└─ ...
|
↓
path found
|
↓
execute_command()
|
↓
fork()
|
├─ child → execv(path, argv)
|
├─ success → program runs
└─ failure → execution error
```

So `access()` **isn't a replacement for** `execv()` **and** `execv()` **isn't a replacement for** `lookup`. They serve different purposes.

And given the assignment's wording — *“if no executable is found”* — using `access()` / `stat()` during the lookup phase is the much cleaner interpretation.



i'm trying to make sure the parent shell doesn't accidentally get replaced by the command. so i'm assuming the flow for something like ``echo hello`` is basically: parse argv, resolve the executable path, ``fork()``, child sets up anything it needs and calls ``execv()``, parent waits with ``waitpid()``, then prints the prompt again. is that the basic structure i should be building C1 around before i add redirection and pipes later?



Yes — **that's exactly the basic C1 structure I'd build first**. You're thinking about it in the right order.

For:

```
echo hello
```

your shell should conceptually do:

```
read line
  ↓
lex + parse
  ↓
argv = ["echo", "hello", NULL]
  ↓
resolve_command("echo")
  ↓
/usr/bin/echo      ← for example
  ↓
fork()
  |
  |— child
  |   |— set up redirection/pipes later
  |   |— execv("/usr/bin/echo", argv)
  |
  |— parent
      |— waitpid(child)
          ↓
      print prompt
```

The critical point

Only the child calls `execv()`.

If the parent did:

```
execv(path, argv);
```

then your shell process itself would be replaced by `echo`.

Instead:

```

pid_t pid = fork();

if (pid == 0) {
    // CHILD
    execv(path, argv);

    // execv only returns if it failed
    perror("cshell");
    _exit(126);
}

if (pid > 0) {
    // PARENT
    waitpid(pid, NULL, 0);
}

```

After the child exits, the parent is still your shell process, so it can go back around its main loop:

```

waitpid()
  ↓
prompt
  ↓
getline()
  ↓
...

```

And this is a really good foundation for Part C

You can deliberately leave a "child setup" stage:

```

if (pid == 0) {
    setup_redirections(cmd);
    setup_pipes(cmd);

    execv(path, argv);
    ...
}

```

For C1, those functions can effectively do nothing because there aren't any redirections/pipes yet.

Then later:

```

C1:
fork
├─ child → execv
└─ parent → waitpid

C2/C3:
fork
├─ child → dup2(...) → execv
└─ parent → waitpid

```

Pipes:

fork

```
|— child 1 → dup2(pipe[1], STDOUT) → execv
|— child 2 → dup2(pipe[0], STDIN) → execv
└— parent → close pipes → waitpid(...)
```

So you're building the right abstraction now rather than rewriting your executor later.

One other thing: **resolve the executable before** `fork()` if possible. Then the child receives an already-resolved path and only has to perform the actual execution setup.

That gives you a very clean division:

PARENT SHELL

```
|— parse
|— validate
|— resolve command
|— fork
|
|— CHILD
|   |— redirection/pipes
|   └— execv
|
└— PARENT
    └— wait
```

That's a solid C1 skeleton to build the later sections on.



C2: Input Redirection [12]

Syntax: ``command < filename``

Purpose: The input redirection operator allows a command to read its standard input from one or more files instead of the terminal.

Requirements

1. The shell must open each specified file for reading using the ``open()`` system call with ``O_RDONLY`` flag.
2. If any file does not exist or cannot be opened, the shell must print ``cshell: no such file or directory`` and not execute the command.
3. A command may have more than one input redirection (e.g. ``command < f1 < f2``). The command's standard input must consist of the contents of every listed file, read in the order given, as one continuous stream.
4. The shell must redirect the command's standard input (``STDIN_FILENO``) to this stream using ``dup2()``.
5. The shell must close any file descriptors it opens once they are no longer needed.

Example

```
```\n<goatam@iit:~> cat < notes.txt\nmeeting at 5pm\n<goatam@iit:~> echo "line one" > a.txt\n<goatam@iit:~> echo "line two" > b.txt\n<goatam@iit:~> cat < a.txt < b.txt\nline one\nline two\n<goatam@iit:~> cat < missing.txt\ncshell: no such file or directory\n```\n
```

bro i'm trying to understand how i should actually implement ``<`` because for one file it seems simple — ``open()`` it, ``dup2()`` it onto stdin, close the original fd — but then the assignment allows ``cat < f1.txt < f2.txt`` and says stdin has to contain both files in order. i obviously can't just ``dup2()`` both because the second one would replace the first. am i supposed to create a pipe, have some feeder process read f1 and f2 sequentially into the pipe, and then connect the command's stdin to the read end?



Yes — **a pipe + feeder process is one valid way to implement the “concatenated stdin”**

**semantics**, but there's an important detail: you shouldn't have the command process itself fill the pipe

before `exec`, because that can deadlock on large input.

The conceptual requirement is:

```
f1.txt └─┐
 │→ one continuous stream →> command's stdin
f2.txt └─┘
```

So for:

```
cat < f1.txt < f2.txt
```

the command should see:

```
contents(f1.txt)
contents(f2.txt)
```

as though they were one input stream.

## A good implementation with a pipe

You can do:

```
 feeder
 / \
f1.txt ────> / \
 \ \
f2.txt ────> pipe ──> cat's stdin
 /
```

The feeder does:

```
read f1 → write pipe
read f2 → write pipe
close pipe
exit
```

Meanwhile `cat` has:

```
dup2(pipe_read, STDIN_FILENO);
execv(...);
```

Because the feeder and `cat` run concurrently, the pipe can be consumed while it's being filled. That avoids the pipe-buffer deadlock.

---

## But there's an important requirement in your spec

It says:

*If any file does not exist or cannot be opened, ... **not execute the command**.*

So you should **validate/open all the input files before starting the command**.

For:

```
cat < f1 < missing < f2
```

you should do something like:

```
open(f1) → success
open(missing) → failure
 ↓
 print error
 ↓
 DO NOT fork/execute cat
```

Don't start `cat` and only discover the missing file halfway through a feeder process.

So your parent shell can first open every file:

```
int fds[n];

for each input redirection:
 fd = open(filename, O_RDONLY);

 if (fd == -1) {
 print error;
 close everything already opened;
 return;
 }

 fds[i] = fd;
```

Now you know **all inputs are valid**.

---

## Then you have a couple of implementation choices

### Option 1: Pipe + feeder

This is probably the most natural interpretation.

After validation:

```
parent shell
|
|— create pipe
|
|— fork command
| |
| |— stdin = pipe read end
| | exec command
|
|— fork feeder
 |
 |— read f1 → pipe
```

```
└─ read f2 → pipe
└─ close pipe
```

The feeder should write the files **in order**.

You'd want something conceptually like:

```
for (each fd) {
 while ((n = read(fd, buffer, sizeof buffer)) > 0) {
 write_all(pipe_write, buffer, n);
 }
}
```

Then close the write end.

When the feeder closes the pipe, the command eventually sees EOF after all the contents of all files.

---

## Option 2: Temporary file

There's another potentially simpler approach:

```
f1 └─
f2 └─→ temporary file
f3 └─

 ↓
 lseek(..., 0)
 ↓
 dup2(tempfd, stdin)
 ↓
 exec
```

This avoids needing a feeder process entirely.

But now you're managing a temporary file, and depending on what the assignment expects, that may be more machinery than necessary.

For this assignment, **I'd probably use the pipe + feeder approach if you're comfortable with processes.**

---

## One thing you absolutely shouldn't do

This:

```
dup2(fd1, STDIN_FILENO);
dup2(fd2, STDIN_FILENO);
```

does **not** concatenate them.

After:

```
dup2(fd1, STDIN_FILENO);
```

stdin points at **f1**.

Then:

```
dup2(fd2, STDIN_FILENO);
```

simply replaces that:

```
stdin → f2
```

**f1** is no longer stdin.

---

## And this is a useful way to think about the requirement

For one input:

```
< f1
```

```
f1 —————> stdin
```

For multiple inputs:

```
< f1 < f2 < f3
```

```
f1 ┘
```

```
f2 ┘→ concatenated stream → stdin
```

```
f3 ┘
```

So `dup2()` **only performs the final connection**. Something else has to create the combined stream first.

And because the combined stream can be arbitrarily large, **a pipe needs a concurrent writer**. That's why your feeder-process idea is sound.

One architectural suggestion: make redirection setup a separate stage in your child execution code:

```
fork
├─ child
│ ├── setup_input_redirection()
│ ├── setup_output_redirection()
│ ├── setup_pipes()
│ └─ execv()
```

Then C2 can implement the input side without making your main executor turn into a huge special-case block.



i'm stuck on when the input files should actually be opened and validated. the requirement says if *any* file doesn't exist, the command must not execute, so for something like ``cat < f1.txt < missing.txt < f2.txt``, should i first open/check every input file before even doing the ``fork()``? because otherwise i could start the command, it could already read f1, and only afterwards discover that missing.txt doesn't exist, which seems to violate the requirementttttt



YES 🙄 — **you've got exactly the right concern.** The requirement forces you to validate **all input redirections before the command starts executing.**

For:

```
cat < f1.txt < missing.txt < f2.txt
```

you should do:

```
parse
↓
identify f1, missing, f2
↓
open f1 ✓
open missing ✗
↓
close f1
↓
print:
cshell: no such file or directory
↓
DO NOT fork/execute cat
```

You should **not** do:

```
fork cat
↓
cat starts reading f1
↓
open missing.txt
↓
oops, failure
```

because then the command has already executed, violating:

*"If any file does not exist or cannot be opened, ... not execute the command."*

**So the clean architecture is a two-phase process**

## Phase 1 — Validate/open everything

In the **parent shell**:

```
int fds[n];

for (int i = 0; i < n; i++) {
 fds[i] = open(files[i], O_RDONLY);

 if (fds[i] == -1) {
 printf("cshell: no such file or directory\n");

 // close all previously opened fds
 for (int j = 0; j < i; j++)
 close(fds[j]);

 return; // crucial: no fork
 }
}
```

At the end of this phase:

```
f1.txt ✓ open
missing.txt ✓ open
f2.txt ✓ open
```

or, if anything failed:

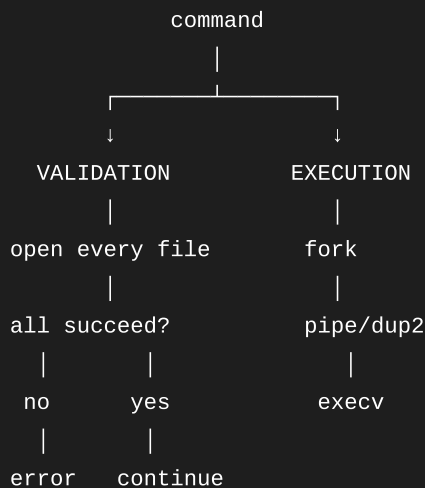
```
anything failed
 ↓
close everything
 ↓
don't execute
```

## Phase 2 — Actually execute

**Only after all opens succeeded:**

```
all files successfully opened
 ↓
create pipe(s) / feeder
 ↓
fork command
 ↓
command gets combined stream as stdin
 ↓
execv()
```

This separation is really useful:



## There's also a nice reason to open them in the parent

You can pass the already-open file descriptors to your feeder process later.

For example:

```

parent
|
├─ open f1 → fd 3
├─ open f2 → fd 4
|
├─ pipe()
|
├─ fork feeder
| └─ read fd 3 → pipe
| └─ read fd 4 → pipe
|
└─ fork command
 └─ pipe → stdin → exec

```

You don't need to reopen the files after forking.

And remember to close the appropriate copies in **every process**, especially the pipe ends. Otherwise the command might never see EOF because some process still has the pipe's write end open.

So your instinct is absolutely right:

**Open and validate every `< filename` first. If even one fails, clean up and return to the prompt without forking the command at all.**

That's one of those requirements where the wording really does dictate the architecture.



i'm trying to understand the fd ownership here because after ``open()`` + ``pipe()`` + ``fork()`` there are suddenly a ridiculous number of file descriptors. if the child gets the read end of the input pipe as stdin, which copies should the parent and child close, and when? especially if i'm feeding multiple files through one pipe, i don't want the parent accidentally keeping a write end open and preventing EOF from ever reaching the command :(



YES — the pipe EOF issue is exactly about **which processes still have a write-end FD open**. The easiest way to reason about it is to treat every FD as an owned resource and explicitly close copies after `fork()`.

For your multiple-input-redirection design, I'd have **three parties**:

```
parent shell
|
|— feeder process
| |— reads f1, f2, ... → pipe write end
|
|— command process
| |— pipe read end → stdin
```

Suppose:

```
f1 = fd 3
f2 = fd 4
pipe read = fd 5
pipe write = fd 6
```

## Before forking

Parent has:

```
parent:
fd 3 → f1
fd 4 → f2
fd 5 → pipe READ
fd 6 → pipe WRITE
```

## Fork the feeder

After `fork()`, **both** parent and feeder temporarily have copies:

```
parent:
fd 3 → f1
fd 4 → f2
fd 5 → pipe READ
fd 6 → pipe WRITE
```

```
feeder:
 fd 3 → f1
 fd 4 → f2
 fd 5 → pipe READ
 fd 6 → pipe WRITE
```

The feeder only needs:

```
f1
f2
pipe WRITE
```

So immediately in the feeder:

```
close(pipe_read);
```

Then it reads:

```
f1 → pipe_write
f2 → pipe_write
...
```

and finally:

```
close(fd_f1);
close(fd_f2);
close(pipe_write);
_exit(0);
```

**That final `close(pipe_write)` is what allows the command to eventually see EOF.**

---

## Parent after forking feeder

The parent doesn't need the input file FDs anymore because the feeder owns them.

So after the feeder has been created:

```
close(fd_f1);
close(fd_f2);
```

The parent should also close its copy of the pipe write end:

```
close(pipe_write);
```

This is **super important**.

Otherwise:

```
feeder
 ↓
writes all data
```

↓  
closes its write end

BUT

parent  
↓  
still has write end open

The command sees:

data  
data  
data  
...  
(no EOF!)

because from the kernel's perspective **there is still a writer attached to the pipe.**

So the parent should not retain `pipe_write`.

---

## Then fork the command

At this point, ideally the parent has only:

parent:  
    pipe\_read

plus its normal shell FDs.

Then:

```
pid_t child = fork();
```

Both temporarily have:

parent:  
    pipe\_read

child:  
    pipe\_read

The child does:

```
dup2(pipe_read, STDIN_FILENO);
close(pipe_read);
execv(...);
```

After `dup2()`:

child:

```
stdin —————> pipe read end
```

The parent doesn't need the read end once the command has been started:

```
close(pipe_read);
```

So eventually:

```
parent:
 no pipe FDs

feeder:
 no pipe FDs

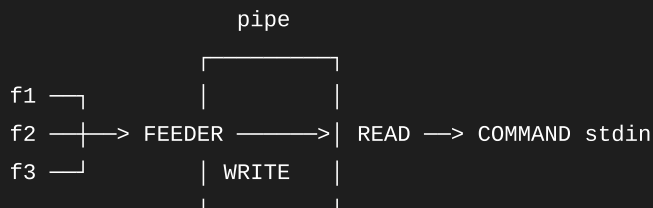
command:
 stdin → pipe read end
```

Beautiful. 😊

---

## The whole lifecycle

I'd literally draw this on paper while implementing it:



### Parent

Before forks:

```
f1
f2
pipe_read
pipe_write
```

After feeder fork:

```
close f1
close f2
close pipe_write
```

After command fork:

```
close pipe_read
wait for feeder
wait for command
```

## Feeder

After fork:

```
close pipe_read
```

Then:

```
read f1 → pipe_write
read f2 → pipe_write
...
```

Finally:

```
close f1
close f2
close pipe_write
exit
```

## Command

After fork:

```
close pipe_write // it should never have a write end
dup2(pipe_read, STDIN_FILENO)
close(pipe_read)
execv(...)
```

---

## The critical invariant

For pipes, remember this:

***The reader gets EOF only when all copies of the write end have been closed.***

So if your command hangs after printing all the expected input, the first thing I'd investigate is:

```
"Who still has pipe_write open?"
```

Usually the culprit is the parent accidentally retaining it.

---

## One simplification I'd recommend

You actually don't need the parent to keep the input file FDs after creating the feeder. The ownership can be:

```
Parent:
 creates/open files
 creates pipe
 forks feeder
 closes file FDs + write end
 forks command
```

```
closes read end
waits
```

Feeder:

```
owns input file FDs
owns pipe write end
```

Command:

```
owns pipe read end → stdin
```

That's a very clean ownership model.

And one more subtle point: **close unused pipe ends immediately after each** `fork()`, rather than waiting until the very end. It makes the ownership obvious and prevents exactly the EOF bug you're worried about.



for the simple case `cat < f1.txt`, do i actually need a pipe and feeder process at all, or can i just open f1 and `dup2()` it directly onto stdin? i'm wondering whether i should have two implementations depending on whether there is one input file versus multiple files, or whether it's cleaner to always use the same pipe-based mechanism even for one file.



For the simple case, **you absolutely don't need a pipe or feeder.**

For:

```
cat < f1.txt
```

the natural implementation is:

```
parent:
 open("f1.txt", O_RDONLY)
 ↓
 validate succeeds
 ↓
 fork()
 ↓
child:
 dup2(fd, STDIN_FILENO)
 close(fd)
 execv("cat", ...)
 ↓
parent:
 close(fd)
 waitpid()
```

That's much simpler.

## For multiple files, you need the combined stream

For:

```
cat < f1.txt < f2.txt
```

you can't do:

```
dup2(fd1, STDIN_FILENO);
dup2(fd2, STDIN_FILENO);
```

because the second `dup2()` replaces the first.

So you need something like:

```
f1 └─
 │ → pipe → stdin → cat
f2 └─
```

with a feeder process (or another concatenation mechanism).

## I'd absolutely use two paths

I wouldn't force every redirection through a pipe just for uniformity.

Conceptually:

```
if (input_count == 1) {
 // direct redirection
 dup2(input_fd, STDIN_FILENO);
} else {
 // create combined stream
 // pipe + feeder
}
```

That gives you:

```
1 input:
file → stdin

2+ inputs:
file1 └─
file2 ┤ → combined stream → stdin
file3 └─
```

And it has a nice practical advantage: **the common case doesn't need an extra process or pipe at all.**

## But keep the interface unified

You don't want your whole executor to know about these details.

I'd make something like:

```
setup_input_redirection(command);
```

and let that function decide:

```
setup_input_redirection()
|
| 0 files → do nothing
|
| 1 file → dup2(file_fd, STDIN_FILENO)
|
| >1 files → pipe + feeder
```

Your child execution remains:

```
if (pid == 0) {
 setup_input_redirection(cmd);
 setup_output_redirection(cmd);
 setup_pipes(cmd);

 execv(path, argv);
}
```

One caveat: **all input files still need to be opened/validated before the command is forked**, even in the one-file case, because of the requirement that *any* failed input redirection prevents execution.

So for:

```
cat < f1 < missing
```

you validate both first, discover `missing` fails, and don't fork `cat` at all.

For:

```
cat < f1
```

once validation succeeds, the child can simply `dup2()` the already-open `f1` FD.

**So yes: one-file → direct `dup2()`, multiple-files → combined stream. That's cleaner than unnecessarily creating a feeder for the simple case.**



### ### C3: Output Redirection [12]

**Syntax**: ``command > filename`` or ``command >> filename``

**Purpose**: The output redirection operators allow a command's output to be written to one or more files instead of the terminal.

**Requirements**

1. A command may have more than one output redirection (e.g. ``command > f1 >> f2 > f3``). Every listed file must receive the complete output of the command.
2. Each redirection keeps its own mode independently: ``>`` truncates its file (or creates it with permissions ``0644`` if it doesn't exist), ``>>`` appends to its file (or creates it with ``0644`` if it doesn't exist).
3. When output redirection is present, the command's output must not appear on the terminal, only in the listed files.
4. If any listed file cannot be opened for writing, the shell must print ``cshell: unable to create file for writing`` and not execute the command.
5. Input and output redirection must work together (e.g. ``command < input.txt > output.txt >> log.txt``).

**Example**

```

```
<goatam@iiit:~> echo hi > out1.txt > out2.txt
```

```
<goatam@iiit:~> cat out1.txt
```

```
hi
```

```
<goatam@iiit:~> cat out2.txt
```

```
hi
```

```
<goatam@iiit:~> echo again >> out1.txt > out3.txt
```

```
<goatam@iiit:~> cat out1.txt
```

```
hi
```

```
again
```

```
<goatam@iiit:~> cat out3.txt
```

again i'm confused about the fact that every output redirection has to keep its own mode. like with `echo hi > a.txt >> b.txt > c.txt`, i need a.txt to be truncated, b.txt to be opened with append, and c.txt to be truncated, but all three still need exactly the same output. should i open all the files first and store their fds plus their individual flags, and then have the writer process just write the same buffer to each fd? or is there some cleaner way to do this?

```



Yes — your first idea is basically the right one, with one important distinction:

**You don't want a “writer process” for output redirection.** The command itself should write its output, and your shell needs to arrange for that output to be duplicated to every redirection file.

The cleanest way is to create **one pipe for the command's stdout**, then have a **fan-out process** read from that pipe and write every byte to every output FD.

For:

```
echo hi > a.txt >> b.txt > c.txt
```

you want:

```
 ┌─> a.txt (truncate)
 └─> b.txt (append)
echo ──> stdout ──> pipe ┤─> c.txt (truncate)
 └─> terminal? NO
```

## Why you can't just `dup2()` all three

If you do:

```
dup2(fd_a, STDOUT_FILENO);
dup2(fd_b, STDOUT_FILENO);
dup2(fd_c, STDOUT_FILENO);
```

you end up with:

```
stdout → c.txt
```

The previous destinations are replaced.

So unlike normal Unix redirection, your assignment's requirement is **broadcast semantics**, not "last redirection wins."

---

## Your opening idea is exactly right

Before executing the command, open **every output file** with its own flags:

```
> a.txt → O_WRONLY | O_CREAT | O_TRUNC
>> b.txt → O_WRONLY | O_CREAT | O_APPEND
> c.txt → O_WRONLY | O_CREAT | O_TRUNC
```

with:

```
mode_t mode = 0644;
```

So your internal representation could simply be:

```
typedef struct {
 int fd;
```

```
bool append;
} Output;
```

Although once you've opened the file, you don't actually need to retain `append`; the FD already has its open status. So you could just store:

```
typedef struct {
 int fd;
} Output;
```

if you only need the FDs during execution.

---

## And validate ALL of them before executing

This is important, just like C2.

For:

```
echo hi > a.txt >> b.txt > inaccessible/c.txt
```

you should:

```
open a.txt ✓
open b.txt ✓
open c.txt x
 ↓
close a.txt
close b.txt
 ↓
print:
cshell: unable to create file for writing
 ↓
DO NOT execute echo
```

Don't start `echo` and discover the third output later.

---

## Then the fan-out

Once all files are successfully opened:

```
command
|
| stdout
|
↓
pipe
|
|
↓
fan-out process
├─ write → a.txt
├─ write → b.txt
└─ write → c.txt
```

The fan-out process essentially does:

```
char buffer[4096];

while ((n = read(pipe_read, buffer, sizeof buffer)) > 0) {
 for (each output fd) {
 write_all(output_fd, buffer, n);
 }
}
```

So if the command produces:

```
hello\n
```

the fan-out writes that same byte sequence to every output FD.

This is much more general than trying to capture the command's output into a giant buffer.

---

### One subtle but important issue: partial `write()`

Don't assume:

```
write(fd, buffer, n) == n
```

You should have a helper that keeps writing until all `n` bytes have been written:

```
write_all(fd, buffer, n)
```

because `write()` is allowed to write fewer bytes than requested.

Then:

```
for each output:
 write_all(fd, buffer, n)
```

---

## What happens to stdout?

When output redirection exists, the command's stdout should **not** go to the terminal.

So the command child does:

```
dup2(pipe_write, STDOUT_FILENO)
```

rather than:

```
dup2(output_fd, STDOUT_FILENO)
```

Then:

```
command stdout
 ↓
 pipe
```

```
↓
fan-out
↓
files only
```

No terminal output.

## Your architecture could therefore be

```
parent
|
├─ validate/open ALL input files
|
├─ validate/open ALL output files
|
├─ create input pipe if needed
|
├─ create output pipe if needed
|
├─ fork feeder if multiple inputs
|
├─ fork output fan-out if outputs exist
|
└─ fork command
 |
 ├─ stdin → input stream
 ├─ stdout → output pipe
 └─ execv()
```

That's a bit more process-heavy, but it maps beautifully onto the requirements.

## And the modes remain completely independent

For:

```
echo hi > a.txt >> b.txt > c.txt
```

you open:

```
a.txt → O_WRONLY | O_CREAT | O_TRUNC
b.txt → O_WRONLY | O_CREAT | O_APPEND
c.txt → O_WRONLY | O_CREAT | O_TRUNC
```

The fan-out process doesn't care **why** each FD was opened. It simply broadcasts every byte to each FD.

That's actually a very clean separation:

```
redirection parser
↓
"these are my output destinations"
↓
```

```
open each with its specified mode
```

```
↓
```

```
fan-out layer
```

```
↓
```

```
write identical stream to every destination
```

So yes: **open all files first, retain their FDs, then broadcast the command's stdout to all of them.**

The key correction is that the command's output should flow through a pipe to a fan-out process; you don't need to make the command itself know about all the output files.



bro i'm trying to deal with the failure case and i'm not sure when i should actually start the command. if i have `echo hello > good.txt > missing\_dir/bad.txt > another.txt`, the requirement says echo shouldn't execute at all if any output can't be opened. so do i need to pre-open every output file before forking the command, and if one fails, close everything i've already opened and abort? i'm worried that if i set up the command first and only discover the bad file afterwards, i've already violated the requirement."



YES 🤖 — **exactly the same principle as C2 input redirection.** You need to validate/open **every output file before the command is allowed to execute.**

For:

```
echo hello > good.txt > missing_dir/bad.txt > another.txt
```

the order should be:

```
parse
↓
open good.txt
✓
↓
open missing_dir/bad.txt
✗
↓
close good.txt
↓
print:
cshell: unable to create file for writing
↓
return to prompt
```

**No `fork()`, no `echo`, no output pipe, nothing.**

## Why this matters

If you did:

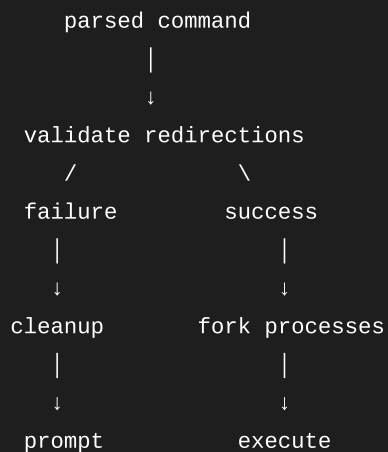
```
fork echo
↓
echo starts running
↓
set up good.txt
↓
try missing_dir/bad.txt
```

↓  
failure

you've already violated:

*If any listed file cannot be opened for writing, ... **not execute the command.***

So I'd make your execution pipeline explicitly have a **validation/preparation phase** before process creation.



## For output redirections

Something like:

```
for (int i = 0; i < output_count; i++) {
 int flags = O_WRONLY | O_CREAT;

 if (outputs[i].append)
 flags |= O_APPEND;
 else
 flags |= O_TRUNC;

 int fd = open(outputs[i].filename, flags, 0644);

 if (fd == -1) {
 printf("cshell: unable to create file for writing\n");

 // close every fd successfully opened so far
 for (int j = 0; j < i; j++)
 close(outputs[j].fd);

 return;
 }

 outputs[i].fd = fd;
}
```

At the end, you have a guarantee:

```
output 1 → valid FD
output 2 → valid FD
output 3 → valid FD
...
```

**Only then** do you create pipes/fork the command.

## And importantly: opening itself has the required side effect

There's one thing to be aware of:

```
echo hello > good.txt > missing_dir/bad.txt
```

If `good.txt` doesn't already exist, your `open(... O_CREAT ...)` will create it **before** you discover that the second output fails.

That's okay under the stated requirement. The requirement says the **command must not execute**; it doesn't say successfully opened/created files must be rolled back if a later redirection fails.

So you might end up with an empty `good.txt`, but `echo` must never run.

## Same architecture for C2 and C3

You're seeing a useful pattern emerging:

1. Parse entire command
2. Open/validate ALL redirection files
3. If anything fails:
  - close what was opened
  - print error
  - don't fork
4. Set up pipes/processes
5. fork
6. exec

That's a **really good boundary** to establish in your executor. It prevents exactly the partial-execution bugs you're worried about.



i'm trying to test the difference between `>` and `>>` properly and i'm not sure if just checking the final contents is enough. for example, if i do `echo one > a.txt` followed by `echo two >> a.txt`, i should get both, while another `echo three > a.txt` should wipe everything before it. is that basically enough to verify that i'm opening the files with `O\_TRUNC` and `O\_APPEND` correctly, or is there some edge case where the output could look right even though i'm using the wrong flags?



Yep — that sequence is a **good basic test**, but I'd add a few tests that specifically catch accidentally using the wrong flags.

Your core test:

```
echo one > a.txt
echo two >> a.txt
echo three > a.txt
```

should give:

```
three
```

because the final `>` truncates the file before writing.

A stronger test sequence is:

```
echo one > a.txt
cat a.txt
```

→ one

```
echo two >> a.txt
cat a.txt
```

→

```
one
two
```

```
echo three > a.txt
cat a.txt
```

→

```
three
```

That tests both behaviors independently.

## Test > on an existing file

This catches missing `O_TRUNC` :

```
printf 123456789 > a.txt
printf hi > a.txt
cat a.txt
```

Expected:

```
hi
```

If you accidentally use only:

```
O_WRONLY | O_CREAT
```

you could get:

```
hi3456789
```

which immediately exposes the bug.

## Test >> on an existing file

This catches accidentally using `O_TRUNC` :

```
printf abc > a.txt
printf XYZ >> a.txt
cat a.txt
```

Expected:

```
abcXYZ
```

If >> accidentally uses `O_TRUNC` , you'd get:

```
XYZ
```

## Test creation of a nonexistent file

Both operators need `O_CREAT` :

```
rm -f a.txt
echo hello > a.txt
cat a.txt
```

and separately:

```
rm -f b.txt
echo hello >> b.txt
```

```
cat b.txt
```

Both should create the file containing `hello`.

## A particularly useful test for your assignment

Since you have **multiple independent output redirections**, test:

```
echo hello > a.txt >> b.txt > c.txt
```

with all three files initially containing different garbage.

Afterward, all three should contain exactly:

```
hello
```

Then:

```
echo again >> b.txt
```

should make only `b.txt`:

```
hello
again
```

while `a.txt` and `c.txt` remain:

```
hello
```

That verifies that you're preserving the mode **per redirection**, rather than accidentally applying one mode to every output FD.

## One subtle edge case

Test `>` and `>>` against an **already existing empty file** too:

```
: > a.txt
echo hello >> a.txt
```

Expected:

```
hello
```

This isn't as useful for detecting truncation because the file starts empty, but it's useful for confirming that `>>` works on an existing file without unnecessarily truncating it.

So the most revealing test is probably:

```
printf '0123456789' > a.txt
printf 'XY' > a.txt
cat a.txt
```

```
XY

printf '0123456789' > a.txt
printf 'XY' >> a.txt
cat a.txt
0123456789XY
```

Those two tests make `O_TRUNC` vs `O_APPEND` almost impossible to fake accidentally.



i'm trying to write the code for multiple outputs and i'm stuck. if i have something like `echo hello > a.txt > b.txt >> c.txt`, can i first loop through all the redirection structs, open every file and store the returned fds in an array, then create a pipe for the command's stdout? what would the basic parent/child/writer process structure look like in code?



Yes — that's a good structure. The key is that **the parent opens/validates all output files first**, then creates the output pipe, then creates the command and fan-out writer processes.

For:

```
echo hello > a.txt > b.txt >> c.txt
```

you want:

```
graph LR
 echo -- stdout --> pipe
 pipe --> a["a.txt (truncate)"]
 pipe --> writer
 writer --> b["b.txt (truncate)"]
 writer --> c["c.txt (append)"]
```

## 1. Parent opens every output first

Conceptually:

```
typedef struct {
 char *filename;
 int fd;
 bool append;
} OutputRedir;
```

Then:

```
for (int i = 0; i < output_count; i++) {
 int flags = O_WRONLY | O_CREAT;

 if (outputs[i].append)
 flags |= O_APPEND;
 else
 flags |= O_TRUNC;

 outputs[i].fd = open(outputs[i].filename, flags, 0644);

 if (outputs[i].fd == -1) {
 // close all previously opened fds
 // print error
 }
}
```

```
 // return without executing
 }
}
```

At this point you have something like:

```
outputs[0].fd → a.txt
outputs[1].fd → b.txt
outputs[2].fd → c.txt
```

with their individual open modes already established.

---

## 2. Create the output pipe

```
int pipefd[2];

if (pipe(pipefd) == -1) {
 // cleanup
}
```

Conventionally:

```
pipefd[0] = read end
pipefd[1] = write end
```

Your command will write to `pipefd[1]`, while the writer reads from `pipefd[0]`.

---

## 3. Fork the command

```
pid_t cmd_pid = fork();

if (cmd_pid == 0) {
 // CHILD: command

 close(pipefd[0]);

 dup2(pipefd[1], STDOUT_FILENO);
 close(pipefd[1]);

 execv(command_path, argv);

 // exec failed
 _exit(126);
}
```

So the command sees:

```
stdout → output pipe
```

It doesn't know or care that there are three output files.

---

## 4. Fork the fan-out writer

The parent then creates the writer:

```
pid_t writer_pid = fork();

if (writer_pid == 0) {
 // CHILD: fan-out writer

 close(pipefd[1]);

 char buffer[4096];

 ssize_t n;

 while ((n = read(pipefd[0], buffer, sizeof(buffer))) > 0) {
 for (int i = 0; i < output_count; i++) {
 write_all(outputs[i].fd, buffer, n);
 }
 }

 close(pipefd[0]);

 for (int i = 0; i < output_count; i++)
 close(outputs[i].fd);

 _exit(0);
}
```

The important bit is:

```
for (int i = 0; i < output_count; i++)
 write_all(outputs[i].fd, buffer, n);
```

Every chunk coming from the command gets written to **every output FD**.

---

## 5. Parent closes everything it doesn't need

After both children have been created, the parent should close:

```
close(pipefd[0]);
close(pipefd[1]);

for (int i = 0; i < output_count; i++)
 close(outputs[i].fd);
```

The parent shouldn't retain the output file descriptors or the pipe ends.

Then:

```
waitpid(cmd_pid, NULL, 0);
waitpid(writer_pid, NULL, 0);
```

## The resulting FD ownership

This is the part I'd keep very clear in your head.

### Command child

```
pipe WRITE
 ↓
STDOUT_FILENO
```

It has **no output file FDs**.

### Writer child

```
pipe READ
 ↓
a.txt FD
b.txt FD
c.txt FD
```

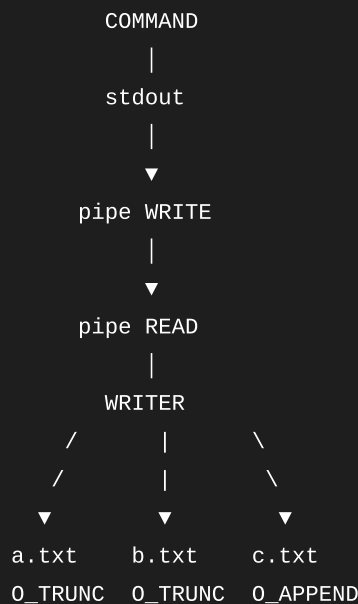
It owns all the output files.

### Parent

none of those FDs

It just waits.

So:



## One important ordering detail

I'd actually fork the **writer before the command**:

```
open all outputs
 ↓
create pipe
 ↓
fork writer
 ↓
fork command
 ↓
parent closes everything
 ↓
wait
```

That way the reader is ready before the command can produce a lot of output. Otherwise, theoretically, the command could fill the pipe before your writer exists and block.

The writer and command then run concurrently.

---

## And your validation boundary remains intact

Your whole C3 flow becomes:

```
parse
 ↓
open ALL output files
 |
 |— any failure → close opened files → error → prompt
 |
 |— all succeed
 ↓
 pipe()
 ↓
 fork writer
 ↓
 fork command
 ↓
 command writes stdout
 ↓
 writer broadcasts to every file
 ↓
 EOF
 ↓
 wait for both
 ↓
 prompt
```

One caveat: if you later combine this with **input redirection and pipes**, don't let this C3-specific process arrangement harden into your entire executor architecture. The same "command + helper process" idea will

interact with pipeline processes. But for **multiple output redirections alone**, this is a very reasonable implementation.



bro i'm confused about the exact order of `pipe()` and `fork()`. should i create all `n-1` pipes first and then run a loop that forks all the children, or should i create one pipe, fork the corresponding child, then create the next pipe? it seems like creating all of them first makes the indexing much easier, but i'm worried that having all the pipe fds open while forking every child could cause some weird fd inheritance problem.



Bro, **creating all `n-1` pipes first and then forking all `n` children is actually the cleaner approach** for a normal pipeline. Your concern about FD inheritance is real, but it's not a reason to avoid it — you just have to close the unused descriptors in **each child**.

For:

```
A | B | C
```

create:

```
pipe[0] = A → B
pipe[1] = B → C
```

**before any forks.**

Then:

```
Parent initially:
 pipe0: read/write
 pipe1: read/write

fork A
 A inherits both pipes
 parent still has both

fork B
 B inherits both pipes
 parent still has both

fork C
 C inherits both pipes
 parent still has both
```

That's okay because each child immediately closes everything it doesn't need.

**Child A**

Needs:

```
pipe0 write
```

So:

```
dup2(pipes[0][1], STDOUT_FILENO);
```

and closes **all** pipe FDs afterward.

## Child B

Needs:

```
pipe0 read
pipe1 write
```

So:

```
dup2(pipes[0][0], STDIN_FILENO);
dup2(pipes[1][1], STDOUT_FILENO);
```

then closes all pipe FDs.

## Child C

Needs:

```
pipe1 read
```

So:

```
dup2(pipes[1][0], STDIN_FILENO);
```

then closes all pipe FDs.

## Parent

After **all children have been forked**, close every pipe FD:

```
for (int i = 0; i < n - 1; i++) {
 close(pipes[i][0]);
 close(pipes[i][1]);
}
```

Then wait for the children.

---

## Why closing them matters

This is exactly the EOF problem you were worrying about earlier.

Suppose `c` is reading from `pipe1`.

If the parent accidentally keeps:

```
pipe1 write end
```

open, then even after **B** exits, **C** may never see EOF.

Likewise, if **A** accidentally keeps a read end open, that's usually harmless for EOF on the write side, but it's still an unnecessary FD and can cause other problems.

So the standard pattern is:

1. Create ALL pipes
2. Fork ALL children
3. In each child:
  - dup2() the 1-2 FDs it needs
  - close ALL original pipe FDs
  - exec()
4. Parent:
  - close ALL pipe FDs
  - wait for children

## Why I prefer this over "pipe → fork → pipe → fork"

If you did:

```
pipe 0
fork A
pipe 1
fork B
```

then **A** wouldn't inherit **pipe1**, which is actually convenient in some respects.

But now your FD state depends on **when** each pipe was created. With larger pipelines, the logic becomes more complicated.

Creating everything first gives you a simple invariant:

```
pipes[i][0] = read end of pipe i
pipes[i][1] = write end of pipe i
```

and child **i** simply needs:

```
stdin:
 pipes[i-1][0] if i > 0

stdout:
 pipes[i][1] if i < n-1
```

That's beautifully regular.

For **A | B | C | D**:

```
pipes[0] A → B
pipes[1] B → C
pipes[2] C → D

child 0:
 stdout = pipes[0][1]

child 1:
 stdin = pipes[0][0]
 stdout = pipes[1][1]

child 2:
 stdin = pipes[1][0]
 stdout = pipes[2][1]

child 3:
 stdin = pipes[2][0]
```

Then every child closes the **entire** `pipes` **array** after its `dup2()` calls.

So your instinct is right: **create all the pipes first. The inheritance is intentional; just aggressively close unused copies after each fork.**



### ### C4: Command Piping [18]

**\*\*Syntax\*\*:** ``command1 | command2 | ... | commandN``

**\*\*Purpose\*\*:** The pipe operator connects one command's output to the next command's input.

**\*\*Requirements\*\***

1. The shell must create pipes using ``pipe()`` for each ``|`` in the command.
2. For each command in the pipeline, the shell must fork a child process.
3. The shell must redirect just the standard output of ``command[i]`` to the write end of ``pipe[i]``.
4. The shell must redirect the standard input of ``command[i+1]`` to the read end of ``pipe[i]``.
5. The parent shell must close every pipe file descriptor it holds after forking.
6. Each child must close any pipe file descriptors it does not use before executing its command.
7. The parent shell must wait for all commands in the pipeline to complete before continuing.
8. If a command in the pipeline fails to execute, the shell prints ``cshell: command not found (commandname)`` for that stage, and the rest of the pipeline still runs.
9. File redirection and pipes must work together (e.g. ``command1 < input.txt | command2 > output.txt``).

**\*\*Example\*\***

```
```\n<goatam@iiit:~> printf "banana\\napple" | sort\napple\nbanana\n<goatam@iiit:~> badcmd | sort\ncshell: command not found (badcmd)\n<goatam@iiit:~> cat < in.txt | sort > out.txt\n```\n
```

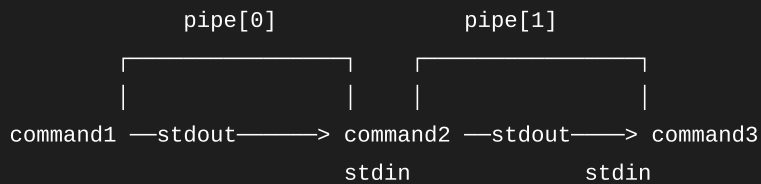


Yep — C4 is basically the part where your earlier `pipe()` / `fork()` reasoning all comes together. The key is to **treat each pipeline command as an independent child**, with the parent creating all pipes first.

For:

```
command1 | command2 | command3
```

you have:



The clean implementation structure

1. Resolve/validate everything you can before forking

You should first parse the pipeline into:

```

commands[0]
commands[1]
...
commands[n-1]

```

For each command, resolve its executable path according to C1.

But there's an important C4 requirement:

If a command in the pipeline fails to execute, the rest of the pipeline still runs.

So **don't abort the whole pipeline just because** `resolve_command()` **fails for one stage.**

For:

```
badcmd | sort
```

you can represent:

```

command 0 → path = NULL
command 1 → path = /usr/bin/sort

```

Then you still fork both children.

2. Create all pipes first

For `N` commands:

```
N - 1 pipes
```

So:

```
int pipes[N - 1][2];
```

and:

```

for (int i = 0; i < N - 1; i++) {
    if (pipe(pipes[i]) == -1) {
        // handle shell-level pipe creation failure
    }
}

```

```
}  
}
```

For:

A | B | C

you get:

```
pipes[0] = A → B  
pipes[1] = B → C
```

3. Fork every command

Then:

```
for (int i = 0; i < N; i++) {  
    pid_t pid = fork();  
  
    if (pid == 0) {  
        // child i  
    }  
  
    // parent stores pid  
}
```

Each child needs at most **two** pipe ends:

```
command i:  
  
stdin:  
    pipes[i-1][0]    if i > 0  
  
stdout:  
    pipes[i][1]      if i < N-1
```

So the child setup is beautifully regular:

```
if (i > 0) {  
    dup2(pipes[i - 1][0], STDIN_FILENO);  
}  
  
if (i < N - 1) {  
    dup2(pipes[i][1], STDOUT_FILENO);  
}
```

Then:

```
for (int j = 0; j < N - 1; j++) {  
    close(pipes[j][0]);  
}
```

```
    close(pipes[j][1]);  
}
```

Close all of them, even the ones you just used with `dup2()`.

That's because `dup2()` has already created the desired `stdin` / `stdout` descriptor.

Then the child handles its own redirections and executes.

The redirection interaction is important

For:

```
cat < in.txt | sort > out.txt
```

you have:

```
cat  
├─ stdin ← in.txt  
└─ stdout → pipe  
  
sort  
├─ stdin ← pipe  
└─ stdout → out.txt
```

So conceptually the child setup is:

`cat` **child**

```
input redirection  
  ↓  
stdin ← in.txt  
  
pipe setup  
  ↓  
stdout → pipe[0] write
```

`sort` **child**

```
pipe setup  
  ↓  
stdin ← pipe[0] read  
  
output redirection  
  ↓  
stdout → out.txt
```

The important thing is that **redirections and pipes both ultimately manipulate stdin/stdout**.

I'd define a clear precedence/order for your implementation based on the grammar/spec, but structurally you want each child to end up with its **final** stdin/stdout before `execv()`.

4. What happens with `badcmd | sort`?

This requirement is slightly different from ordinary shell behavior.

You need:

```
badcmd child
  ↓
cannot resolve/execute
  ↓
print:
cshell: command not found (badcmd)
  ↓
exit

sort child
  ↓
still exists
  ↓
runs normally
```

So **the parent must not decide "badcmd failed, therefore don't fork sort."**

Both children are created.

The failed child can do:

```
if (path == NULL) {
    dprintf(STDERR_FILENO,
           "cshell: command not found (%s)\n",
           command_name);
    _exit(...);
}
```

while the other children continue normally.

This is also why the error should happen **inside the relevant child**, after its pipe/redirection setup, rather than making the parent abort the pipeline.

Parent cleanup

After all children have been forked:

```
for (int i = 0; i < N - 1; i++) {
    close(pipes[i][0]);
    close(pipes[i][1]);
}
```

This is **mandatory for EOF behavior**.

Then:

```
for (int i = 0; i < N; i++) {
    waitpid(pids[i], NULL, 0);
}
```

```
}
```

Only after **all** children finish does the shell print the next prompt.

One thing I'd strongly recommend architecturally

Don't make your pipeline executor responsible for parsing commands or resolving paths.

Something like:

```
parser
  ↓
Pipeline {
  commands[0]
  commands[1]
  ...
}
  ↓
executor
  ↓
  resolve each command
    ↓
  create N-1 pipes
    ↓
  fork N children
    ↓
  configure each child
    ↓
  exec
    ↓
  parent closes pipes + waits
```

And each child can conceptually run:

```
setup_input_redirection(command);
setup_output_redirection(command);

if (has_previous_pipe)
    dup2(previous_pipe_read, STDIN_FILENO);

if (has_next_pipe)
    dup2(next_pipe_write, STDOUT_FILENO);

close_all_pipe_fds();

execv(...);
```

But be careful about the ordering of redirection vs pipe setup. Since both can target stdin/stdout, you need to decide which should win based on the assignment's intended semantics. The example `command1 < input.txt | command2 > output.txt` clearly tells you that the first command's `<` supplies its stdin and its `|` supplies its stdout, while the second command's `|` supplies stdin and `>` supplies stdout.

So effectively:

```
command1:
    stdin  = input redirection
    stdout = pipe

middle command:
    stdin  = previous pipe
    stdout = next pipe

commandN:
    stdin  = previous pipe
    stdout = output redirection
```

That's the mental model I'd use while implementing C4.

And yes — your earlier decision to **create all pipes first, then fork all children, then close unused FDs aggressively** is exactly the right pattern for this section.



for the actual command execution inside each pipeline child, should i reuse my existing C1 execution function completely, or do i need a separate pipeline-specific version? because C1 normally forks a child before doing ``execv()``, but in C4 i've already forked the child for the pipeline stage. if i call my existing executor from there and it forks again, i'll end up with an extra process for every stage. what's the cleanest way to separate 'resolve command + exec' from 'fork a command' so the same code can work in both cases?



Exactly — **don't reuse a function whose responsibility is "fork + execute" inside a pipeline child.** You'll get the unnecessary double-fork you noticed.

The clean solution is to split C1 into two layers:

```
resolve_command()
    ↓
actual executable path
    ↓
exec_command()
    ↓
execv()
```

and have a separate wrapper that handles `fork()` when you actually need a standalone command.

Layer 1: resolution

This should be completely independent of processes:

```
char *resolve_command(const char *name, ...);
```

It answers:

"Which executable should run for this command?"

For example:

```
echo      → /usr/bin/echo
%echo     → /usr/bin/echo
./foo     → ./foo
badcmd    → NULL
```

No `fork()`. No `execv()`.

Layer 2: execution in the current process

Then have something like:

```

void exec_resolved_command(const char *path, char **argv)
{
    execv(path, argv);

    // only returns on failure
    perror("cshell");
    _exit(126);
}

```

This function assumes:

I am already the process that should become the command.

Therefore it does **not** fork.

Layer 3: standalone C1 wrapper

Your ordinary command execution can then be:

```

void run_command(Command *cmd)
{
    char *path = resolve_command(cmd->argv[0], ...);

    if (path == NULL) {
        printf("cshell: command not found (%s)\n", ...);
        return;
    }

    pid_t pid = fork();

    if (pid == 0) {
        exec_resolved_command(path, cmd->argv);
    }

    waitpid(pid, NULL, 0);
    free(path);
}

```

So C1 gets:

```

parent shell
  |
  resolve
  |
  fork
 /  \
parent child
      |
      execv

```

Pipeline uses the exact same execution layer

For:

```
echo hello | sort
```

you've **already** done:

```
pid_t pid = fork();
```

So inside the child:

```
if (pid == 0) {
    setup_pipeline_fds(...);
    setup_redirections(...);

    exec_resolved_command(path, cmd->argv);
}
```

No second fork.

So C4 becomes:

```
parent
|
|— fork stage 0
|   |— child → setup → execv()
|
|— fork stage 1
|   |— child → setup → execv()
|
|— wait for all
```

instead of:

```
parent
|
|— fork pipeline child
|   |— run_command()
|       |— fork AGAIN ← bad
|           |— exec
```

This separation will help a LOT later

I'd think of your executor as three fundamentally different operations:

```
resolve_command()
    "What should I run?"

exec_command()
    "I am already the child. Replace myself with it."
```

```
run_command()  
    "I'm the shell. Fork a child and make it exec."
```

Then:

| Situation | Function |
|---------------------------|--|
| Normal C1 command | <code>run_command()</code> |
| Pipeline stage | <code>exec_command()</code> after pipeline's <code>fork()</code> |
| Future background command | probably <code>fork()</code> + <code>exec_command()</code> |
| Redirection | setup before <code>exec_command()</code> |
| Pipe | setup before <code>exec_command()</code> |

That's a really good abstraction boundary because `execv()` **belongs to the process that is becoming the command, while** `fork()` **belongs to whatever execution context is creating that process.**

One more small thing: if `resolve_command()` returns an allocated path, make sure the child doesn't need the parent to free it before `execv()` —the child gets its own copy of the address space after `fork()`. You can simply free it in the parent after the relevant `waitpid()` and in the child after `execv()` failure if needed.



i'm trying to decide how much of this should actually live in ``exec.c``. would it make sense to have something like ``execute_pipeline(commands, count)`` that creates the pipes, forks the stages and handles all the fd plumbing, while each child eventually calls a smaller ``execute_command()`` function that just resolves the executable and calls ``execv()``? because if i keep putting pipe logic, redirection logic and command lookup into one giant function, i'm pretty sure i'm going to lose track of which process owns which fd.



Yes. **That's exactly the separation I'd recommend**, especially given the assignment's "monolithic code will be penalized" warning.

I'd make `exec.c` coordinate execution, but split responsibilities into small functions.

A good structure

```
exec.c
|
├─ execute_command(...)      ← standalone command: fork + wait
|
├─ execute_pipeline(...)     ← N commands: pipes + forks + waits
|
├─ resolve_command(...)      ← lookup only
|
└─ exec_resolved(...)        ← child is already forked → execv()
```

Then separately, if the project is getting larger:

```
redirect.c
├─ prepare_input_redirection(...)
├─ prepare_output_redirection(...)
└─ ...
```

```
pipe.c
├─ create_pipeline_pipes(...)
├─ setup_child_pipes(...)
└─ ...
```

You don't *have* to make `pipe.c` immediately; the important thing is the **responsibility boundaries**.

The key distinction

I'd think of these three functions like this:

```
execute_command()
    "I am the shell. Create a process to run this command."
```

```
execute_pipeline()
    "I am the shell. Create N processes and connect them."

exec_resolved()
    "I am already a child. Become this executable."
```

So standalone C1:

```
execute_command(cmd)
    |
    |— resolve_command()
    |
    |— fork()
    |
    |— child
        |— exec_resolved()
```

Pipeline C4:

```
execute_pipeline(commands, n)
    |
    |— resolve each command
    |— create n-1 pipes
    |
    |— fork child 0
        |— setup FDs
        |   exec_resolved()
    |
    |— fork child 1
        |— setup FDs
        |   exec_resolved()
    |
    |— ...
    |
    |— parent closes pipes + waits
```

No pipeline child ever calls `execute_command()`, because that function would fork again.

I'd also separate FD setup from execution

This is probably the biggest thing that will save you headaches.

Instead of:

```
execute_pipeline()
{
    // 200 lines of:
    // pipes
    // dup2
    // redirections
    // close
    // fork
```

```

    // lookup
    // exec
}

```

make the child path conceptually:

```

child_process(command, pipeline_context)
{
    setup_pipeline_fds(...);
    setup_redirections(...);
    close_unused_fds(...);

    exec_resolved(...);
}

```

Then `execute_pipeline()` only has to worry about **process-level orchestration**:

```

create pipes
  ↓
fork
  ↓
child setup
  ↓
fork
  ↓
child setup
  ↓
...
  ↓
parent closes
  ↓
wait

```

This makes FD ownership much easier to reason about.

For example

Your pipeline loop could conceptually be:

```

for (int i = 0; i < count; i++) {
    pid = fork();

    if (pid == 0) {
        setup_pipeline_fds(pipes, count, i);
        setup_redirections(&commands[i]);

        close_all_pipes(pipes, count);

        exec_resolved(commands[i].path, commands[i].argv);
    }
}

```

```

    pids[i] = pid;
}

```

And `setup_pipeline_fds()` has exactly one job:

```

stage i
  ↓
if i > 0:
    stdin ← pipes[i-1][0]

if i < count-1:
    stdout ← pipes[i][1]

```

That's much easier to verify against C4's requirements.

One thing I'd change from your wording

You said:

each child eventually calls a smaller `execute_command()` function that just resolves the executable and calls `execv()`

I'd actually **not resolve inside the child if you can avoid it**.

Prefer:

```

parent:
    resolve command
    ↓
    store resolved path
    ↓
    fork
    ↓
child:
    setup FDs
    execv(resolved_path)

```

That keeps command lookup as parent-side preparation and makes the child almost entirely:

```

dup2
close
exec

```

It also means a lookup failure doesn't require weird interaction with pipeline setup.

For C4, though, remember that a failed lookup **mustn't prevent the other pipeline stages from being forked**. So you can store:

```

commands[i].resolved_path = NULL;

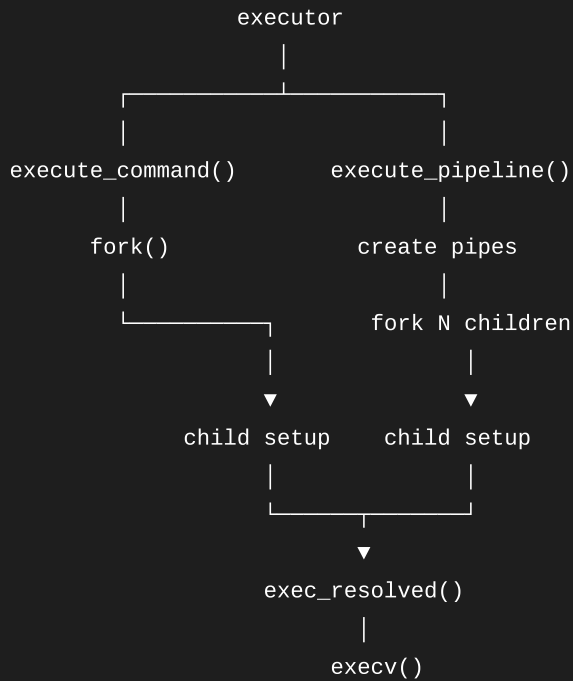
```

for a bad stage, and that particular child prints:

```
cshell: command not found (badcmd)
```

and exits, while the other stages continue.

So I'd aim for something like this



And **parser never needs to know about FDs, `fork()`, or `execv()`**.

That's a very reasonable level of modularity for this assignment: not absurdly fragmented into 30 files, but also not one `exec.c` containing the entire shell.